

# **Arab Academy for Science, Technology, and Maritime Transport**

College of Computing and Information Technology — Cairo  
Department of Cybersecurity — Information Systems

B.Sc. Final Year Project

---

## **LuminaFPM**

**A Proactive, Read-Only, Multi-Vendor  
Firewall Anomaly Management Platform**

---

### **Presented By:**

Youssef Kandeel   Youssef Hazem   Yassin Osama  
Ali Hesham   Hamza Ahmed Selim

### **Supervised By:**

Dr. Mohamed Elhamahmy

June 2026

# Declaration

We hereby certify that this material, which we now submit for assessment on the program of study leading to the award of Bachelor of Science in Cybersecurity, is entirely our own work, that we have exercised reasonable care to ensure that the work is original, and does not to the best of our knowledge breach any law of copyright, and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of our work.

**Student Name:** Youssef Kandeel  
**Registration No.:** 221006460  
**Signed:** \_\_\_\_\_  
**Date:** \_\_\_\_\_

**Student Name:** Youssef Hazem  
**Registration No.:** 221006612  
**Signed:** \_\_\_\_\_  
**Date:** \_\_\_\_\_

**Student Name:** Yassin Osama  
**Registration No.:** 221005138  
**Signed:** \_\_\_\_\_  
**Date:** \_\_\_\_\_

**Student Name:** Ali Hesham  
**Registration No.:** 221005580  
**Signed:** \_\_\_\_\_  
**Date:** \_\_\_\_\_

**Student Name:** Hamza Ahmed Selim  
**Registration No.:** 221004258  
**Signed:** \_\_\_\_\_  
**Date:** \_\_\_\_\_

# Acknowledgment

First and foremost, we are deeply thankful to Allah for granting us the strength, patience, and guidance throughout our academic journey and during the development of this graduation project.

We would like to express our sincere gratitude and appreciation to our supervisor, Dr. Mohamed Elhamahmy, for his invaluable guidance, continuous support, and constructive feedback throughout the development of this project. His expertise, academic insight, and dedication greatly contributed to shaping the technical direction of the work and enhancing its overall quality. We are especially thankful for his patience and encouragement, which helped us overcome challenges and maintain a clear vision across all phases of the project.

This project would not have been completed without his mentorship, and we are truly grateful for the opportunity to work under his supervision.

# Abstract

Modern enterprise networks are defended by large, long-lived, multi-vendor firewall rulebases that accumulate *rule bloat*: shadowed, redundant, correlated, generalized, and conflicting rules, overly broad permissions, missing logging or inspection, stale temporary entries, and inconsistent security posture across devices. As rulebases grow into the hundreds or thousands of rules and span heterogeneous platforms with incompatible configuration formats, security teams lose unified visibility and the ability to reason rigorously about what their policies actually permit. This project presents **LuminaFPM**, a strictly read-only, multi-vendor Firewall Policy and Anomaly Management Platform that acquires firewall policies from Fortinet FortiGate (via the FortiOS REST/JSON API) and Palo Alto Networks PAN-OS (via the XML API), normalizes them into a single canonical policy model through a dedicated parsing, normalization, and object/service correlation layer, and analyzes them with a *deterministic, explainable* anomaly-detection engine. The engine is grounded in a set-theoretic model in which each firewall rule is a tuple of sets over source, destination, service, and action; shadowing is detected as a subset/superset covering relation, redundancy as set equivalence, and conflict as overlapping match with divergent action. Operating only on normalized data, the engine never inspects raw vendor configuration and remains repeatable and benchmarkable rather than statistical or machine-learned. On top of detection, LuminaFPM adds a separate risk-scoring engine, API-based cyber-threat-intelligence (CTI) enrichment, and an AI-assisted, evidence-grounded subsystem that generates SOC and executive reports without ever serving as the anomaly authority. Cross-vendor inconsistency detection surfaces divergent security posture between devices. The platform is built on FastAPI, SQLAlchemy, PostgreSQL, Celery, and Redis with a React/TypeScript/Vite frontend, orchestrated through Docker Compose, and uses Google Gemini as the default large-language-model provider with a deterministic offline fallback. Validation against a live VMware lab containing a real FortiGate and Palo Alto firewall, scored by a ground-truth benchmark matrix, achieves **precision, recall, and F1 of 1.0** on the configuration-detectable anomaly taxonomy.

# Contents

|                                             |            |
|---------------------------------------------|------------|
| <b>Declaration</b>                          | <b>i</b>   |
| <b>Acknowledgment</b>                       | <b>ii</b>  |
| <b>Abstract</b>                             | <b>iii</b> |
| <b>List of Figures</b>                      | <b>x</b>   |
| <b>List of Tables</b>                       | <b>xi</b>  |
| <b>List of Acronyms and Abbreviations</b>   | <b>xii</b> |
| <b>1 Introduction</b>                       | <b>1</b>   |
| 1.1 Purpose . . . . .                       | 1          |
| 1.2 Intended Audience . . . . .             | 2          |
| 1.3 Intended Use . . . . .                  | 3          |
| 1.4 Scope . . . . .                         | 3          |
| 1.4.1 Problem Statement . . . . .           | 4          |
| 1.4.2 Objectives . . . . .                  | 5          |
| 1.4.3 Goals . . . . .                       | 6          |
| 1.4.4 Included and Excluded Scope . . . . . | 6          |
| 1.5 Definitions . . . . .                   | 8          |
| 1.6 Standards Used . . . . .                | 10         |
| 1.6.1 Security standards . . . . .          | 10         |
| 1.6.2 Network standards . . . . .           | 10         |
| 1.6.3 Programming standards . . . . .       | 10         |
| 1.6.4 Data-privacy standards . . . . .      | 10         |
| 1.7 Realistic Constraints . . . . .         | 11         |
| 1.8 SWOT Analysis . . . . .                 | 11         |
| 1.8.1 Strengths . . . . .                   | 12         |
| 1.8.2 Weaknesses . . . . .                  | 12         |
| 1.8.3 Opportunities . . . . .               | 12         |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 1.8.4    | Threats . . . . .                                                  | 12        |
| 1.9      | PESTLE Analysis . . . . .                                          | 12        |
| 1.9.1    | Political . . . . .                                                | 13        |
| 1.9.2    | Economic . . . . .                                                 | 13        |
| 1.9.3    | Social . . . . .                                                   | 13        |
| 1.9.4    | Technological . . . . .                                            | 13        |
| 1.9.5    | Legal . . . . .                                                    | 14        |
| 1.9.6    | Environmental . . . . .                                            | 14        |
| <b>2</b> | <b>Literature Review</b>                                           | <b>15</b> |
| 2.1      | Background . . . . .                                               | 15        |
| 2.1.1    | Firewall Policy Management . . . . .                               | 15        |
| 2.1.2    | The Firewall Policy-Anomaly Taxonomy . . . . .                     | 16        |
| 2.1.3    | Normalisation of Heterogeneous Vendor Policies . . . . .           | 17        |
| 2.1.4    | Deterministic and Formal Methods versus Machine Learning . . . . . | 17        |
| 2.1.5    | CTI and Policy-Derived Exposure . . . . .                          | 18        |
| 2.2      | Market Analysis . . . . .                                          | 19        |
| 2.3      | Related Projects and Similar Systems . . . . .                     | 19        |
| 2.3.1    | Tufin Orchestration Suite . . . . .                                | 19        |
| 2.3.2    | AlgoSec Security Management Suite . . . . .                        | 20        |
| 2.3.3    | FireMon . . . . .                                                  | 20        |
| 2.3.4    | Skybox Security . . . . .                                          | 20        |
| 2.3.5    | Palo Alto Panorama . . . . .                                       | 21        |
| 2.3.6    | Comparative Feature Analysis . . . . .                             | 21        |
| 2.4      | Survey and User Insights . . . . .                                 | 22        |
| 2.5      | The Four P's of Marketing . . . . .                                | 23        |
| 2.5.1    | Product . . . . .                                                  | 23        |
| 2.5.2    | Price . . . . .                                                    | 24        |
| 2.5.3    | Place . . . . .                                                    | 24        |
| 2.5.4    | Promotion . . . . .                                                | 24        |
| 2.6      | Research Gaps . . . . .                                            | 24        |
| 2.7      | Summary . . . . .                                                  | 25        |
| <b>3</b> | <b>System Analysis and Design</b>                                  | <b>26</b> |
| 3.1      | Overview . . . . .                                                 | 26        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 3.2      | Methodology . . . . .                                             | 27        |
| 3.3      | User Requirements and System Privileges . . . . .                 | 30        |
| 3.3.1    | Security Analyst (Firewall Engineer / SOC Analyst) . . . . .      | 30        |
| 3.3.2    | SOC Lead / Administrator (Platform Admin) . . . . .               | 31        |
| 3.3.3    | Auditor / Viewer . . . . .                                        | 31        |
| 3.4      | Assumptions, Constraints, Limitations, and Dependencies . . . . . | 32        |
| 3.4.1    | Assumptions . . . . .                                             | 32        |
| 3.4.2    | Constraints . . . . .                                             | 32        |
| 3.4.3    | Limitations . . . . .                                             | 32        |
| 3.4.4    | Dependencies . . . . .                                            | 33        |
| 3.5      | Intended Technologies . . . . .                                   | 33        |
| 3.6      | Functional Requirements . . . . .                                 | 34        |
| 3.7      | Non-Functional Requirements . . . . .                             | 37        |
| 3.7.1    | Security . . . . .                                                | 37        |
| 3.7.2    | Reliability and Availability . . . . .                            | 37        |
| 3.7.3    | Performance . . . . .                                             | 37        |
| 3.7.4    | Maintainability and Extensibility . . . . .                       | 37        |
| 3.7.5    | Usability . . . . .                                               | 38        |
| 3.7.6    | Portability . . . . .                                             | 38        |
| 3.8      | Virtual Environment and Laboratory Topology . . . . .             | 39        |
| 3.9      | System Architecture . . . . .                                     | 40        |
| 3.10     | Use Case Diagram . . . . .                                        | 42        |
| 3.11     | Misuse Case Diagram . . . . .                                     | 44        |
| 3.12     | Context Diagram . . . . .                                         | 46        |
| 3.13     | Data Flow Diagram . . . . .                                       | 47        |
| 3.14     | Entity Relationship Diagram . . . . .                             | 48        |
| 3.14.1   | Entity Descriptions . . . . .                                     | 48        |
| 3.14.2   | Relationship Summary . . . . .                                    | 49        |
| 3.15     | Class Diagram . . . . .                                           | 51        |
| 3.16     | Sequence Diagrams . . . . .                                       | 52        |
| <b>4</b> | <b>Implementation and Testing</b>                                 | <b>55</b> |
| 4.1      | Implementation Overview . . . . .                                 | 55        |
| 4.2      | Development Environment and Tools . . . . .                       | 56        |
| 4.3      | Project Structure . . . . .                                       | 57        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 4.4      | Acquisition Layer . . . . .                                         | 58        |
| 4.5      | Parsing and Normalization . . . . .                                 | 60        |
| 4.6      | Anomaly Detection Engine . . . . .                                  | 61        |
| 4.6.1    | The Set-Theoretic Conceptual Model . . . . .                        | 61        |
| 4.6.2    | The Deterministic Engine . . . . .                                  | 62        |
| 4.6.3    | Severity, Confidence, and Detection Mode . . . . .                  | 63        |
| 4.6.4    | Cross-Device Detectors . . . . .                                    | 64        |
| 4.6.5    | Anomaly Types and Policy-Rule Implementation . . . . .              | 64        |
| 4.7      | Risk Scoring . . . . .                                              | 66        |
| 4.8      | Threat Intelligence (CTI) . . . . .                                 | 67        |
| 4.9      | AI-Assisted Reporting . . . . .                                     | 68        |
| 4.10     | Backend API . . . . .                                               | 69        |
| 4.11     | Frontend Implementation . . . . .                                   | 70        |
| 4.12     | Settings, Scheduling, and Notifications . . . . .                   | 74        |
| 4.13     | Testing and Evaluation Strategy . . . . .                           | 75        |
| 4.14     | Unit Testing . . . . .                                              | 76        |
| 4.15     | Benchmark Evaluation (Acceptance Gate) . . . . .                    | 78        |
| 4.15.1   | Metrics: TP / FP / FN and the derived rates . . . . .               | 78        |
| 4.15.2   | Ground-truth construction . . . . .                                 | 79        |
| 4.15.3   | Deployment filtering and <code>config_only</code> scoping . . . . . | 79        |
| 4.16     | Test Cases and Outputs . . . . .                                    | 80        |
| 4.16.1   | Representative rule-level cases . . . . .                           | 80        |
| 4.16.2   | Aggregate run output (run #25) . . . . .                            | 81        |
| 4.16.3   | Benchmark scorecard (the acceptance gate) . . . . .                 | 82        |
| 4.16.4   | CTI test case . . . . .                                             | 83        |
| 4.16.5   | AI-report test case . . . . .                                       | 83        |
| 4.17     | Findings and Limitations . . . . .                                  | 84        |
| <b>5</b> | <b>Conclusion and Future Work</b>                                   | <b>86</b> |
| 5.1      | Conclusion . . . . .                                                | 86        |
| 5.1.1    | What was achieved . . . . .                                         | 86        |
| 5.1.2    | Empirical validation . . . . .                                      | 88        |
| 5.2      | Future Work . . . . .                                               | 89        |
| 5.2.1    | Interactive logical topology on Cytoscape.js . . . . .              | 89        |
| 5.2.2    | Real authentication and RBAC enforcement . . . . .                  | 89        |

|                   |                                                                    |           |
|-------------------|--------------------------------------------------------------------|-----------|
| 5.2.3             | Broadening the deterministic engine to the full taxonomy . . . . . | 89        |
| 5.2.4             | Activating the threat-intelligence and dark-web module . . . . .   | 90        |
| 5.2.5             | Additional vendors . . . . .                                       | 90        |
| 5.2.6             | Report export to PDF . . . . .                                     | 90        |
| 5.2.7             | Operator-supplied topology and asset context . . . . .             | 91        |
| 5.2.8             | Production hardening . . . . .                                     | 91        |
| <b>References</b> |                                                                    | <b>92</b> |

# List of Figures

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1  | The LuminaFPM end-to-end processing pipeline (nine logical layers). . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 28 |
| 3.2  | Parallel shared-network virtual laboratory topology. LuminaFPM polls both firewalls read-only over the management network; the firewalls independently govern the shared data-plane subnets. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 39 |
| 3.3  | Layered system architecture of LuminaFPM. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 41 |
| 3.4  | Use case diagram of LuminaFPM. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 43 |
| 3.5  | Misuse case diagram of LuminaFPM. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 44 |
| 3.6  | Context diagram: LuminaFPM and its external entities. All firewall interaction is strictly read-only. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 46 |
| 3.7  | Data flow diagram of the LuminaFPM analysis pipeline. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 47 |
| 3.8  | Entity-relationship diagram of LuminaFPM Schema v4. NavyBlue boxes are entities and MidBlue boxes are association (bridge) tables; gray boxes are standalone tables with no foreign key into the core graph. The diagram shows vendor/device ownership; the normalized policy model (the <code>policy_rule</code> ↔ <code>network_object</code> and <code>policy_rule</code> ↔ <code>service_object</code> bridges, and the canonical normalization layer reached through <code>object_/service_normalization_mapping</code> ); the analysis tier ( <code>rule_anomaly</code> with a self-referential <code>related_rule</code> , the <code>anomaly_execution_log</code> run that aggregates anomalies and <code>risk_assessment</code> ); the CTI <code>cti_indicator/cti_observation</code> axis (including the device firmware axis); the <code>benchmark_case/benchmark_result</code> pair; acquisition ( <code>acquisition_job</code> , <code>raw_artifact</code> , <code>rule_snapshot</code> ); and reporting ( <code>llm_report</code> ), with the standalone <code>schedule_config</code> and <code>notification</code> tables. Cardinalities are shown as 1 / * (and 0..1 for optional). Dashed edges mark the many-to-many correlations resolved by bridge tables. . . . . | 50 |
| 3.9  | Backend service and domain classes: connectors, parsers, normalization engine, anomaly engine and detectors, risk scorer, CTI providers/runner, LLM provider/reporter, benchmark scorer/runner. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 51 |
| 3.10 | Sequence diagram: end-to-end policy acquisition, normalization, anomaly analysis, and reporting. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 53 |
| 4.1  | Control flow of the deterministic anomaly-detection engine. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 63 |
| 4.2  | LuminaFPM dashboard (overview). . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 71 |
| 4.3  | LuminaFPM policy audit and rule inspector. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 72 |

|     |                                                                                                                                                                                                                                                                                                                        |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.4 | LuminaFPM AI-assisted SOC report viewer. . . . .                                                                                                                                                                                                                                                                       | 73 |
| 4.5 | LuminaFPM logical policy topology. . . . .                                                                                                                                                                                                                                                                             | 74 |
| 4.6 | The functional Settings console: the <i>Firewalls</i> tab manages the device<br>fleet (add, edit, delete, set/rotate the encrypted credential, test connection,<br>and acquire) while remaining strictly read-only toward the firewalls; the<br><i>Scheduling</i> and <i>Notifications</i> tabs sit alongside. . . . . | 75 |
| 4.7 | LuminaFPM benchmark center (acceptance gate). . . . .                                                                                                                                                                                                                                                                  | 82 |
| 4.8 | LuminaFPM risk posture screen. . . . .                                                                                                                                                                                                                                                                                 | 84 |
| 5.1 | Summary of the principal achievements delivered by LuminaFPM. . . . .                                                                                                                                                                                                                                                  | 88 |
| 5.2 | Future-work roadmap for LuminaFPM. . . . .                                                                                                                                                                                                                                                                             | 91 |

# List of Tables

|     |                                                                                                                                                                  |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Explicit in-scope capabilities for LuminaFPM v1. . . . .                                                                                                         | 6  |
| 1.2 | Explicit out-of-scope items and future work. . . . .                                                                                                             | 7  |
| 1.3 | Key terms and definitions. . . . .                                                                                                                               | 8  |
| 1.4 | SWOT summary. . . . .                                                                                                                                            | 13 |
| 2.1 | Comparative feature analysis of LuminaFPM versus representative commercial firewall policy-management systems. . . . .                                           | 21 |
| 2.2 | Summary of Term-1 survey dimensions and principal findings. . . . .                                                                                              | 23 |
| 3.1 | User categories, responsibilities, and platform privileges. . . . .                                                                                              | 31 |
| 3.2 | Intended technology stack by architectural layer. . . . .                                                                                                        | 33 |
| 3.3 | Functional requirements (FR-01 – FR-27). . . . .                                                                                                                 | 35 |
| 3.4 | Non-functional requirements summary. . . . .                                                                                                                     | 38 |
| 3.5 | Misuse cases, threat actors, attack vectors and implemented countermeasures. . . . .                                                                             | 45 |
| 3.6 | Core entities of the normalized Schema v4 and their salient attributes. . . . .                                                                                  | 48 |
| 4.1 | Core technology stack and tooling. . . . .                                                                                                                       | 56 |
| 4.2 | Implemented anomaly taxonomy: the config-detectable core actually emitted by the engine, versus the simulated/future remainder of the 46-type framework. . . . . | 65 |
| 4.3 | Principal LuminaFPM REST endpoints (acquisition, analysis, reporting, scheduling, and notifications). . . . .                                                    | 69 |
| 4.4 | Backend unit-test coverage by subsystem ( <b>backend/tests/</b> ). . . . .                                                                                       | 77 |
| 4.5 | Representative live rule-level test cases and the engine’s findings (run #25). . . . .                                                                           | 80 |
| 4.6 | Anomaly-type distribution and benchmark coverage, run #25. . . . .                                                                                               | 81 |

# List of Acronyms and Abbreviations

| Acronym | Definition                                                                     |
|---------|--------------------------------------------------------------------------------|
| FPM     | Firewall Policy Management (LuminaFPM, the platform presented in this project) |
| CTI     | Cyber Threat Intelligence                                                      |
| LLM     | Large Language Model                                                           |
| AI      | Artificial Intelligence                                                        |
| API     | Application Programming Interface                                              |
| REST    | Representational State Transfer                                                |
| JSON    | JavaScript Object Notation                                                     |
| XML     | Extensible Markup Language                                                     |
| ORM     | Object–Relational Mapping                                                      |
| SQL     | Structured Query Language                                                      |
| RBAC    | Role-Based Access Control                                                      |
| JWT     | JSON Web Token                                                                 |
| TLS     | Transport Layer Security                                                       |
| AES     | Advanced Encryption Standard                                                   |
| VM      | Virtual Machine                                                                |
| VDOM    | Virtual Domain (FortiGate administrative partition)                            |
| VSYS    | Virtual System (Palo Alto PAN-OS administrative partition)                     |
| CVE     | Common Vulnerabilities and Exposures                                           |
| NVD     | National Vulnerability Database                                                |
| KEV     | Known Exploited Vulnerabilities (CISA catalog)                                 |
| IOC     | Indicator of Compromise                                                        |
| SOC     | Security Operations Center                                                     |
| IDS     | Intrusion Detection System                                                     |
| FW      | Firewall                                                                       |
| ACL     | Access Control List                                                            |
| NAT     | Network Address Translation                                                    |
| DMZ     | Demilitarized Zone                                                             |
| CIDR    | Classless Inter-Domain Routing                                                 |
| RFC1918 | RFC 1918 Private Address Space                                                 |
| CRUD    | Create, Read, Update, Delete                                                   |
| SSE     | Server-Sent Events                                                             |
| ERD     | Entity–Relationship Diagram                                                    |
| DFD     | Data Flow Diagram                                                              |
| UML     | Unified Modeling Language                                                      |
| CI      | Continuous Integration                                                         |

# Chapter 1

## Introduction

The firewall remains the foundational arbiter of trust in enterprise network defense, governing which flows are permitted between protected internal segments and untrusted external networks. Yet the operational reality of administering these devices in heterogeneous, multi-vendor estates is one of accumulating entropy. Over years of operation, rulebases succumb to *rule bloat*: temporary access rules, migration exceptions, vendor on-boarding entries, emergency changes, and decommissioned business requirements remain in production long after their purpose has expired. The result is a security posture that is opaque on paper and quietly degraded in practice, riddled with shadowed, redundant, and conflicting rules that human auditors cannot feasibly reason about by hand — particularly when the same business intent is expressed differently by a Fortinet FortiGate and a Palo Alto Networks device.

**LuminaFPM** (full title *LuminaFPM — Proactive Multi-Vendor Firewall Policy Management*) is a strictly read-only, multi-vendor **Firewall Anomaly Management Platform** engineered to convert this disorder into normalized security intelligence. It acquires firewall configurations from different vendors, normalizes their heterogeneous rulebases into a single canonical model, and runs a *deterministic, explainable* anomaly-detection engine over that normalized model. Around this core it layers rule- and device-level risk scoring, API-based Cyber Threat Intelligence (CTI) enrichment, and AI-assisted, evidence-grounded Security Operations Centre (SOC) reporting. The platform’s canonical one-line delivery statement is unambiguous: “*LuminaFPM converts multi-vendor firewall configurations into normalized security intelligence. It identifies policy anomalies, cross-vendor inconsistencies, and risk-prioritized findings without modifying firewall devices.*” It never writes to a firewall.

This chapter establishes the platform’s purpose, audience, intended use, scope, and the formal vocabulary used throughout the report. It then enumerates the standards adhered to, the realistic constraints under which the system was built and validated in a controlled laboratory, and concludes with structured strategic analyses (SWOT and PESTLE) framed for both the Egyptian and global contexts.

### 1.1 Purpose

The *raison d’être* of LuminaFPM is to transform firewall policy management from a reactive, periodic, manual maintenance task into a continuous, deterministically verified

security operation. Its primary purpose is to provide firewall administrators and security architects with a unified platform that automates the auditing of complex, multi-vendor firewall policies and surfaces the “silent” configuration errors — shadowed rules that are syntactically valid yet logically unreachable, redundant rules that waste lookup cycles, and conflicting rules whose outcome depends entirely on evaluation order — that render security controls ineffective.

A second, deliberately subordinate purpose is to enrich this static configuration view with dynamic threat context. A rule that appears safe today becomes risky if a referenced IP address or fully-qualified domain name (FQDN) turns malicious, or if a Common Vulnerabilities and Exposures (CVE) entry emerges for the deployed firmware. LuminaFPM therefore couples its deterministic anomaly findings with a *separate* risk-scoring and CTI engine, keeping threat context distinct from — and never overriding — the deterministic anomaly authority.

Critically, the purpose is to *inform human decisions, not to enact changes*. The platform provides high-confidence visibility, analysis, prioritization, and reporting so that authorized firewall engineers and SOC analysts can make informed remediation decisions; the corrective action itself remains entirely under the control of authorized organizational personnel. This read-only posture is a defining, non-negotiable property of the system and is reflected in every design decision documented in this report.

## 1.2 Intended Audience

This report and the system it describes are aimed at a spectrum of stakeholders within the cybersecurity and IT-infrastructure domains. The platform’s own role-based access control (RBAC) model formalizes these audiences into five read-centric roles (Platform Admin, Firewall Engineer, SOC Analyst, Auditor, and Viewer), described in detail in the security chapters.

- **Firewall Administrators and Network Security Engineers.** The professionals responsible for the design, implementation, and maintenance of the network perimeter. They require granular visibility into how rules interact across different devices and vendors. For this audience the platform provides the normalized policy repository, the deterministic anomaly framework, and the parsing/normalization logic that resolves vendor-specific FortiGate REST/JSON and Palo Alto PAN-OS XML configurations into a single comparable model.
- **Security Operations Centre (SOC) Analysts.** Operational staff tasked with monitoring and investigating risk. They consume the risk views, the CTI Threat Center, and the AI-generated, evidence-grounded SOC and executive reports, using policy-derived blast-radius-style reasoning to understand the potential impact of a finding without relying on live packet telemetry.

- **Security Auditors and Compliance Managers.** Personnel responsible for demonstrating adherence to regulatory and best-practice frameworks (e.g. NIST SP 800-41 firewall guidance, least-privilege principles). LuminaFPM provides reproducible, benchmark-validated evidence that a rulebase has been examined for redundancy, conflict, and over-permissive access — a key requirement for technical compliance audits.
- **The Cybersecurity Development Team and Academic Reviewers.** Engineers extending the platform (adding connectors, parsers, or detectors) and the academic examiners assessing the project. For this audience the report documents the layered architecture, the data model, and the deterministic detection contract in sufficient depth to reproduce and extend the system.

## 1.3 Intended Use

LuminaFPM is engineered for deployment within complex, heterogeneous enterprise networks that follow a “best-of-breed” security strategy. It is designed to function as a centralized analysis overlay that abstracts the underlying complexities of disparate vendor operating systems, with an initial v1 focus on Palo Alto Networks (PAN-OS) and Fortinet (FortiOS), normalizing their proprietary XML and JSON configuration formats into a unified internal schema.

The system operates in a strictly **non-intrusive, out-of-band** capacity. It does not sit in the data plane and never blocks, forwards, or inspects live traffic; instead it polls the management plane of each device asynchronously (over Celery workers) to retrieve configuration for offline analysis. This intended-use model ensures that the computational load of anomaly detection, risk scoring, and graph rendering never affects the throughput or latency of the production network. Connectors may issue only read-oriented operations (FortiGate `GET` on `/api/v2/cmdb` and `/api/v2/monitor`; PAN-OS `keygen`, `config&action=show`, and `op` calls); the platform’s code-review gate rejects any `set`, `edit`, `delete`, `move`, `commit`, `enable`, or `disable` firewall operation.

The platform is intended to be hosted on-premises or within a private cloud using Docker Compose, so that sensitive configuration data (IP addressing, object definitions, logical topology) never leaves the organization’s control boundary. A crucial usage caveat applies to all outputs: the topology graph and analysis results describe *logical policy relationships and configured intent*, not observed runtime packet forwarding. LuminaFPM analyzes policy posture, not live network sessions, and this distinction is made explicit in every user-facing view.

## 1.4 Scope

### 1.4.1 Problem Statement

The management of firewall policies in large-scale, multi-vendor organizations is a domain plagued by complexity and entropy. Administrators continuously append rules to support new applications, business partnerships, and temporary access requirements, yet the removal of those rules upon service decommissioning is frequently neglected for fear of disrupting critical business flows. This additive discipline produces rule bloat, which manifests as five concrete, interlocking systemic failures that LuminaFPM is built to address.

- P1. Lack of unified visibility.** Organizations run multiple firewall vendors, each representing policies, objects, zones, services, applications, logging, and inspection differently. Without normalization, administrators must manually compare structurally incompatible configurations and inconsistent terminology, a task that does not scale.
- P2. Logical policy anomalies.** As rulebases grow into the hundreds or thousands of entries, it becomes intractable for a human operator to verify the interaction between every pair of rules; a change to rule #50 can have unforeseen consequences for rule #5000. The most damaging consequences are *shadowing* (a specific rule placed beneath a broader rule of differing action becomes unreachable “dead code” that gives a false sense of control), *redundancy* (a rule whose packet space is already fully handled by a preceding rule of identical action, wasting lookup cycles), and *conflict/correlation* (rules that overlap partially but enforce different actions, so the outcome depends purely on order). To these the platform adds over-permissive access, missing logging, weak inspection profiles, stale temporary rules, and poor descriptions.
- P3. Manual-audit limitations.** Manual audits are slow, inconsistent, and quickly outdated. They struggle with large rulebases, object/group expansion, rule-order reasoning, historical configuration drift, and cross-vendor comparison, and they cannot be reproduced or benchmarked — two reviewers may reach two different conclusions about the same rulebase.
- P4. Cross-vendor blind spots.** A FortiGate rule and a Palo Alto rule may look equivalent from a business viewpoint yet differ in action, logging, inspection profile, schedule, or underlying object definition. Such divergences are invisible to single-device tooling and must be surfaced as explicit cross-device inconsistencies, which is only possible after normalization onto a shared canonical model.
- P5. Static risk versus dynamic threat context.** Standard vulnerability management relies on the National Vulnerability Database and vendor advisories, but there is a latency — often weeks — between an exploit becoming active and a patch or even a CVE identifier being published. A rule that is benign today becomes dangerous when a referenced address turns malicious or a CVE emerges for the deployed firmware version. Purely static auditing tools cannot bridge this gap, leaving organizations compliant on paper yet exposed in practice.

## 1.4.2 Objectives

To address the problem above, the project establishes the following specific and measurable objectives. Each is grounded in a delivered capability of the built system rather than an aspirational feature.

- O1. Unified acquisition and normalization.** Develop read-only connectors and parsers that ingest both vendor configuration formats — FortiGate REST/JSON and Palo Alto PAN-OS XML — and normalize them into a single canonical rule, object, and service model with a correlation layer, before any analysis is performed. *Measure:* both acquisition paths validated end-to-end against the laboratory firewalls, and heterogeneous objects collapsing into shared canonical `normalized_object` records.
- O2. Deterministic, explainable anomaly detection.** Implement a deterministic rule-based anomaly engine that operates *only* on the normalized repository (never on raw vendor output), covering a 46-category anomaly framework. *Measure:* every finding carries the full evidence contract — `anomaly_type`, severity, confidence, description, evidence, recommendation, detection mode, the affected rule, and (where applicable) the related rule — and produces identical output on identical input (no randomness, no machine learning, no network access at analysis time).
- O3. Cross-vendor inconsistency detection.** Use the correlation model to detect both policy conflicts and security-posture differences (object mismatches, logging/inspection divergence) between a FortiGate and a Palo Alto device. *Measure:* cross-device findings emitted from comparison of normalized rules and objects.
- O4. Quantitative risk scoring.** Aggregate anomaly findings into a rule-level and device-level risk score on a fixed 0–100 scale with five tiers (Critical, High, Medium, Low, Informational), stored and reproducible per run. *Measure:* `risk_assessment` rows persisted for both `rule` and `device` scope under a versioned calculation.
- O5. Threat-intelligence enrichment.** Provide an API-based CTI engine that extracts indicators (public IPs, FQDNs, firmware/CVEs), correlates them to rules, and updates risk — kept architecturally separate from deterministic anomaly detection.
- O6. Evidence-grounded AI reporting.** Generate CTI queries and SOC/executive technical reports through a pluggable LLM provider abstraction with a deterministic offline fallback, where the AI is bounded to summarization and report generation and is never the anomaly authority. *Measure:* reports cite the concrete findings they summarize.
- O7. Benchmark-validated correctness.** Prove detection correctness against an authoritative ground-truth benchmark matrix rather than visual inspection, reporting precision, recall,  $F_1$ , and severity-match rate over atomic and compound cases. *Measure:* the benchmark run acts as the platform’s formal acceptance gate.

### 1.4.3 Goals

#### Short-term goals

Deliver a working, containerized v1 covering the two target vendors: read-only acquisition, parsing, normalization onto a canonical model, the deterministic anomaly engine, risk scoring, and a React/TypeScript dashboard with Policy Explorer and Anomaly, Risk, CTI, and Benchmark Centers. Validate the full pipeline end-to-end against the controlled VMware laboratory and pass the ground-truth benchmark acceptance gate.

#### Medium-term goals

Harden and broaden the platform: enrich risk scoring and CTI correlation, mature the AI reporting prompts and provider fallback behavior, expand the benchmark matrix with additional compound and cross-device cases, and improve the logical topology visualization. Strengthen the historical-snapshot and incremental re-synchronization capability so configuration drift can be tracked over time.

#### Long-term goals

Extend coverage and reach beyond v1 through the connector/parser extension points: add further firewall vendors and, optionally, vendor managers (Panorama, FortiManager). Subject to explicit legal, ethical, and operational approval, mature the currently quarantined dark-web threat-intelligence module (the Lumina Threat-Intel / “Robin” subsystem) from a disabled research extension into a supported optional capability. Position the platform as an accessible, open analysis overlay for small and medium enterprises and educational institutions.

### 1.4.4 Included and Excluded Scope

Table 1.1 fixes what is delivered in v1; Table 1.2 fixes what is deliberately out of scope or deferred to future work. These boundaries are hard constraints.

**Table 1.1.** *Explicit in-scope capabilities for LuminaFPM v1.*

| Area             | Included in v1                                                                                            |
|------------------|-----------------------------------------------------------------------------------------------------------|
| Firewall vendors | Fortinet FortiGate (REST/JSON) and Palo Alto PAN-OS (XML API) only.                                       |
| Acquisition      | Direct firewall APIs, read-only, asynchronous via Celery workers.                                         |
| Operation mode   | Read-only extraction and analysis; config-only, no live traffic.                                          |
| Database         | PostgreSQL as the single authoritative source of truth.                                                   |
| Normalization    | Unified canonical rule/object/service model plus object/service correlation, applied before any analysis. |

| Area                   | Included in v1                                                                                                |
|------------------------|---------------------------------------------------------------------------------------------------------------|
| Anomaly analysis       | Deterministic 46-category anomaly framework over normalized data only.                                        |
| Cross-vendor detection | Policy conflicts and security-posture differences via the correlation model.                                  |
| Risk scoring           | Rule-level and device-level 0–100 scores with five tiers, persisted per run.                                  |
| Historical state       | Snapshots and incremental re-synchronization keyed on (device, VDOM/VSYS, vendor UUID).                       |
| CTI                    | API-based indicator extraction and correlation (a separate risk engine).                                      |
| AI / LLM               | Query generation, CTI review, and SOC/executive report generation — evidence-grounded, with offline fallback. |
| Benchmarking           | Atomic and compound ground-truth benchmark matrix; the acceptance gate.                                       |
| Visualization          | Dashboard, Policy Explorer, Anomaly/Risk/CTI/Benchmark Centers, and a <i>logical</i> policy graph.            |
| Security               | Encrypted credentials, per-run auditability, RBAC with five roles.                                            |

**Table 1.2.** *Explicit out-of-scope items and future work.*

| Area                                                   | Status               | Rationale                                |
|--------------------------------------------------------|----------------------|------------------------------------------|
| Firewall modification (push / commit / edit / reorder) | Excluded permanently | Strict read-only mandate.                |
| Live traffic analysis (NetFlow, packet capture, DPI)   | Excluded             | Config-only; no live telemetry.          |
| Real routed traversal between firewalls                | Excluded             | Parallel shared-network lab, not inline. |
| Vulnerability scanning                                 | Excluded             | Not a scanner.                           |
| SIEM / SOAR replacement                                | Excluded             | Complements, does not replace.           |
| Endpoint security (EDR / antivirus)                    | Excluded             | Out of domain.                           |
| Cisco implementation and examples                      | Excluded from v1     | De-scoped; future work only.             |
| Vendor coverage beyond Fortinet + Palo Alto            | Excluded from v1     | Extensible later via connectors/parsers. |
| Panorama / FortiManager integration                    | Future work          | v1 uses direct device APIs.              |

| Area                                               | Status            | Rationale                                                               |
|----------------------------------------------------|-------------------|-------------------------------------------------------------------------|
| Dark-web scraping / Lumina<br>Threat-Intel (Robin) | Future / optional | Disabled by default,<br>legally gated; API-based<br>CTI is primary.     |
| External topology input feed                       | Future            | v1 derives logical rela-<br>tionships from config-<br>ured intent only. |

It is stated plainly, in keeping with the read-only-honesty requirement, that two areas are partial. *Runtime anomalies* are not fully in scope: with no live telemetry, runtime-dependent categories are labeled telemetry-supported or benchmark-simulated rather than fully implemented. The *dark-web / Lumina Threat-Intel* subsystem exists in the codebase but is quarantined behind a disabled-by-default, legally-gated feature flag and removed from the default Docker Compose; it is therefore presented as a documented research extension, not a v1 capability. Cisco, mentioned in the Term-1 proposal, is *not* a supported vendor in this build and appears here only as possible future work.

## 1.5 Definitions

Table 1.3 defines the key terms used throughout this report. Where a term has a formal set-theoretic meaning in the anomaly model, that meaning is given precisely; the conceptual set-theory framing is retained as the formal model, while the implementation is a deterministic rule-based engine over the normalized model.

**Table 1.3.** *Key terms and definitions.*

| Term                   | Definition                                                                                                                                                                                                                                      |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Firewall policy / rule | An ordered access-control entry that permits or denies traffic. Formally modeled as a tuple over source set $S$ , destination set $D$ , service/port set $P$ , protocol $Pr$ , and action $A$ .                                                 |
| Object                 | A named, reusable definition of an address, address group, or host referenced by rules (e.g. <code>WEB_SERVER</code> ).                                                                                                                         |
| Zone / interface       | A logical grouping of interfaces (or a single interface) representing a network segment such as LAN, DMZ, or DB, used as a rule's source or destination context.                                                                                |
| Service                | A named protocol/port definition (e.g. <code>TCP/443</code> ) referenced by rules; vendors model these differently, requiring normalization.                                                                                                    |
| Normalization          | The process of mapping vendor-specific parser models onto one canonical model, mapping vendor actions to canonical ones (e.g. <code>accept/allow/permit</code> $\rightarrow$ <code>allow</code> ) while retaining the original value for audit. |

| Term                   | Definition                                                                                                                                                                                                                                        |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Canonical model        | The unified, vendor-agnostic rule/object/service schema over which all analysis runs; the interchange format the anomaly engine consumes.                                                                                                         |
| Correlation model      | The object/service mapping scheme in which vendor-owned objects remain separate while canonical normalized objects sit above them, linked via mapping tables — enabling cross-vendor object comparison.                                           |
| Shadowing              | A rule $R_y$ is shadowed by an earlier rule $R_x$ when $R_y$ 's match space is a subset of $R_x$ 's and their actions differ ( $S_y \subseteq S_x \wedge D_y \subseteq D_x \wedge P_y \subseteq P_x \wedge A_y \neq A_x$ ): $R_y$ is unreachable. |
| Redundancy             | A rule $R_y$ is redundant to an earlier rule $R_x$ when $R_y$ 's match space is a subset of $R_x$ 's and their actions are equal: $R_y$ adds overhead without changing posture.                                                                   |
| Conflict / correlation | Two rules overlap partially (neither a subset of the other) with differing actions ( $S_x \cap S_y \neq \emptyset \wedge D_x \cap D_y \neq \emptyset \wedge A_x \neq A_y$ ); the outcome depends on rule order.                                   |
| Generalization         | An earlier rule whose match space is a superset of a later rule's, with a differing action — the inverse relation of shadowing.                                                                                                                   |
| Anomaly                | Any deterministically detected defect in the normalized rule-base (one of the 46 categories), carrying an evidence contract: type, severity, confidence, evidence, recommendation, and affected rule(s).                                          |
| Risk score             | A 0–100 value aggregated from a rule's or device's anomaly findings, mapped to a tier (Critical/High/Medium/Low/Informational).                                                                                                                   |
| CTI / IOC              | Cyber Threat Intelligence; an Indicator of Compromise (e.g. a malicious IP, FQDN, or CVE) extracted from configuration and correlated against external threat APIs.                                                                               |
| Read-only              | The mandate that the platform may issue only read operations and must never modify firewall configuration in any way.                                                                                                                             |
| VDOM / VSYS            | A virtual domain (FortiGate) or virtual system (Palo Alto): an isolated logical firewall instance within one physical device; part of the re-synchronization key.                                                                                 |
| Benchmark              | An authoritative ground-truth matrix of expected anomalies (atomic and compound) against which detected output is scored.                                                                                                                         |
| Ground truth           | The set of known-correct expected anomalies provisioned in the lab, used to compute precision, recall, $F_1$ , and severity-match rate.                                                                                                           |

## 1.6 Standards Used

LuminaFPM adheres to established standards across security, networking, software engineering, and data privacy. These are summarized below and revisited in the relevant technical chapters.

### 1.6.1 Security standards

The platform follows least-privilege and defense-in-depth principles. Connectors authenticate using read-only API accounts wherever the device permits; device credentials are encrypted at rest using **Fernet** (an authenticated symmetric scheme built on AES in CBC mode with HMAC) via the Python `cryptography` library, are never hardcoded, never committed to source control, and never exposed to the frontend, with token rotation supported. The detection and reporting design is aligned with the spirit of the OWASP Application Security Verification Standard (ASVS) for input handling and secret management, and with NIST SP 800-41 firewall-management guidance for least-privilege rule hygiene. XML acquisition from PAN-OS uses hardened parsing (`defusedxml`) to mitigate XML entity-expansion and external-entity attacks, and the API enforces a restricted CORS allowlist rather than permissive wildcards.

### 1.6.2 Network standards

Acquisition is performed over the standard vendor management interfaces: HTTPS REST for FortiGate and the HTTPS XML API for PAN-OS (with an explicit, lab-only HTTP override for a single firmware-broken host, detailed in the constraints below). Address objects and segmentation follow **RFC 1918** private-address conventions for the laboratory subnets, and services are modeled over standard TCP/UDP/ICMP semantics. All device communication is confined to the management plane and never the data plane.

### 1.6.3 Programming standards

The Python/FastAPI backend follows **PEP 8** style conventions and uses typed configuration (`pydantic-settings`) and Pydantic models for request/response validation. The React/TypeScript frontend is governed by **ESLint** (with React Hooks and Fast-Refresh rule sets) and a `strict` TypeScript configuration; the codebase is mid-migration from JavaScript to TypeScript. The REST API follows resource-oriented conventions under a versioned `/api/v1` prefix and is documented via OpenAPI. Database schema is owned by **Alembic** migrations rather than destructive auto-synchronization.

### 1.6.4 Data-privacy standards

Because firewall configuration is itself sensitive (revealing addressing, segmentation, and exposure), the platform is designed for on-premises or private-cloud hosting so that

configuration data never leaves the organization’s control boundary. Credentials are encrypted at rest, the database is not published to the host, and every analysis run is auditable. The design is mindful of data-protection obligations (e.g. GDPR-style least-data and accountability principles) by minimizing data collection to configuration strictly necessary for analysis.

## 1.7 Realistic Constraints

The system was designed and validated under a set of realistic constraints, several arising from the controlled laboratory environment. These constraints are stated plainly rather than smoothed over.

- **Strict read-only mandate.** The platform may never modify, push, delete, reorder, enable, disable, or commit any firewall configuration. This is a release-blocking constraint enforced at code review and bounds every connector to read operations only.
- **FortiGate-VM ten-policy cap.** The laboratory FortiGate runs as an unlicensed evaluation virtual machine, whose firewall-policy table is capped at **ten policies** (the device returns “reached the maximum number of entries” beyond this). Benchmark scenarios are therefore designed to fit within this limit.
- **Unlicensed-VM and firmware limitations.** The evaluation firmware exhibits quirks — notably a broken HTTPS administrative binding on the FortiGate (the firmware lacks a usable TLS cipher for its configured versions), requiring a lab-only HTTP override scoped to that single host while production deployments retain HTTPS. The Palo Alto management plane is resource-sensitive under single-host contention.
- **Laboratory, not production.** The validation environment is a controlled, parallel shared-network VMware lab (FortiGate at 192.168.55.10, Palo Alto at 192.168.55.20 on 192.168.55.0/24) running alongside the Docker stack on a single Windows host. It is primarily an anomaly-generation environment, not a realistic enterprise routing simulation.
- **Config-only, no live traffic.** With no NetFlow, packet capture, or session telemetry, runtime-dependent anomalies are benchmark-simulated or telemetry-supported rather than directly observed. All findings derive from configured intent.
- **LLM dependency with offline fallback.** AI-assisted reporting depends on an external LLM provider (default Google Gemini). To avoid a hard external dependency, the provider abstraction includes a deterministic, network-free **OfflineProvider** fallback that activates when no API key is configured, preserving core functionality without connectivity.

## 1.8 SWOT Analysis

### 1.8.1 Strengths

LuminaFPM’s deterministic, explainable anomaly engine yields reproducible, benchmarkable findings, avoiding the false-positive fatigue of heuristic tools; each finding carries a complete evidence contract. Its cross-vendor normalization onto a single canonical model is a genuine technical contribution, exercised by two maximally different API styles (REST/JSON and XML). The strict read-only mandate makes it safe to deploy against production devices, and its containerized, open architecture lowers the barrier to entry for small teams.

### 1.8.2 Weaknesses

v1 supports only two vendors (FortiGate and Palo Alto), so estates with other vendors are not yet covered. The platform reasons over configured intent, not live traffic, so genuinely runtime-dependent anomalies are simulated rather than observed. Backend dependencies are currently unpinned (a reproducibility risk), the frontend ships as a development container, and the AI reporting layer depends on an external LLM unless the offline fallback is used.

### 1.8.3 Opportunities

The connector/parser extension points make additional vendors and vendor managers (Panorama, FortiManager) a clear growth path. Tightening data-protection regulation worldwide and in Egypt creates demand for reproducible least-privilege audit evidence. The commoditization of LLMs enables sophisticated, evidence-grounded reporting at low cost, and an open, accessible model can serve SMEs and educational institutions underserved by costly enterprise suites. The quarantined dark-web intelligence module is a differentiated future capability if legal and ethical gates are met.

### 1.8.4 Threats

Mature commercial incumbents (e.g. Tufin, AlgoSec) hold deep vendor ecosystems and enterprise relationships. Vendor API changes can break connectors and require ongoing maintenance. Reliance on external LLM providers introduces cost, availability, and data-governance considerations, and any future dark-web capability carries legal and ethical risk that must be carefully gated.

## 1.9 PESTLE Analysis

The following analysis frames the platform’s external environment in both the Egyptian and global contexts.

**Table 1.4.** *SWOT summary.*

| <b>Strengths (internal, positive)</b>                                                                                                              | <b>Weaknesses (internal, negative)</b>                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Deterministic, explainable, benchmark-validated detection; cross-vendor normalization; strict read-only safety; containerized, open, accessible.   | Two vendors only in v1; config-only (no live traffic); unpinned backend deps; dev-mode frontend; external LLM dependency. |
| <b>Opportunities (external, positive)</b>                                                                                                          | <b>Threats (external, negative)</b>                                                                                       |
| Extensible to more vendors/managers; rising audit-evidence demand; cheap, capable LLMs; underserved SME/education market; future gated CTI module. | Entrenched commercial incumbents; vendor API churn; LLM cost/governance; legal-ethical risk of dark-web intelligence.     |

### 1.9.1 Political

The fragmentation of global internet governance and the rise of cyber-conflict have led governments to tighten regulation (e.g. the EU Cyber Resilience Act and US Executive Order 14028). In Egypt, national cybersecurity strategy and the work of the National Telecom Regulatory Authority and the Egyptian Computer Emergency Response Team push organizations toward rigorous security governance. This climate mandates auditing tools and creates a built-in demand for reproducible firewall-policy assurance.

### 1.9.2 Economic

Global economic pressure forces organizations to optimize IT spending, accelerating the shift away from expensive monolithic enterprise suites toward modular, open, container-deployable alternatives. In the Egyptian and wider regional market, where security budgets at SMEs are constrained, a low-cost analysis overlay that runs on commodity Docker infrastructure — with only optional LLM API fees — is economically favorable.

### 1.9.3 Social

The democratization of offensive tooling has driven a broad security-awareness culture in which non-specialists increasingly need visibility into security posture. LuminaFPM’s intuitive dashboards and logical visualization lower the cognitive barrier to understanding complex multi-vendor rulebases, and the growing pool of cybersecurity graduates in Egypt provides both an operator base and a talent pipeline.

### 1.9.4 Technological

The commoditization of generative AI and provider abstractions allows small teams to build sophisticated, evidence-grounded reporting that was previously the domain of large vendors, while the maturity of FastAPI, React, PostgreSQL, Celery, and Docker makes a robust, reproducible stack attainable. The continued evolution of vendor APIs (FortiGate

REST, PAN-OS XML) is both an enabler and a maintenance burden that the connector architecture is designed to absorb.

### 1.9.5 Legal

Stringent data-protection regimes — GDPR globally and Egypt’s Personal Data Protection Law (Law No. 151 of 2020) regionally — impose obligations and penalties for negligence. Tools that produce reproducible evidence of least-privilege enforcement and policy hygiene provide essential due-diligence and audit cover. Conversely, any future dark-web intelligence capability must remain behind explicit legal and ethical gates, which is precisely why that subsystem is disabled by default.

### 1.9.6 Environmental

Although the platform is digital, code and configuration efficiency have a marginal environmental footprint. By identifying and enabling the removal of redundant and dead rules, LuminaFPM helps reduce wasted lookup cycles on network appliances, contributing modestly to lower energy consumption in data centers — aligned with broader Green-IT objectives.

# Chapter 2

## Literature Review

This chapter situates LuminaFPM within the academic and commercial landscape of firewall policy management (FPM). It first establishes the theoretical background that underpins the platform’s design: the nature of firewall rulebases, the classical taxonomy of policy anomalies introduced by Al-Shaer and Hamed, the problem of normalising heterogeneous multi-vendor policies into a single analysable model, the contrast between deterministic (formal) and machine-learning (ML) approaches to policy analysis, and the role of Cyber Threat Intelligence (CTI) in deriving exposure from configuration. It then surveys the market and the dominant commercial systems — Tufin, AlgoSec, FireMon, Skybox and Palo Alto Panorama — and analyses them comparatively against LuminaFPM. Finally, it summarises the primary user survey conducted during the Term-1 phase, presents the project’s marketing positioning through the Four P’s framework, and distils the research gaps that motivate the built system. Throughout, the discussion is grounded in the system as actually implemented: a strictly read-only, multi-vendor firewall anomaly management platform for Fortinet FortiGate (REST/JSON) and Palo Alto PAN-OS (XML), whose detection engine is a deterministic, benchmark-validated rule-based engine, and whose CTI and AI-assisted reporting are evidence-grounded subsystems layered above — never replacing — that deterministic core.

### 2.1 Background

#### 2.1.1 Firewall Policy Management

A firewall enforces a network security policy as an ordered list of access-control rules. Each rule matches packets on a set of fields — typically source address, destination address, service (protocol and port), and optionally application, zone, schedule and user — and applies an action such as *allow*, *deny* or *reject*. Because rules are evaluated top-down with first-match semantics, the *order* of rules is as significant as their content: a packet is handled by the first rule whose match space contains it, and all later rules that would also match are never consulted. This ordering dependence is the source of most of the subtle defects that FPM tools exist to detect.

In production, firewall rulebases grow organically. Temporary access rules, migration exceptions, third-party vendor access, emergency changes and outdated business requirements are routinely added but rarely removed, because operators fear disrupting

business-critical flows whose dependencies are no longer documented. The result is a condition widely described as *rule bloat*: rulebases swell into the hundreds or thousands of entries, accumulating shadowing, redundancy, conflicts, duplicate rules, overly broad “any–any” access, missing logging, missing inspection profiles, and stale temporary rules. As a rulebase grows, the number of pairwise rule interactions grows combinatorially, making manual verification of the interaction between every pair of rules effectively intractable for a human operator. A change to one rule can silently alter the reachability of a distant rule, and the security posture of the device becomes opaque even to the team that maintains it. Firewall policy management is the discipline of restoring visibility and correctness to this state — acquiring the live configuration, analysing it for anomalies and risk, and presenting actionable findings to administrators, auditors and Security Operations Centre (SOC) analysts.

### 2.1.2 The Firewall Policy-Anomaly Taxonomy

The formal study of firewall policy anomalies originates with Al-Shaer and Hamed’s *Firewall Policy Advisor* framework (2003–2004), which introduced a rigorous classification of the ways two rules in a policy can relate inconsistently. Their taxonomy distinguishes four canonical anomaly classes, each defined by the set-relationship between the match spaces of a pair of rules and by whether their actions agree:

- **Shadowing.** A rule  $R_y$  is *shadowed* by a preceding rule  $R_x$  when  $R_x$  matches every packet that  $R_y$  would match (i.e.  $R_y$ ’s match space is a subset of  $R_x$ ’s) but the two rules enforce *different* actions. Because  $R_x$  has priority,  $R_y$  is unreachable code: it can never be triggered, and the security control it was intended to enforce is silently neutralised. Shadowing is the most dangerous class because it gives auditors a false sense of coverage.
- **Redundancy.** A rule  $R_y$  is *redundant* when a preceding rule (or set of rules) matches the same packet space and enforces the *same* action. The redundant rule does not change the security outcome, but it inflates the rulebase, wastes lookup cycles, and complicates maintenance.
- **Correlation.** Two rules are *correlated* when their match spaces overlap partially — neither is a subset of the other — and their actions *differ*. The outcome for a packet falling in the intersection depends entirely on rule order, creating ambiguity and a latent security gap.
- **Generalization.** A rule  $R_x$  is a *generalization* of a following rule  $R_y$  when  $R_x$  matches a superset of  $R_y$ ’s traffic with a different action. This is the structural inverse of shadowing and typically indicates a specific exception being overridden by a broader subsequent policy.

This taxonomy is the conceptual seed of LuminaFPM’s detection engine. The platform retains the **set-theoretic framing** as its formal model: a firewall rule is treated as a tuple of sets over source, destination and service, so that shadowing reduces to a

subset/superset relation, redundancy to set equivalence under equal action, and conflict to a non-empty intersection under differing action. LuminaFPM extends the classical four-class model into a broader **46-category anomaly framework** that additionally captures operationally important defects — duplicate rules, overly permissive access, missing logging, missing inspection, stale temporary rules, weak descriptions — and, critically, *cross-device* inconsistencies between rules on different vendors’ firewalls, which the original intra-firewall framework did not address. Whereas Al-Shaer and Hamed’s work analysed conflicts within a single rulebase, modern multi-vendor estates require inter-firewall analysis in which an upstream policy on one device may render a downstream rule on another irrelevant.

### 2.1.3 Normalisation of Heterogeneous Vendor Policies

A precondition for any cross-vendor analysis is *normalisation*: the translation of disparate, proprietary vendor representations into a single, vendor-agnostic policy model. Each vendor represents policies, address and service objects, zones, applications, logging and inspection differently, and exposes them through incompatible management interfaces. LuminaFPM’s v1 targets two deliberately heterogeneous styles — Fortinet FortiGate over a REST/JSON API and Palo Alto PAN-OS over an XML API (`type=keygen`, `type=config&action=show`, `type=op`). These two interface paradigms maximally exercise the normalisation layer and make the cross-vendor contribution credible: a finding that holds across both REST/JSON and XML representations cannot be an artefact of one vendor’s data format.

Without normalisation, administrators must manually reconcile incompatible structures and terminology before they can even begin to reason about consistency. LuminaFPM resolves this with a deterministic normalisation engine that maps each vendor’s parsed model into a canonical rule/object/service model, mapping vendor actions onto canonical ones (for example `accept/allow/permit` → `allow`) while retaining the original vendor value for audit traceability. Objects and services are reconciled through a **correlation model**: vendor-owned objects are preserved unchanged, canonical normalised objects are created *above* them, and mapping tables link the two. This design enables device ownership and vendor traceability, object-mismatch and duplicate detection, drift analysis over time, and — most importantly — the cross-vendor rule comparison that the deterministic anomaly engine consumes. The engine operates strictly on normalised data, never on raw vendor configuration, so that the same logical defect is detected identically regardless of which vendor produced the rule.

### 2.1.4 Deterministic and Formal Methods versus Machine Learning

There are two broad philosophies for policy analysis. *Heuristic and machine-learning* approaches infer “likely” anomalies from statistical features or learned patterns; they scale to unstructured signals but produce probabilistic verdicts, suffer from false-positive fatigue, require labelled training data, and — decisively for a security-auditing tool — cannot *prove*

that a reported anomaly exists. *Deterministic and formal* approaches, by contrast, define anomalies as precise set-relationships over rule match spaces and decide them exactly: a shadowing finding is not an estimate but a proof that one rule’s match space is contained in another’s under a conflicting action.

LuminaFPM is firmly in the deterministic camp. Its anomaly engine is a deterministic, rule-based engine whose findings are repeatable, explainable and benchmarkable; the design explicitly bars artificial intelligence from being the anomaly authority. Detection uses canonical correlation keys and a small algebra of set predicates — coverage (subset under an “any” sentinel), overlap (non-empty intersection), and same-access equality — to decide redundancy, shadowing and conflict over rule pairs. Every finding carries evidence, a timestamp, the affected rule, the related rule where applicable, severity, confidence and a recommended action, so that a human can verify the conclusion from the evidence alone. This determinism is also what makes the engine *benchmarkable*: because identical inputs always produce identical outputs, the engine’s correctness can be proven against a ground-truth matrix of expected anomalies rather than asserted by visual inspection (Section 2.6). Machine learning and large language models are retained in LuminaFPM only for tasks where probabilistic reasoning is appropriate and where their output is grounded in deterministic evidence: CTI query generation and SOC report synthesis.

### 2.1.5 CTI and Policy-Derived Exposure

A rulebase that is internally consistent today can become risky tomorrow if a referenced address or domain turns malicious, or if a CVE emerges for the deployed firmware version. This is the gap that Cyber Threat Intelligence (CTI) addresses: correlating static configuration against the dynamic external threat landscape to surface *exposure* that no purely structural analysis could reveal. Crucially, this is a fundamentally different kind of judgement from deterministic anomaly detection — it is probabilistic, time-varying and dependent on external feeds — and LuminaFPM therefore keeps it in a **separate risk/CTI engine**, decoupled from the deterministic anomaly authority.

LuminaFPM’s CTI subsystem extracts indicators (public IP addresses, CIDRs and FQDNs) from normalised network objects under a strict *data-minimisation* rule: internal RFC1918, loopback and link-local addresses, and the “any” sentinels (0.0.0.0/0, ::/0), are never transmitted to external providers unless an operator explicitly opts in. Indicators are enriched through a provider abstraction (**CtiProvider**) that always includes a deterministic, network-free offline provider — so the capability is demonstrable without internet access — and activates real REST providers such as AbuseIPDB only when an API key is supplied. The runner picks the worst verdict per indicator and performs *cross-device correlation*: for a malicious indicator it locates every rule across all managed devices that references the matching object and raises a **threat\_exposure** finding only on rules whose action is **allow** — that is, rules that actually permit traffic to or from a known-bad indicator. This yields a policy-derived, blast-radius-style notion of exposure that is grounded in configured intent rather than live packet telemetry, consistent with the platform’s logical (not runtime) analysis model.

## 2.2 Market Analysis

Firewall and network-security policy management sits within the broader network-security-policy-management (NSPM) segment, which industry analysts size in the low single-digit billions of US dollars annually and project to grow at a double-digit compound annual rate through the latter half of the decade. Several structural drivers sustain this growth. First, the proliferation of *hybrid and multi-vendor estates* — enterprises rarely standardise on a single firewall vendor, and the addition of cloud security groups multiplies the number of distinct policy dialects an organisation must manage. Second, *regulatory pressure* from frameworks such as NIST SP 800-41, PCI-DSS, ISO 27001 and a tightening international regulatory climate mandates demonstrable, repeatable evidence that rulebases enforce least-privilege and are free of conflict. Third, the chronic shortage of skilled security staff forces organisations to *automate* audit work that was historically performed manually in spreadsheets. Fourth, the convergence of policy management with threat intelligence reflects a demand for *context-aware* risk rather than static compliance snapshots.

The market segments along several axes. By *buyer size*, the incumbents (Tufin, Algo-Sec, FireMon, Skybox) target large enterprises and the Fortune 500, with licensing that frequently reaches six figures and deployment complexity that excludes small and medium enterprises (SMEs), educational institutions and smaller security teams. By *primary function*, vendors cluster into change-orchestration platforms, application-connectivity platforms, real-time compliance/risk platforms, and vendor-native managers (Panorama, FortiManager). By *delivery model*, the market is shifting from monolithic on-premises appliances toward modular, container-deployable and increasingly SaaS-native offerings. LuminaFPM positions itself in an under-served gap of this segmentation: an open, academically transparent, container-deployable platform whose differentiator is not change orchestration but *provable*, cross-vendor anomaly detection with evidence-grounded AI reporting — accessible to the SME and education tier that the incumbents price out.

## 2.3 Related Projects and Similar Systems

The following subsections review the dominant commercial systems, each examined for its overview, principal features, limitations, and the specific gap that LuminaFPM addresses relative to it.

### 2.3.1 Tufin Orchestration Suite

**Overview.** Tufin is widely regarded as the industry standard for network-security-policy *orchestration*, tailored to large enterprises. Its central strength is automating the change-request lifecycle, from ticket to implementation, across a heterogeneous device estate.

**Features.** Extensive vendor-support ecosystem (Check Point, Cisco, Juniper, Palo Alto, cloud security groups); deep integration with IT-service-management platforms such as ServiceNow; robust topology mapping and path analysis; automated rule-recertification

and compliance workflows. **Limitations.** Licensing is prohibitive for SMEs, frequently reaching six figures; the platform is complex to deploy and operate; and its threat-intelligence capabilities are secondary to compliance automation, relying on conventional feeds rather than evidence-grounded correlation against the specific deployed inventory. **Gap addressed.** LuminaFPM forgoes write-back orchestration in favour of a strictly read-only, provable detection core that is accessible to organisations Tufin prices out, and it treats cross-vendor inconsistency and policy-derived CTI exposure as first-class outputs rather than add-ons.

### 2.3.2 AlgoSec Security Management Suite

**Overview.** AlgoSec distinguishes itself with a business-application-centric model, mapping technical rules to business services (for example, a “Billing App”) to preserve connectivity and uptime through change. **Features.** Best-in-class visualisation of application traffic flows; highly automated reporting for PCI-DSS, HIPAA and similar standards; “zero-touch” change automation; risk and connectivity analysis tied to applications. **Limitations.** The interface is dated relative to SaaS-native platforms; like Tufin, its intelligence is largely reactive — driven by published vulnerabilities rather than by correlation of the live inventory against current threat indicators — and it is an enterprise-priced, closed product. **Gap addressed.** LuminaFPM contributes a deterministic, mathematically provable anomaly core (minimising false-positive fatigue) and an evidence-grounded AI reporting layer, in an open, academically reproducible form.

### 2.3.3 FireMon

**Overview.** FireMon focuses on *real-time* security-policy management, continuous compliance assessment and rule-usage analytics across large, distributed firewall estates. **Features.** Continuous monitoring and change detection; rule-usage and “rule-effectiveness” analytics that flag unused or overly permissive rules; customisable compliance assessments; API-driven integration into security operations. **Limitations.** Enterprise pricing and operational complexity; analytics are predominantly heuristic and usage-driven rather than founded on a formal, provable model of rule relationships; multi-vendor normalisation is a means to inventory rather than a vehicle for cross-vendor *proof*. **Gap addressed.** LuminaFPM replaces heuristic effectiveness scoring with a deterministic, benchmark-validated anomaly model whose every finding is independently verifiable from its evidence.

### 2.3.4 Skybox Security

**Overview.** Skybox builds a model of the network from firewall and routing configurations to perform attack-surface and vulnerability analysis, including access-path simulation and exposure assessment. **Features.** Network and attack-surface modelling; access-path analysis across multi-vendor topologies; vulnerability-context prioritisation; change and compliance management. **Limitations.** Heavyweight to deploy and model-maintain;

expensive and enterprise-oriented; its strength is path/attack modelling rather than a transparent, provable rule-anomaly taxonomy, and its threat context derives from conventional vulnerability data rather than evidence-grounded, inventory-specific correlation. **Gap addressed.** LuminaFPM offers a lightweight, container-deployable alternative whose cross-vendor analysis is grounded in an explicit, reproducible anomaly framework and whose CTI is correlated specifically to rules that permit traffic to known-bad indicators.

### 2.3.5 Palo Alto Panorama

**Overview.** Panorama is Palo Alto Networks’ *vendor-native* centralised management platform for PAN-OS firewalls, providing unified configuration, logging and policy administration. **Features.** Centralised, authoritative management and *write-back* configuration of Palo Alto devices; aggregated logging and reporting; templates and device groups for scale; deep native integration with the PAN-OS ecosystem. **Limitations.** It is single-vendor by design — it does not manage or normalise Fortinet (or other) policies — and it is an operational management tool, not an independent, vendor-neutral auditor; cross-vendor inconsistency detection is outside its scope. **Gap addressed.** LuminaFPM is explicitly multi-vendor and read-only: it normalises FortiGate and PAN-OS into one model and reasons across them, providing the independent cross-vendor consistency view that no single vendor’s native manager can offer.

### 2.3.6 Comparative Feature Analysis

Table 2.1 compares the surveyed systems against LuminaFPM across the capability dimensions most relevant to this project. A ✓ denotes full support, × denotes absence, and “partial” denotes limited or indirect support. The comparison emphasises the dimensions on which LuminaFPM is differentiated: multi-vendor normalisation feeding a *provable* anomaly model, explicit cross-vendor inconsistency detection, a strictly read-only operating mode, integrated risk scoring, API-based CTI enrichment, AI evidence-grounded reporting, benchmark-based validation of detection correctness, and open/academic availability.

**Table 2.1.** Comparative feature analysis of LuminaFPM versus representative commercial firewall policy-management systems.

| Capability                           | Tufin   | AlgoSec | FireMon | Skybox  | Panorama | LuminaFPM |
|--------------------------------------|---------|---------|---------|---------|----------|-----------|
| Multi-vendor normalisation           | ✓       | ✓       | ✓       | ✓       | ×        | ✓         |
| Deterministic anomaly proof          | partial | partial | ×       | partial | ×        | ✓         |
| Cross-vendor inconsistency detection | partial | partial | partial | ✓       | ×        | ✓         |
| Strictly read-only operation         | ×       | ×       | ×       | ✓       | ×        | ✓         |
| Integrated risk scoring              | ✓       | ✓       | ✓       | ✓       | partial  | ✓         |

| Capability                     | Tufin   | AlgoSec | FireMon | Skybox  | Panorama | LuminaFPM |
|--------------------------------|---------|---------|---------|---------|----------|-----------|
| API-based CTI enrichment       | partial | partial | partial | partial | partial  | ✓         |
| AI evidence-grounded reporting | ×       | ×       | ×       | ×       | ×        | ✓         |
| Benchmark-validated detection  | ×       | ×       | ×       | ×       | ×        | ✓         |
| Open / academically accessible | ×       | ×       | ×       | ×       | ×        | ✓         |

The comparison makes LuminaFPM’s positioning explicit. The incumbents lead on breadth of vendor support and change orchestration, but none combines a strictly read-only mandate, a *provable* (benchmark-validated) deterministic anomaly model, and evidence-grounded AI reporting in an open, reproducible package. Panorama, as a vendor-native manager, is the clearest contrast: it is authoritative and write-capable for a single vendor, where LuminaFPM is deliberately read-only and multi-vendor.

## 2.4 Survey and User Insights

To validate the problem framing, a structured survey was conducted during the Term-1 phase, targeting practitioners across the cybersecurity and IT-infrastructure domains. Respondents spanned network security architects and engineers, SOC analysts, compliance managers and auditors, and DevSecOps engineers — the same stakeholder spectrum the platform is designed to serve. The instrument probed the perceived complexity of managing multi-vendor firewall estates, the prevalence and impact of human error in rulebase maintenance, the perceived value of mathematically provable (rather than heuristic) anomaly detection, the importance of AI assistance, and the preference for a single consolidated dashboard over fragmented per-vendor tools.

The dominant themes were consistent and reinforced the project’s core hypotheses. A large majority of respondents rated multi-vendor policy management as *complex* or *highly complex*, attributing this directly to incompatible vendor terminology and the absence of a unified view. Human error in rulebase maintenance was widely recognised as a leading cause of misconfiguration, with respondents acknowledging that manual verification of large rulebases is impractical. The value of *mathematically provable* anomaly detection was rated highly: practitioners expressed clear preference for findings they can verify over heuristic alerts that erode trust through false positives. AI assistance was viewed favourably for accelerating analysis and report generation, provided it remains grounded in verifiable evidence rather than acting as an unaccountable authority. Finally, an overwhelming preference emerged for a single “one dashboard for everything” console that

unifies inventory, anomalies, risk, threat intelligence and reporting. Table 2.2 summarises the principal survey dimensions and their findings.

**Table 2.2.** *Summary of Term-1 survey dimensions and principal findings.*

| Survey dimension                          | Principal finding                                                                                                                             |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Respondent roles                          | Network security architects/engineers, SOC analysts, compliance managers/auditors and DevSecOps engineers — the platform’s intended audience. |
| Complexity of multi-vendor management     | A clear majority rated it complex or highly complex, citing incompatible vendor terminology and the lack of a unified view.                   |
| Human error in rulebase maintenance       | Widely recognised as a leading cause of misconfiguration; manual verification of large rulebases is regarded as impractical.                  |
| Value of provable (math-based) anomalies  | Rated highly; strong preference for verifiable findings over heuristic alerts prone to false positives.                                       |
| Importance of AI assistance               | Viewed favourably for accelerating analysis and reporting, conditional on evidence-grounding rather than autonomous decision-making.          |
| Preference for one consolidated dashboard | Strong preference for a single console unifying inventory, anomalies, risk, CTI and reports over fragmented per-vendor tools.                 |

These insights directly shaped LuminaFPM’s design priorities: a unified normalised model and single console answer the complexity and one-dashboard findings; the deterministic, benchmark-validated engine answers the demand for provable anomalies and the distrust of false positives; and the evidence-grounded, non-authoritative role assigned to the LLM answers the qualified appetite for AI assistance.

## 2.5 The Four P’s of Marketing

### 2.5.1 Product

LuminaFPM is positioned as a strictly read-only, multi-vendor *firewall anomaly management platform* that converges deterministic policy auditing with evidence-grounded threat intelligence. Its core value is a single pane of glass that acquires FortiGate and PAN-OS policies, normalises them into one canonical model, and produces *provable* anomaly findings, rule- and device-level risk scores, API-based CTI exposure correlation, and AI-assisted SOC and executive reports — all without ever writing to a firewall. The differentiators are the benchmark-validated deterministic engine, explicit cross-vendor inconsistency detection, and the read-only mandate that makes the platform safe to deploy against production infrastructure.

### 2.5.2 Price

LuminaFPM follows an open, academically transparent model rather than enterprise licensing. The platform is container-deployable via Docker Compose, so adopters incur only infrastructure cost and, optionally, metered API usage for an external LLM provider (Gemini by default) or CTI provider (for example AbuseIPDB) — both of which degrade gracefully to a deterministic, network-free offline fallback when unkeyed. This zero-licence posture deliberately targets the SME, education and smaller-security-team tier that the six-figure incumbents exclude.

### 2.5.3 Place

Distribution is through containerised images and a public source repository, enabling consistent deployment across Linux, Windows and private-cloud environments that support Docker. Because all analysis runs on-premises or in a private cloud, sensitive configuration data — addressing, topology, rule content — never leaves the organisation’s control boundary, which is itself a distribution advantage for security-sensitive and regulated adopters. The academic provenance of the project also places it naturally within university and research contexts as a reproducible reference implementation.

### 2.5.4 Promotion

Promotion relies on the openness and reproducibility of the work rather than on a commercial sales motion: a transparent, benchmark-validated methodology that practitioners can independently verify; technical write-ups demonstrating provable anomaly detection and cross-vendor normalisation; and engagement with the practitioner and academic security communities. The benchmark framework itself doubles as a promotional asset, because it provides objective, reproducible evidence of detection correctness rather than unverifiable marketing claims.

## 2.6 Research Gaps

The literature and the commercial landscape together reveal several gaps that LuminaFPM is designed to close:

1. **From intra-firewall to cross-vendor proof.** Classical anomaly theory (Al-Shaer and Hamed) addresses conflicts within a single rulebase, and commercial tools provide multi-vendor inventory, but none elevates *cross-vendor* inconsistency — the same logical access expressed differently, or with differing action/logging/inspection, on a FortiGate versus a PAN-OS device — to a provable, first-class finding over a unified normalised model.
2. **Provability over heuristics.** Commercial “rule-effectiveness” analytics are largely heuristic or usage-driven and produce probabilistic alerts. There is a gap for a

deterministic, set-theoretic engine whose every finding is independently verifiable from its evidence, eliminating false-positive fatigue.

3. **Benchmark-validated detection.** Detection correctness in industry tools is typically asserted, not demonstrated against ground truth. LuminaFPM closes this with an explicit benchmark framework that scores engine output against an authoritative expected-anomaly matrix — computing true positives, false positives, false negatives, precision, recall,  $F_1$  and severity-match rate — making the benchmark the *acceptance gate* for the engine.
4. **Evidence-grounded AI, not AI authority.** Where AI appears in this domain it is often opaque. LuminaFPM constrains the LLM to query generation, CTI review and report synthesis, grounded in deterministic evidence, and explicitly bars it from being the anomaly authority — a disciplined separation that the survey indicates practitioners want.
5. **Accessibility.** Provable, cross-vendor policy analysis has been the preserve of expensive enterprise platforms. An open, container-deployable, academically reproducible implementation lowers the barrier for SMEs, educational institutions and researchers.

## 2.7 Summary

This chapter established the academic and commercial context for LuminaFPM. It traced firewall policy management from the problem of rule bloat in growing multi-vendor estates, through Al-Shaer and Hamed’s foundational anomaly taxonomy (shadowing, redundancy, correlation, generalization), to the requirement for normalisation of heterogeneous vendor policies and the decisive contrast between deterministic and machine-learning approaches to policy analysis. It positioned CTI as a complementary, deliberately separate source of policy-derived exposure. A market analysis and a comparative review of Tufin, AlgoSec, FireMon, Skybox and Palo Alto Panorama (Table 2.1) showed that, while incumbents lead on vendor breadth and change orchestration, none combines a strictly read-only mandate, a benchmark-validated deterministic anomaly model, explicit cross-vendor inconsistency detection, and evidence-grounded AI reporting in an open form. The Term-1 survey (Table 2.2) corroborated the problem framing and the demand for provable anomalies and a unified console. From these foundations, five research gaps were identified that LuminaFPM directly addresses. The chapters that follow detail the system description, requirements, design and implementation that realise these contributions.

# Chapter 3

## System Analysis and Design

This chapter elaborates the analytical and architectural foundation of **LuminaFPM**, the strictly read-only, multi-vendor Firewall Anomaly Management Platform built for this project. It is divided into two complementary halves. The present half (Part A — System Analysis) establishes *what* the system is required to do and *under which conditions*: the project methodology expressed as a layered processing pipeline, the categories of users and their privileges, the operating assumptions, constraints, limitations and dependencies, the intended technology stack, the catalogue of functional and non-functional requirements, and the virtualized laboratory in which the platform was validated. The second half (Part B — System Design) then translates these requirements into concrete design artefacts (data-flow, sequence, class, deployment and entity-relationship diagrams). Throughout, the analysis is grounded in the system as it was actually built — its source packages, database schema, REST endpoints, and the parallel VMware lab — rather than in the earlier Term-1 proposal, and any divergence between the two is stated plainly.

### 3.1 Overview

LuminaFPM addresses a narrow but high-value problem in firewall operations: enterprises routinely operate firewalls from *different vendors* whose policy languages, object models, and management APIs are mutually incompatible, which makes it difficult to reason about the combined security posture and to find the latent rule-level defects (shadowed rules, redundant rules, conflicting rules, over-permissive rules, missing logging, and cross-device inconsistencies) that accumulate in long-lived rule bases. The platform acquires firewall configuration directly from the devices over their native APIs, *normalizes* the heterogeneous vendor data into a single canonical model, and runs a *deterministic* rule-based anomaly-detection engine over that normalized model. On top of the deterministic core it layers risk scoring, API-based Cyber Threat Intelligence (CTI) enrichment, and AI-assisted (LLM) evidence-grounded SOC reporting, surfacing all results through a React dashboard.

Two design commitments dominate the analysis and recur throughout this chapter. First, the platform is **strictly read-only**: it never writes, pushes, deletes, reorders, enables, disables, or commits any firewall configuration; corrective action remains entirely with authorized human operators. Read-only operation is a non-negotiable, release-blocking constraint, and it is enforced structurally — the vendor connectors contain no write/commit code path whatsoever. Second, the deterministic engine analyzes the **normalized model**

**only**; it must never inspect raw Fortinet JSON or Palo Alto XML directly, because vendor-specific differences must be resolved by the parsing and normalization layers before any analysis takes place.

The first-version (v1) vendor scope is deliberately limited to two vendors: **Fortinet FortiGate** (consumed over its REST/JSON API) and **Palo Alto Networks PAN-OS** (consumed over its XML API). These were chosen because the validation laboratory exercises both acquisition paths end-to-end. Additional vendors are an explicit extension point — new connectors and parsers can be added later — but only Fortinet and Palo Alto are implemented in v1. (The Term-1 proposal mentioned Cisco; it is de-scoped from v1 and appears, if at all, only as future work.) Likewise, the dark-web / Lumina-Threat-Intel (LTI) subsystem exists in the codebase but is disabled by default and is presented here as an optional, future research module rather than a core v1 capability; API-based CTI is the v1 threat-intelligence path.

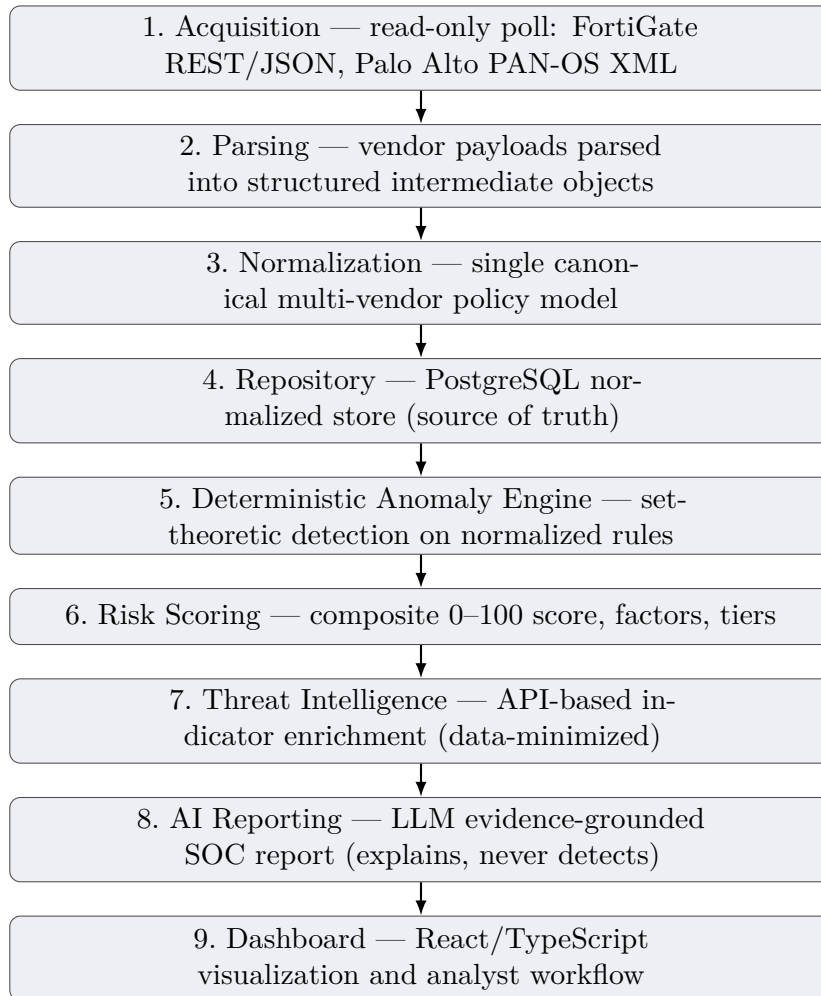
The conceptual model underlying detection is best expressed with elementary **set theory**: a firewall rule is treated as a tuple of sets over source, destination, service, and action; shadowing corresponds to a subset/superset relation between two earlier-and-later rules; redundancy corresponds to set equivalence; and a conflict corresponds to an overlapping match with differing actions. This set-theoretic framing is the formal model used to reason about the anomalies, while the *implementation* is a deterministic, rule-based engine over the normalized model (not machine learning). Section 3.2 develops this pipeline as the project methodology.

## 3.2 Methodology

The methodology adopted for LuminaFPM is a **strictly layered processing pipeline** in which each stage has a single responsibility, consumes a well-defined input from the stage above it, and produces a well-defined output for the stage below it. The governing architectural rule is that *no layer may bypass the layer beneath it*: in particular, acquisition is the only layer permitted to touch a firewall, and the anomaly engine is permitted to read only the normalized repository. This separation is what makes the platform multi-vendor without polluting the analytical core with vendor-specific logic, and it is what makes the analytical results deterministic, explainable, and reproducible. Figure 3.1 depicts the end-to-end pipeline and the data structures handed between successive layers.

Each conceptual stage in Figure 3.1 maps onto a concrete Python package under **backend/services/** (and, for presentation, onto the React frontend). The stages are as follows.

1. **Acquisition** (**services/acquisition/**). The only firewall-facing layer, and read-only by construction. A vendor connector (**FortiGateConnector** or **PaloAltoConnector**, selected at runtime by `get_connector(config)`) authenticates to the device API, polls only read endpoints, and returns an **AcquisitionBundle** of raw vendor responses. FortiGate is polled over the FortiOS REST API



**Figure 3.1.** *The LuminaFPM end-to-end processing pipeline (nine logical layers).*

(`/api/v2/cmdb/firewall/*` and `/api/v2/monitor/system/status`); Palo Alto is polled over the PAN-OS XML API (`type=keygen`, `type=config&action=show`, `type=op`). Each raw response is persisted to disk as a `RawArtifact` with a SHA-256 manifest for replay and audit, and the acquisition job advances through a documented state machine (`queued`  $\rightarrow$  `running`  $\rightarrow$  `success/partial_success/failed/...`). Acquisition runs asynchronously on a Celery worker and *never* inside the web-request cycle.

2. **Parsing** (`services/parsing/`). Vendor-aware but anomaly-agnostic. `parse_artifacts(...)` replays the stored artifacts and converts raw FortiGate JSON or Palo Alto XML into a vendor-tagged `ParsedDevicePayload` capturing rule name, vendor rule ID, vendor UUID, rule order, zones, source/destination/service objects, action, logging, profile group, schedule, NAT, description, tags, and enabled state. No anomaly logic runs here.
3. **Normalization** (`services/normalization/`). `normalize_payloads([payload])` is a pure, database-free function that maps each vendor parser model into one canonical rule/object/service model, mapping vendor actions to canonical ones (`accept/allow/permit`  $\rightarrow$  `allow`) while preserving the original vendor value for audit, and building the object *correlation model* (vendor-owned objects are preserved, canonical objects sit above them, and mapping tables connect the two). It returns a `NormalizationResult`.
4. **Normalized Policy Repository** (PostgreSQL, Schema v4). The authoritative source of truth for v1. `persist_result(...)` performs an idempotent upsert keyed on (`device_id`, `vdom_vsys`, `vendor_uuid`) into tables such as `policy_rule`, `network_object`, `normalized_object`, and `normalized_service`. Schema ownership is delegated to Alembic migrations rather than runtime `create_all`, so there is no destructive auto-sync.
5. **Deterministic Anomaly Engine** (`services/anomaly/`). `analyze(result)` consumes a reconstructed `NormalizationResult` and returns a deterministic `list[Finding]` with no randomness, no mock data, no LLM, and no network access. Determinism is guaranteed by a fixed detector order, a stable rule sort keyed on (`device_id`, `rule_order`, `vendor_uuid`, `rule_name`), and pairwise iteration via `itertools.combinations` so that the earlier rule always has the lower order. Findings are written as `rule_anomaly` rows tied to one `AnomalyExecutionLog` run (`engine_version` = "anomaly-engine/1.0.0"), each carrying the full evidence contract (type, severity, confidence, description, evidence, recommendation, detection mode, affected and related rules).
6. **Risk Scoring** (`services/risk/`). A separate engine that does *not* replace anomaly detection. `run_risk(...)` aggregates the findings of each active rule into a 0–100 score and a tier (90–100 Critical, 70–89 High, 40–69 Medium, 1–39 Low, 0 Informational), persisting `risk_assessment` rows at both rule and device scope. It is triggered best-effort at the end of each anomaly run; a risk failure must never fail the analysis.

7. **API-based CTI** (`services/cti/`). `run_cti(...)` extracts indicators (public IPs, FQDNs, firmware/CVEs) from the normalized rules, generates CTI queries via the LLM layer, calls external CTI APIs, correlates the responses back to rules as threat observations, and updates the risk score. CTI is kept distinct from the deterministic anomaly path.
8. **AI / LLM Reporting** (`services/llm/`). A provider abstraction (`LLMProvider`) with a `GeminiProvider` default and a deterministic, network-free `OfflineProvider` fallback. Its role is bounded to CTI query generation, CTI review/summary, and evidence-grounded SOC/executive report generation; the LLM is *never* an anomaly authority.
9. **Presentation & Reporting** (`frontend/`). A React + TypeScript + Vite single-page application rendering the Dashboard, Device Inventory, Policy Explorer, Anomaly/Risk/CTI/Benchmark Centers, Reports, and a logical-policy graph. The graph represents *configured policy relationships only*, never live runtime traffic.

The stages are wired together by an asynchronous task chain so that slow firewall polling never blocks analysis: a `POST /api/devices/{id}/poll` request returns `202 Accepted` immediately after enqueueing `poll_device`; that task chains `normalize_device`; which chains `run_anomaly_analysis`; which finally invokes `run_risk`. Each task runs on a dedicated Celery queue (`acquisition`, `normalization`, `analysis`, `cti`, `reporting`) brokered by Redis, with PostgreSQL serving both as the operational state store and as the Celery result backend. The frontend then reads results back over read-only endpoints. This pipeline *is* the project methodology: every requirement in Section 3.6 is traceable to one of these layers.

## 3.3 User Requirements and System Privileges

LuminaFPM is an analyst- and auditor-facing decision-support tool. Its primary audience is firewall administrators, SOC engineers, and security auditors, together with the cybersecurity development team. Because the platform never enacts changes on firewalls, all user privileges are fundamentally *read-centric*: even the most privileged role manages *platform* configuration (credentials, settings, integrations) rather than *firewall* configuration. The underlying RBAC model defines fine-grained roles (Platform Admin, Firewall Engineer, SOC Analyst, Auditor, Viewer); for the purposes of system analysis these consolidate into three principal user categories, described below.

### 3.3.1 Security Analyst (Firewall Engineer / SOC Analyst)

The Security Analyst is the day-to-day operator of the platform. This user registers firewall devices, configures their read-only poll credentials, triggers acquisition and analysis runs, and investigates the resulting findings. The analyst browses normalized policies in the Policy Explorer, inspects anomaly findings and their evidence in the Anomaly

Center, reviews rule- and device-level risk scores in the Risk Center, examines CTI threat observations, and reads AI-generated SOC reports. The analyst’s responsibility is to interpret the platform’s evidence and to recommend (but not apply, through the platform) remediations to firewall owners.

### 3.3.2 SOC Lead / Administrator (Platform Admin)

The SOC Lead / Administrator superset the analyst’s capabilities and additionally owns *platform* administration: managing device credentials (create, rotate, test), configuring integrations (CTI API keys, LLM provider and model selection), managing users and roles, and tuning system settings. This role is the trust anchor for the encrypted credential store and is the only role permitted to alter platform-wide configuration. Crucially, even this role holds no capability to modify a firewall — the read-only mandate applies uniformly.

### 3.3.3 Auditor / Viewer

The Auditor / Viewer is a strictly read-only consumer of evidence. An Auditor reviews historical findings, risk assessments, benchmark results, and reports as audit evidence, relying on the immutable per-run logs (`anomaly_execution_log`, raw-artifact SHA-256 manifests) for traceability; a Viewer sees only dashboards and summary pages. Neither can trigger analysis runs, manage credentials, or alter configuration.

Table 3.1 summarizes the responsibilities and privileges of the three categories. A check mark (✓) denotes a granted privilege and a cross (×) denotes a denied one.

**Table 3.1.** *User categories, responsibilities, and platform privileges.*

| Privilege / Responsibility                   | Security Analyst | SOC Lead / Admin | Auditor / Viewer |
|----------------------------------------------|------------------|------------------|------------------|
| Register / edit firewall devices             | ✓                | ✓                | ×                |
| Configure read-only poll credentials         | ✓                | ✓                | ×                |
| Trigger acquisition / analysis runs          | ✓                | ✓                | ×                |
| Browse normalized policies (Policy Explorer) | ✓                | ✓                | ✓                |
| View anomaly findings + evidence             | ✓                | ✓                | ✓                |
| View rule / device risk scores               | ✓                | ✓                | ✓                |
| View CTI threat observations                 | ✓                | ✓                | ✓                |
| Read AI-generated SOC reports                | ✓                | ✓                | ✓                |
| View benchmark results / dashboards          | ✓                | ✓                | ✓                |
| Rotate / test device credentials             | ×                | ✓                | ×                |
| Manage CTI / LLM integrations                | ×                | ✓                | ×                |
| Manage users, roles, system settings         | ×                | ✓                | ×                |

| Privilege / Responsibility               | Security Analyst | SOC Lead / Admin | Auditor / Viewer |
|------------------------------------------|------------------|------------------|------------------|
| Modify firewall configuration (any kind) | ×                | ×                | ×                |

## 3.4 Assumptions, Constraints, Limitations, and Dependencies

### 3.4.1 Assumptions

The analysis assumes that each managed firewall exposes a reachable management API (FortiOS REST for FortiGate, PAN-OS XML for Palo Alto) and that a *dedicated, least-privilege, read-only* API account is provisioned on the device for LuminaFPM to use. It assumes that the network objects and services referenced by the rule base (address objects, service objects, zones) already exist on the device, so that normalization can resolve rule references. It assumes that the operator deploys the supporting infrastructure (PostgreSQL, Redis, the Celery worker) as specified by the Docker Compose stack, and that an encryption key (`ENCRYPTION_KEY`) is configured so that stored credentials can be encrypted and later decrypted. Finally, it assumes that human operators, not the platform, carry out any remediation on the firewalls.

### 3.4.2 Constraints

The dominant constraint is the **strict read-only mandate**: the platform must never modify, push, delete, reorder, enable, disable, or commit firewall configuration, and no write/commit code path exists in any connector. A second hard constraint is the **normalized-only analysis rule**: the anomaly engine must analyze normalized records only and must never read raw vendor JSON/XML. The v1 **vendor scope** is constrained to Fortinet FortiGate and Palo Alto PAN-OS only. The **datastore** is constrained to PostgreSQL as the single authoritative source of truth (MongoDB is explicitly excluded). Acquisition is constrained to run **asynchronously** on Celery workers and never inside the FastAPI request cycle. Security constraints further require that device credentials be encrypted at rest with Fernet, never hardcoded, never committed to source control, never exposed to the frontend, and never written to logs (a secret-redacting log filter enforces the last point).

### 3.4.3 Limitations

Several limitations are deliberate v1 scope boundaries rather than defects, and are stated plainly. First, although the formal anomaly taxonomy enumerates 46 categories, v1 ships the *config-only core*: 14 deterministic detectors (plus one CTI-driven `threat_exposure` finding) are implemented, all emitting `detection_mode = "config_only"`;

runtime-dependent modes (`conditional`, `simulated`, `future_enhanced`) are defined in the schema but not yet emitted, because v1 has no live telemetry. Second, the platform performs *no live traffic analysis* — no NetFlow, packet capture, session analysis, or deep packet inspection — and the policy graph therefore represents logical, configured intent only, never observed packet forwarding. Third, the dark-web / LTI subsystem is present in the codebase but disabled by default and treated as optional future research. Fourth, the Docker Compose deployment runs a single Celery worker that drains all routed queues; the specification’s recommended horizontal split into per-queue workers is not yet provisioned. Fifth, the backend `requirements.txt` is unpinned, which is a build-reproducibility caveat worth noting for formal documentation.

### 3.4.4 Dependencies

LuminaFPM depends on a Python 3.11 / FastAPI backend, a Celery worker pool, Redis (Celery broker), and PostgreSQL 16 (operational store and Celery result backend), all orchestrated by Docker Compose. The backend depends on `requests` for FortiGate REST calls and `defusedxml` for hardened PAN-OS XML parsing, on `cryptography` (Fernet) for credential encryption, and on a multi-provider LLM stack (Google Gemini SDK by default, with offline fallback). The frontend depends on React 19, React Router 7, Vite, Recharts, and the Cytoscape graph stack. Externally, the platform depends on the managed firewalls’ management APIs being reachable, on third-party CTI API services when CTI is enabled, and on the configured LLM provider’s API when online reporting is used (the offline provider removes this dependency for deterministic, network-free reports).

## 3.5 Intended Technologies

The technology stack is determined directly by the dependency manifests — `backend/requirements.txt`, `frontend/package.json`, and `docker-compose.yml` — and by the build images `backend/Dockerfile` (`python:3.11-slim`) and `frontend/Dockerfile` (`node:22-slim`). Backend dependencies are unpinned (resolved at image-build time); frontend dependencies are pinned with caret ranges. Table 3.2 maps each architectural layer to the technologies that realize it and the role each plays.

**Table 3.2.** *Intended technology stack by architectural layer.*

| Layer               | Technology               | Role                                                                                               |
|---------------------|--------------------------|----------------------------------------------------------------------------------------------------|
| API / web framework | FastAPI + Uvicorn (ASGI) | REST API surface (routers, dependency injection, OpenAPI); served by Uvicorn.                      |
| Data access / ORM   | SQLAlchemy + Alembic     | ORM models and sessions; schema owned by Alembic migrations (no runtime <code>create_all</code> ). |

| Layer                | Technology                                                                             | Role                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Validation / config  | Pydantic + pydantic-settings                                                           | Request/response schemas, normalized-model validation, typed env configuration.                         |
| Database             | PostgreSQL 16                                                                          | Single authoritative source of truth (Schema v4); Celery result backend.                                |
| Task queue           | Celery ( <code>celery[redis]</code> )                                                  | Async acquisition / normalization / analysis / CTI / reporting jobs on routed queues.                   |
| Broker / cache       | Redis 7                                                                                | Celery broker and SSE Pub/Sub transport.                                                                |
| Firewall acquisition | <code>requests</code> (FortiOS REST/JSON)                                              | Read-only HTTP client for FortiGate <code>cmdb/monitor</code> GET endpoints.                            |
| Firewall acquisition | <code>defusedxml</code> (PAN-OS XML)                                                   | Hardened XML parsing of Palo Alto <code>keygen/config show/op</code> responses.                         |
| Credential security  | <code>cryptography</code> (Fernet)                                                     | AES-128-CBC + HMAC encryption of device secrets at rest.                                                |
| LLM / AI             | <code>google-generativeai</code> (Gemini) + LangChain bindings                         | Default LLM provider for CTI query/review and SOC reporting; offline fallback for deterministic output. |
| Streaming            | <code>sse-starlette</code>                                                             | Server-Sent Events for streaming progress / LLM tokens to the UI.                                       |
| Frontend framework   | React 19 + TypeScript + Vite                                                           | Single-page dashboard application; Vite dev server and build tool.                                      |
| Routing              | React Router 7                                                                         | Client-side routing across the Centers.                                                                 |
| Charts / graph       | Recharts + Cytoscape ( <code>cytoscape-dagre</code> , <code>react-cytoscapejs</code> ) | Dashboard charts and the logical-policy graph view.                                                     |
| Styling / icons      | Tailwind CSS 4 + <code>lucide-react</code>                                             | Utility-first styling and icon set.                                                                     |
| Orchestration        | Docker Compose + Nginx                                                                 | Multi-service deployment; Nginx prepared for production static serving.                                 |

## 3.6 Functional Requirements

The functional requirements (FRs) below define the behaviours the platform must exhibit. They are organized by pipeline layer and prioritized as *Must* (mandatory for v1), *Should* (important but degradable), or *Could* (desirable / future-leaning). Each FR is traceable to a layer described in Section 3.2. Table 3.3 enumerates FR-01 through FR-27.

**Table 3.3.** *Functional requirements (FR-01 – FR-27).*

| <b>ID</b> | <b>Requirement</b>                                                                                                                                                                                                            | <b>Priority</b> |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| FR-01     | The system shall acquire firewall configuration from FortiGate over the FortiOS REST/JSON API using read-only endpoints only.                                                                                                 | Must            |
| FR-02     | The system shall acquire firewall configuration from Palo Alto over the PAN-OS XML API ( <code>keygen</code> , <code>config show</code> , <code>op</code> ) using read-only operations only.                                  | Must            |
| FR-03     | The system shall persist each raw vendor response as a <code>RawArtifact</code> on disk with a SHA-256 manifest, and record artifact metadata in the database for replay and audit.                                           | Must            |
| FR-04     | The system shall run all acquisition asynchronously on Celery workers and never inside the FastAPI request cycle, returning 202 <code>Accepted</code> with a job identifier.                                                  | Must            |
| FR-05     | The system shall track each acquisition job through a defined lifecycle ( <code>queued</code> , <code>running</code> , <code>success</code> , <code>partial_success</code> , <code>failed</code> , and error sub-states).     | Must            |
| FR-06     | The system shall parse raw FortiGate JSON and Palo Alto XML into vendor-tagged parser models without performing any anomaly logic.                                                                                            | Must            |
| FR-07     | The system shall normalize vendor parser models into a single canonical rule/object/service model, mapping vendor actions to canonical actions while preserving original vendor values.                                       | Must            |
| FR-08     | The system shall build an object correlation model that preserves vendor-owned objects and links them to canonical objects via mapping tables.                                                                                | Must            |
| FR-09     | The system shall persist normalized data idempotently into PostgreSQL keyed on ( <code>device_id</code> , <code>vdom_vsys</code> , <code>vendor_uuid</code> ), supporting incremental re-synchronization.                     | Must            |
| FR-10     | The system shall run the deterministic anomaly engine over the normalized repository only, never over raw vendor data.                                                                                                        | Must            |
| FR-11     | The system shall detect single-rule anomalies including over-permissive rules, any-to-sensitive access, unprotected allows, wide port ranges, missing logging, missing descriptions, disabled-rule review, and object sprawl. | Must            |
| FR-12     | The system shall detect intra-device pairwise anomalies including shadowing, redundancy, duplicate rules, and conflicts.                                                                                                      | Must            |
| FR-13     | The system shall produce deterministic, repeatable findings with a fixed detector order and a stable rule sort (no randomness, network, or LLM in the engine).                                                                | Must            |

| ID    | Requirement                                                                                                                                                                                            | Priority |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-14 | Every finding shall carry a full evidence contract: anomaly type, severity, confidence, description, evidence facts, recommendation, detection mode, and affected/related rules.                       | Must     |
| FR-15 | The system shall detect cross-device anomalies (cross-device inconsistency and cross-device security-posture inconsistency) by analyzing all devices together.                                         | Must     |
| FR-16 | The system shall compute a 0–100 risk score and tier (Critical/High/Medium/Low/Informational) per rule and per device, persisting <code>risk_assessment</code> rows while retaining cross-run history. | Must     |
| FR-17 | The system shall trigger risk scoring best-effort at the end of each anomaly run such that a risk failure never fails the analysis.                                                                    | Must     |
| FR-18 | The system shall extract indicators (public IPs, FQDNs, firmware/CVEs) and enrich them via external CTI APIs, correlating threat observations back to rules.                                           | Should   |
| FR-19 | The system shall update risk scores from CTI verdicts and label CTI-derived findings with <code>detection_mode = "cti"</code> .                                                                        | Should   |
| FR-20 | The system shall generate AI-assisted, evidence-grounded SOC/executive reports and CTI queries via a provider abstraction, with a deterministic offline fallback.                                      | Should   |
| FR-21 | The LLM layer shall never act as an anomaly authority; AI output is bounded to query generation, CTI review, and reporting.                                                                            | Must     |
| FR-22 | The system shall provide a benchmark framework that scores engine findings against documented ground truth (precision, recall, F1) as an acceptance gate.                                              | Should   |
| FR-23 | The system shall expose read-only REST endpoints for jobs, devices, normalized rules/objects, anomalies, risk, CTI, benchmark, and reports.                                                            | Must     |
| FR-24 | The system shall present a React dashboard with Policy Explorer and Anomaly/Risk/CTI/Benchmark Centers plus a logical-policy graph that never depicts live traffic.                                    | Must     |
| FR-25 | The system shall allow operators to create, rotate, and test device credentials, returning status only and never exposing secrets to the frontend.                                                     | Must     |
| FR-26 | The system shall never write, push, delete, reorder, enable, disable, or commit any firewall configuration (strict read-only enforcement).                                                             | Must     |
| FR-27 | The system shall negotiate transport scheme per device (HTTPS by default; HTTP only for explicitly configured lab hosts) while keeping all requests read-only.                                         | Should   |

## 3.7 Non-Functional Requirements

The non-functional requirements (NFRs) constrain *how well* the platform must satisfy its functional behaviours. They are grouped by quality attribute below and summarized in Table 3.4.

### 3.7.1 Security

Security is the platform’s defining quality attribute. The read-only mandate is a security control as much as a functional one: by holding only read-only poll keys and containing no write code path, even a compromised LuminaFPM instance cannot alter a firewall. Device secrets are encrypted at rest with Fernet (AES-128-CBC + HMAC), decrypted only in backend/worker memory, never returned to the frontend, and never logged — a secret-redacting log filter masks API keys, tokens, passwords, and bearer headers. CORS is a restricted allowlist (wildcard-with-credentials is rejected), and PostgreSQL and Redis are not published to the host (internal Docker network only).

### 3.7.2 Reliability and Availability

The asynchronous pipeline is tuned for crash-safety: Celery uses late acknowledgement (`task_acks_late`) so a task is acknowledged only after completion, a prefetch multiplier of one to avoid greedy prefetch, and PostgreSQL as a durable result backend. Acquisition tolerates partial failure (`partial_success`) when an optional endpoint fails, and risk scoring is best-effort so it cannot fail an analysis run. The job state machine and immutable per-run logs make failures observable and recoverable.

### 3.7.3 Performance

Per-job-type queue routing ensures slow firewall polling never blocks analysis, CTI, or reporting. The anomaly engine is purely in-memory over the normalized model and uses stable sorting and pairwise combinations, giving predictable, deterministic runtimes. The threat-intel scan path enforces hard and soft time limits (1800 s / 1500 s) to bound worst-case execution.

### 3.7.4 Maintainability and Extensibility

The strict layering and the connector/parser registry pattern make the system extensible: a new vendor is added by implementing a connector and a parser behind the existing interfaces, without touching the analytical core. Schema evolution is managed by Alembic migrations. The LLM and CTI providers sit behind abstractions (`LLMProvider`, `CTIProvider`), allowing providers to be swapped via configuration.

### 3.7.5 Usability

Findings are explainable by construction: every finding carries the concrete evidence that produced it and a remediation recommendation, so analysts can act on results without re-deriving them. The dashboard organizes work into purpose-built Centers, and the logical-policy graph is clearly labelled as configured intent rather than live traffic to avoid misinterpretation.

### 3.7.6 Portability

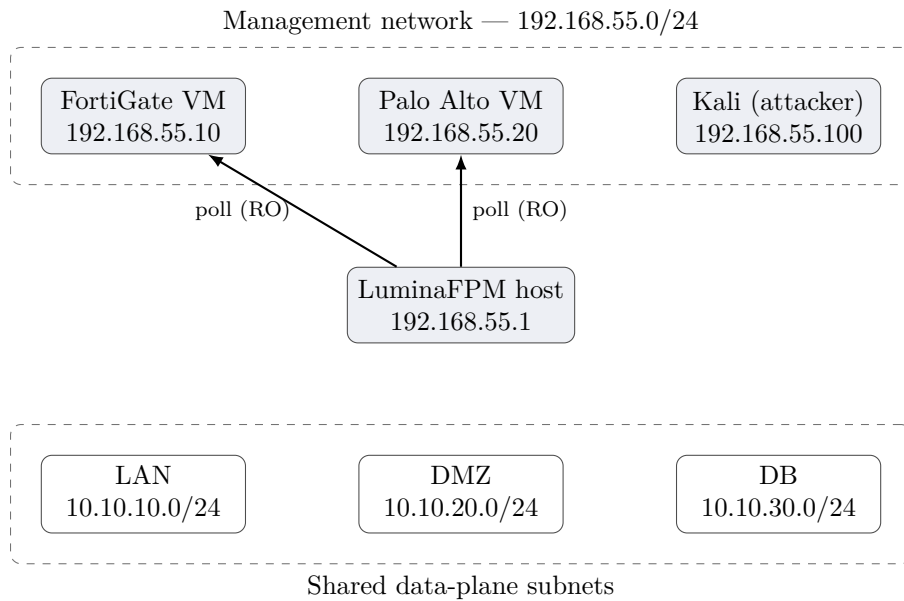
The entire stack is containerized via Docker Compose and built from standard base images (`python:3.11-slim`, `node:22-slim`), so it can be reproduced on any Docker-capable host. Device-local state (the `.env` file, the `postgres_data` volume, and the lab network) is documented as non-transferable, and the credential-encryption key derivation tolerates either a pre-generated Fernet key or a passphrase.

**Table 3.4.** *Non-functional requirements summary.*

| ID     | Attribute       | Requirement                                                                                                                  |
|--------|-----------------|------------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Security        | The platform shall be strictly read-only and shall hold no firewall write capability anywhere in its trust boundary.         |
| NFR-02 | Security        | Device credentials shall be encrypted at rest (Fernet), decrypted only in memory, and never exposed to the frontend or logs. |
| NFR-03 | Security        | The API shall enforce a CORS allowlist and shall not publish the database or cache to the host network.                      |
| NFR-04 | Reliability     | Tasks shall acknowledge only after completion and persist results durably so that worker crashes do not lose work.           |
| NFR-05 | Reliability     | Acquisition shall degrade gracefully to partial success, and risk scoring shall never fail an analysis run.                  |
| NFR-06 | Performance     | Firewall polling shall not block analysis, CTI, or reporting, via per-job-type queue routing.                                |
| NFR-07 | Performance     | The anomaly engine shall be deterministic and in-memory, with bounded, repeatable runtimes.                                  |
| NFR-08 | Maintainability | New vendors shall be addable via connector/parser extensions without modifying the anomaly engine.                           |
| NFR-09 | Maintainability | Database schema changes shall be managed exclusively through Alembic migrations.                                             |
| NFR-10 | Usability       | Every finding shall be explainable, carrying evidence facts and a remediation recommendation.                                |
| NFR-11 | Portability     | The full stack shall be reproducible via Docker Compose on any Docker-capable host.                                          |

### 3.8 Virtual Environment and Laboratory Topology

LuminaFPM was developed and validated against a controlled, parallel, shared-network **VMware laboratory** that hosts both v1 firewalls. The lab follows the *Hybrid Parallel Shared-Network* model: the FortiGate and the Palo Alto firewall both connect to the same logical subnets but operate as **independent policy engines** — they are not inline, and no production traffic is routed from one through the other. This model is deliberately chosen because it is the cleanest environment for controlled cross-vendor anomaly detection under *identical* logical network conditions: it avoids unnecessary routing complexity, supports clean API extraction from each device, and enables benchmark-driven evaluation of the deterministic engine. The lab is primarily a controlled anomaly-*generation* environment, not a realistic enterprise routing simulation. Figure 3.2 shows the topology, its subnets, and the device IP plan.



**Figure 3.2.** *Parallel shared-network virtual laboratory topology. LuminaFPM polls both firewalls read-only over the management network; the firewalls independently govern the shared data-plane subnets.*

#### Networks and IP plan

The lab is partitioned into one management/API network and three data-plane subnets:

- **ADMIN/API network** 192.168.55.0/24 — management and read-only API access. FortiGate management interface is 192.168.55.10, Palo Alto management is 192.168.55.20, the Kali / API host is 192.168.55.100, and the VMware host adapter is 192.168.55.1. LuminaFPM reaches both firewalls only over this network.
- **LAN** 10.10.10.0/24 — internal user network (FortiGate port1 10.10.10.1; Palo Alto eth1/1 10.10.10.2, zone trust).

- **DMZ** 10.10.20.0/24 — public-services segment (FortiGate port2 10.10.20.1; Palo Alto eth1/2 10.10.20.2, zone **dmz**).
- **DB** 10.10.30.0/24 — database segment (FortiGate port3 10.10.30.1; Palo Alto eth1/3 10.10.30.2, zone **db**).

## Read-only poll accounts versus the provisioner's write keys

The lab makes the read-only boundary structural rather than merely procedural by *splitting credentials across trust boundaries*. LuminaFPM is configured with **read-only poll accounts only** — a dedicated FortiGate read-only API token and a read-only Palo Alto API key (or user/password), stored encrypted in the platform. The **write-capable admin keys** that build the benchmark rule bases live entirely with the parallel-lab provisioning tooling under `lab/` (`provision_benchmark.py`, `benchmark_dataset.py`), *outside* LuminaFPM's trust boundary. That tooling is operator write tooling, not part of the read-only platform; it defaults to `-dry-run` and requires both `-apply` and `-confirm` (plus separate write tokens) to push policy. Consequently, even a fully compromised LuminaFPM instance holds no key capable of altering a firewall. After the provisioner writes the benchmark policy set (13 FortiGate rules and 8 Palo Alto rules with documented expected anomalies and cross-device pairs), LuminaFPM read-syncs the devices and runs the engine; the recorded live lab run covered 2 devices, 20 rules, and 54 findings, with benchmark precision/recall/F1 of 100% (47/47).

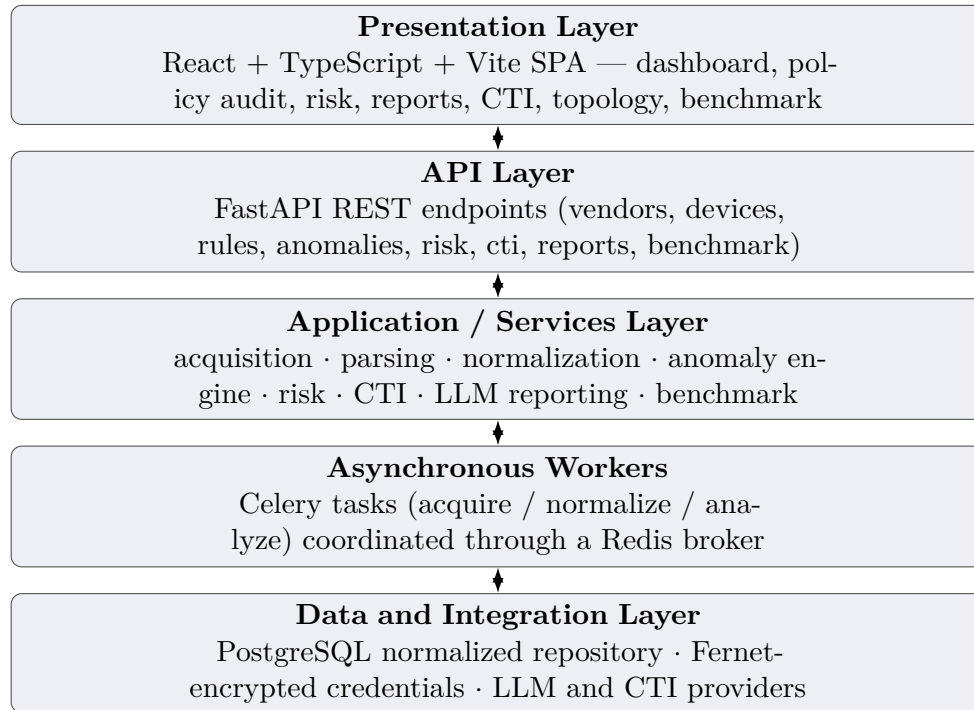
## FortiGate-VM constraints

Two lab-specific FortiGate-VM constraints shaped the implementation. First, the evaluation FortiGate-VM enforces a **10-policy cap**, which bounds the size of the benchmark rule set that can be provisioned on that device. Second, the FortiGate-VM's HTTPS admin service (`httpsd`) does not reliably bind port 443 in the lab, so its REST API is **not reachable over HTTPS**. To accommodate this without weakening production behaviour, acquisition negotiates the transport scheme per device: HTTPS by default, falling back to plain **HTTP only** for hosts explicitly listed in the `FIREWALL_INSECURE_HTTP_HOSTS` setting (`scheme = "http" if device.management_ip in settings.firewall_insecure_http_hosts else "https"`). This escape hatch applies to FortiGate only — the Palo Alto connector hardcodes HTTPS — and the configuration explicitly warns that it must never be enabled in production, where API tokens would otherwise traverse the network unencrypted.

## 3.9 System Architecture

LuminaFPM realises the *Hybrid Parallel Shared-Network Architecture* as a strictly layered system in which no layer bypasses the layer beneath it: each layer owns a single responsibility and hands a well-defined, vendor-neutral data structure to the next. The architecture

is partitioned, both logically and at runtime, into five concerns: a presentation/API tier, an application/services tier, an asynchronous worker tier, a data/repository tier, and an integration tier that mediates all contact with external systems. Figure 3.3 depicts these tiers and the unidirectional flow of policy intent through them.



**Figure 3.3.** Layered system architecture of LuminaFPM.

The **presentation and API tier** is split across two cooperating processes. The user-facing surface is a React + TypeScript + Vite single-page application (the frontend container) that renders the operational Centers — Dashboard, Device Inventory, Policy Explorer, Anomaly Center, Risk Center, CTI Threat Center, Benchmark Center, Graph View and Reports — over a read-only HTTP contract. The server-facing surface is a single FastAPI application (`backend/main.py`, `title="LuminaFPM Backend API"`) that mounts one router per resource under the base prefix `/api/v1`. The API tier performs only fast, transactional work: it validates a request, inserts a job or log row, enqueues a Celery task, and otherwise serves reads from PostgreSQL. A hard architectural rule (Volume 2 §6.5) forbids any firewall polling inside the web request cycle; consequently every long-running operation returns `202 Accepted` with a `job_id` or `task_id` rather than blocking. Cross-Origin Resource Sharing is constrained to a restricted allowlist (`settings.cors_allowed_origins`, default `http://localhost:5173`); the wildcard-with-credentials combination is explicitly rejected.

The **application/services tier** is the deterministic core of the platform. It is realised as a set of pure Python packages under `backend/services/`: acquisition, parsing, normalization, anomaly, risk, cti, llm and benchmark. With the single exception of the acquisition connectors, these services are DB-light or DB-free and contain no I/O against firewalls — they operate over in-memory model objects (chiefly `NormalizationResult`

and **Finding**). This purity is deliberate: it makes the analytical pipeline deterministic, unit-testable in isolation, and free of hidden network dependencies.

The **asynchronous worker tier** executes every operation that is slow, fallible, or contacts an external system. A single Celery application (`Celery("lumina_fpm")`) uses Redis as its broker and PostgreSQL as its result backend, with crash-safe tuning (`task_acks_late=True`, `worker_prefetch_multiplier=1`). Work is routed onto per-job-type queues (`acquisition`, `normalization`, `analysis`, `cti`, `reporting`) so that slow firewall polling never blocks analysis or reporting. The canonical pipeline chains automatically across these queues: `acquisition` → `normalization` → `analysis` → `risk`, each stage enqueueing the next on a different queue. Each worker task opens its own SQLAlchemy session because the worker is a distinct process.

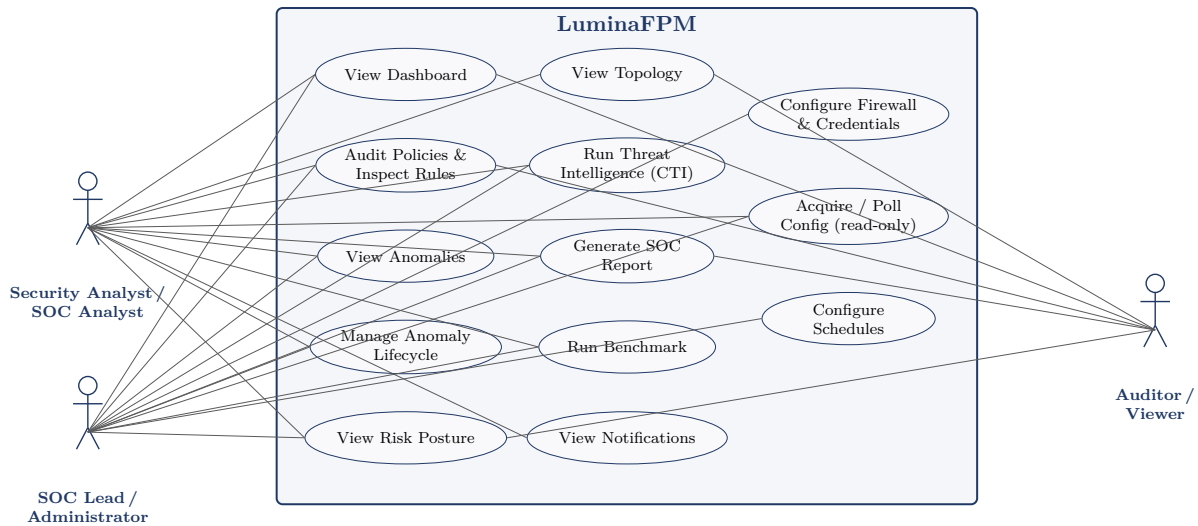
The **data/repository tier** is a single PostgreSQL 16 instance — the v1 source of truth (ADR-008; MongoDB is explicitly excluded by ADR-021). Its schema (Schema v4) is owned by Alembic migrations rather than `Base.metadata.create_all`, which is deliberately retired from the runtime path so that no destructive auto-synchronisation can occur. The repository holds all normalized policy data, anomaly findings, risk assessments, CTI indicators and observations, benchmark cases and results, and AI reports. The anomaly engine never reads raw vendor output; it consumes only a `NormalizationResult` reconstructed from this repository by `normalization.loader.load_normalized_result` (ADR-010).

The **integration tier** encapsulates every boundary crossing. Inward, the acquisition connectors issue read-only requests to the two lab firewalls over the administrative network 192.168.55.0/24. Outward, the CTI providers and the LLM provider call third-party HTTP APIs. The read-only mandate (§3.11 and ADR-006) is enforced structurally at this boundary: connectors invoke only `GET`, `show`, `monitor` and `keygen/op` read endpoints; there is no write, `set`, `commit` or `delete` code path anywhere in the platform. Device secrets are stored encrypted (Fernet) in the `device_credential` table and are decrypted only in worker memory, never returned to the frontend and never logged.

## 3.10 Use Case Diagram

LuminaFPM serves a small set of human actors together with two non-human actors that participate in the platform’s automated behaviour. The human actors are distinguished by responsibility rather than by an implemented role-based access-control layer; the route handlers in the current snapshot carry no per-endpoint authorization dependency, so the actor partition below is an *intended* operational model rather than a code-enforced one. Figure 3.4 summarises the actors and their use cases.

The **Security Analyst** is the primary day-to-day actor. The analyst registers FortiGate and Palo Alto devices, configures and tests their encrypted credentials, and triggers a read-only poll of a device. Once the pipeline completes, the analyst browses the normalized Policy Explorer, reviews anomaly findings in the Anomaly Center, inspects the per-rule



**Figure 3.4.** Use case diagram of LuminaFPM.

and per-device risk scores in the Risk Center, examines the CTI Threat Center, requests an evidence-grounded SOC report, and explores the logical policy graph. The analyst also owns the anomaly lifecycle: a finding may be moved from **open** to **resolved**, **suppressed**, **accepted\_risk** or **false\_positive**, with a mandatory justification on the suppressing transitions.

The **SOC Administrator** is a superset actor who, in addition to the analyst's use cases, manages platform-wide configuration: administrator records, device-to-administrator assignments, CTI and LLM provider keys, and the triggering of all-scope (cross-device) analysis runs. The cross-device run is significant because the platform's signature cross-vendor detectors require all devices to be analysed together.

The **Compliance Auditor** is a read-mostly actor focused on assurance. The auditor consumes the Benchmark Center, where the deterministic engine's output is scored against an authoritative ground-truth matrix (precision, recall, F1, severity-match rate), and reads the immutable history of analysis runs, findings, risk assessments and AI reports. The auditor's principal interest is the platform's explainability and reproducibility guarantee: every finding cites the affected rule, the related rule where applicable, and the concrete evidence that produced it.

Two non-human actors complete the model. The **Acquisition Scheduler** is the actor on whose behalf jobs may be created without a human request (`AcquisitionJob.requested_by` may be "scheduler"); it drives periodic re-polling of devices to keep the normalized repository fresh. The **LLM / CTI Subsystems** are external-facing actors invoked by the platform: the CTI providers (the always-present offline provider plus optionally AbuseIPDB, OTX or VirusTotal when keyed) supply threat verdicts for extracted indicators, and the LLM provider (Google Gemini by default, with a deterministic offline fallback) generates the SOC and executive reports. The key use cases are enumerated below.

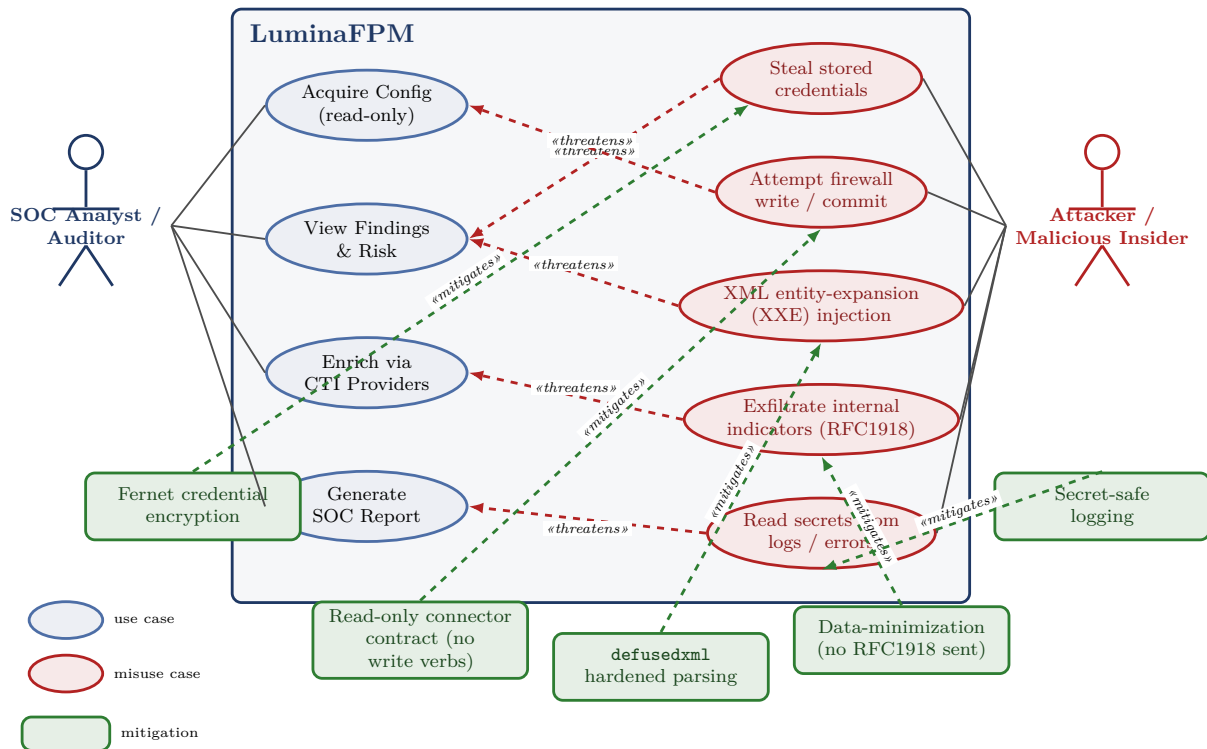
- **Security Analyst:** register/edit device; configure and test credential; trigger device poll; browse normalized policies; review anomalies; inspect rule/device risk; view CTI

Threat Center; generate SOC report; explore logical policy graph; manage anomaly lifecycle (resolve / suppress / accept-risk / mark false-positive).

- **SOC Administrator:** all analyst use cases; manage administrators and device assignments; manage provider keys; trigger all-scope cross-device analysis; recalculate risk and CTI.
- **Compliance Auditor:** run and read the benchmark acceptance report; review analysis-run history; read findings, risk and AI reports; verify evidence traceability.
- **Acquisition Scheduler (system actor):** create scheduled acquisition jobs; keep the normalized repository current.
- **LLM / CTI Subsystems (system actors):** enrich indicators with provider verdicts; generate evidence-grounded reports.

### 3.11 Misuse Case Diagram

Because LuminaFPM holds firewall-management credentials and reaches both internal infrastructure and external intelligence APIs, its threat surface is documented through misuse cases: the negative-space counterpart of the use-case model. Each misuse case pairs an abuser goal with the threat actor, the attack vector, and the countermeasure the built system actually implements. Figure 3.5 overlays these misuse cases on the legitimate actors, and Table 3.5 enumerates them.



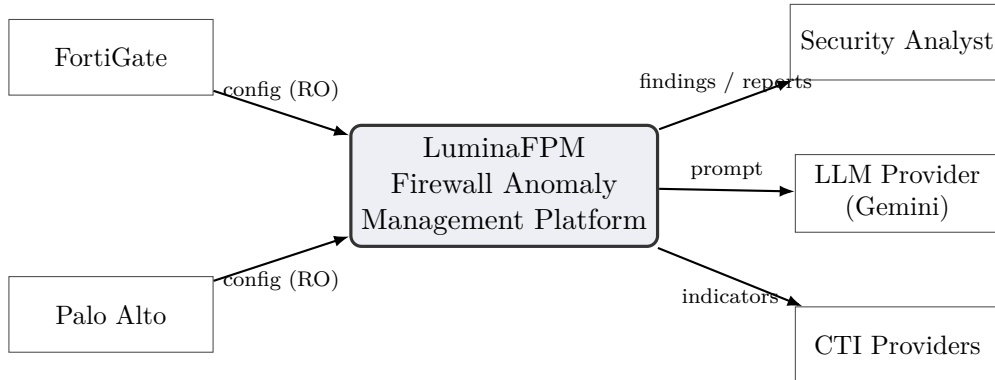
**Figure 3.5.** Misuse case diagram of LuminaFPM.

**Table 3.5.** *Misuse cases, threat actors, attack vectors and implemented countermeasures.*

| Misuse case                                | Threat actor                                          | Attack vector                                                                                      | Countermeasure (as built)                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Credential theft                           | External attacker / malicious insider                 | Reading stored firewall secrets from the database or API responses to obtain device API keys       | Per-device secrets stored as Fernet ciphertext in <code>device_credential.secret_encrypted</code> ; decrypted only in backend/worker memory via <code>credentials.get_decrypted_secret</code> ; never serialised to the frontend, never logged; the frontend receives credential <i>status</i> only.                                                       |
| Tampering with the read-only boundary      | Malicious insider / compromised platform              | Coercing the platform to push, commit, re-order or disable a firewall rule                         | No write/ <code>set/commit/delete</code> code path exists; connectors call only read endpoints (FortiGate <code>cmdb/monitor</code> GETs, PAN-OS <code>keygen/config show/op</code> ). Firewall <i>write</i> keys live with the lab provisioner, outside Lumina’s trust boundary.                                                                          |
| Indicator leakage of RFC1918 space         | External CTI provider / network eavesdropper          | Exfiltrating internal addressing by submitting private IPs to third-party threat APIs              | Data minimisation in <code>cti/extract.py</code> : <code>is_public_ip</code> returns <code>net.is_global</code> , so RFC1918, loopback, link-local and reserved addresses are private; <code>extract_indicators</code> skips non-public indicators unless <code>allow_internal</code> is explicitly set; the 0.0.0.0/0 ANY-sentinel is never an indicator. |
| SQL injection                              | External attacker                                     | Crafted input through API query parameters to subvert repository queries                           | All persistence flows through SQLAlchemy ORM constructs and parameterised queries; no string-concatenated SQL in the route or service layers; inputs validated by Pydantic schemas.                                                                                                                                                                        |
| LLM prompt injection                       | Adversarial firewall configuration / external content | Malicious rule descriptions or object names steering the LLM to fabricate findings or remediations | Guardrail system prompt ( <code>PROMPT_VERSION="soc-report/2.1"</code> ): the model must use <i>only</i> supplied evidence, must not invent CVEs, exploit claims or reputations, and must separate evidence from interpretation. The deterministic engine — not the LLM — decides what is an anomaly; the LLM only explains and prioritises.               |
| Token / secret exposure in transit or logs | Eavesdropper / log reader                             | Harvesting provider keys or device tokens from logs, error messages or job records                 | Structured, secret-safe logging ( <code>core/logging.py</code> ); <code>AcquisitionJob.error_message</code> stores a safe summary with no secrets; <code>db</code> and <code>redis</code> are bound <code>expose-only</code> on the internal Docker network, not published to the host.                                                                    |

## 3.12 Context Diagram

At the highest level of abstraction LuminaFPM is a single process that sits between the firewalls it observes and the people and services that consume its analysis. The context diagram (Figure 3.6) treats the entire platform as one bubble and enumerates the external entities that exchange data with it across the integration boundary.



**Figure 3.6.** Context diagram: LuminaFPM and its external entities. All firewall interaction is strictly read-only.

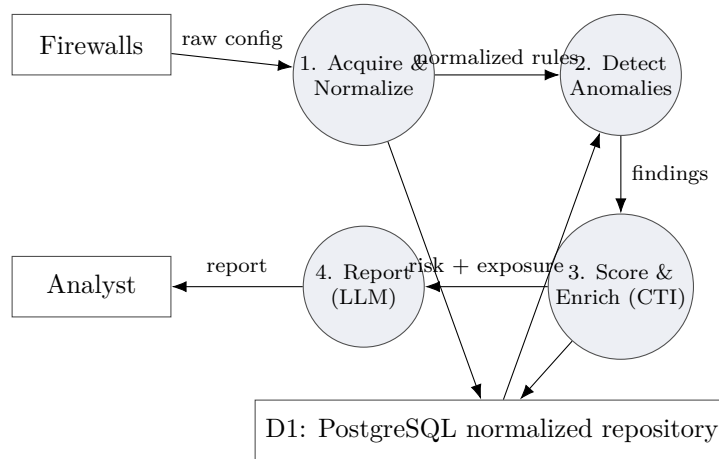
The two **firewall devices** are the upstream data sources. The FortiGate device (192.168.55.10) exposes a FortiOS REST/JSON API under `/api/v2/cmdb/firewall/*` and `/api/v2/monitor/system/status`; the Palo Alto device (192.168.55.20) exposes the PAN-OS XML API (`type=keygen`, `type=config&action=show`, `type=op`). Both are contacted strictly read-only, over HTTPS by default and over HTTP only for hosts in the documented lab escape-hatch list. They return raw configuration — security rules, address and address-group objects, service and service-group objects, and device metadata — and receive nothing but read requests in return.

The **Security Analyst** (standing in for the human actors of §3.10) is the principal downstream consumer: a browser client that drives the platform through the read-only `/api/v1` contract and receives normalized policies, findings, risk scores, CTI results, reports and the logical graph.

Two classes of third-party service are downstream of the platform’s enrichment stages. The **LLM provider** (Google Gemini at `generativelanguage.googleapis.com`, with a network-free offline fallback) receives an evidence-grounded prompt and returns generated report prose. The **CTI providers** (AbuseIPDB at `api.abuseipdb.com`, optionally OTX and VirusTotal, alongside the always-present offline provider) receive *only public* indicators and return threat verdicts. Both external services are invoked exclusively from the asynchronous worker tier, never from the web request cycle, and both are reached over outbound HTTPS that carries no firewall secrets and no private addressing.

### 3.13 Data Flow Diagram

The data-flow view decomposes the platform into the six processing stages that transform raw firewall configuration into scored, enriched, narrated intelligence, together with the data stores that persist each intermediate result. Figure 3.7 shows these processes, the external entities of §3.12, and the flows between them.



**Figure 3.7.** Data flow diagram of the LuminaFPM analysis pipeline.

**Process 1 — Acquire.** The acquisition connectors authenticate to a firewall, poll its read-only endpoints, and persist each response as a raw artifact on disk under `/raw_acquisition/{vendor}/{device}/{job}/` with a SHA-256 manifest. The flow is governed by an `AcquisitionJob` lifecycle (`queued` → `running` → `success/partial_success/failure` states). *Data stores written:* `acquisition_job`, `raw_artifact` (plus the filesystem artifact store).

**Process 2 — Normalize.** The parser replays the stored artifacts into vendor-tagged `ParsedDevicePayload` objects, and the normalization engine (pure, DB-free) maps them into one canonical model — canonical actions, canonical objects and services — while preserving vendor traceability through the correlation-model mapping tables. The thin persist layer performs an idempotent upsert keyed on `(device_id, vdom_vsys, vendor_uuid)`. *Data stores written:* `policy_rule`, `network_object`, `normalized_object`, `object_normalization_mapping`, `service_object`, `normalized_service`, `service_normalization_mapping`, `rule_object_mapping`, `rule_service_mapping`.

**Process 3 — Detect.** The deterministic anomaly engine reconstructs a `NormalizationResult` from the repository and runs the fixed-order detector set over it, producing explainable findings (no randomness, no network, no LLM). *Data stores written:* `anomaly_execution_log`, `rule_anomaly`.

**Process 4 — Score.** The risk scorer aggregates each rule’s findings into a 0–100 score with a factor breakdown, then blends the worst rules into a device-level score. *Data store written:* `risk_assessment`.

**Process 5 — Enrich.** The CTI runner extracts public indicators from the network objects, queries the CTI providers, records the worst verdict per indicator, and correlates malicious indicators back to allow-rules across all devices as `threat_exposure` findings, before recomputing risk so the CTI factor lands. *Data stores written:* `cti_indicator`, `cti_observation`, additional `rule_anomaly` (`detection_mode="cti"`).

**Process 6 — Report.** The LLM reporter assembles an evidence-reference bundle (anomaly IDs, risk ID, CTI observation IDs), calls the provider under the guardrail prompt, and always persists the result. *Data store written:* `llm_report`.

A parallel benchmark process reads `rule_anomaly` (filtered to `config_only` findings) and the ground-truth matrix to write `benchmark_case` and `benchmark_result`, providing the acceptance gate that the auditor of §3.10 consumes.

## 3.14 Entity Relationship Diagram

The persistence layer is a single relational schema (Schema v4) defined in `backend/models/models.py` and owned by Alembic migrations. Its design embodies the platform’s central idea: vendor-owned records are preserved verbatim, a canonical layer sits above them, and explicit mapping tables connect the two so that cross-vendor equivalence is established through normalized correlation keys rather than raw vendor names. Figure 3.8 shows the core entities and their relationships.

### 3.14.1 Entity Descriptions

*Table 3.6. Core entities of the normalized Schema v4 and their salient attributes.*

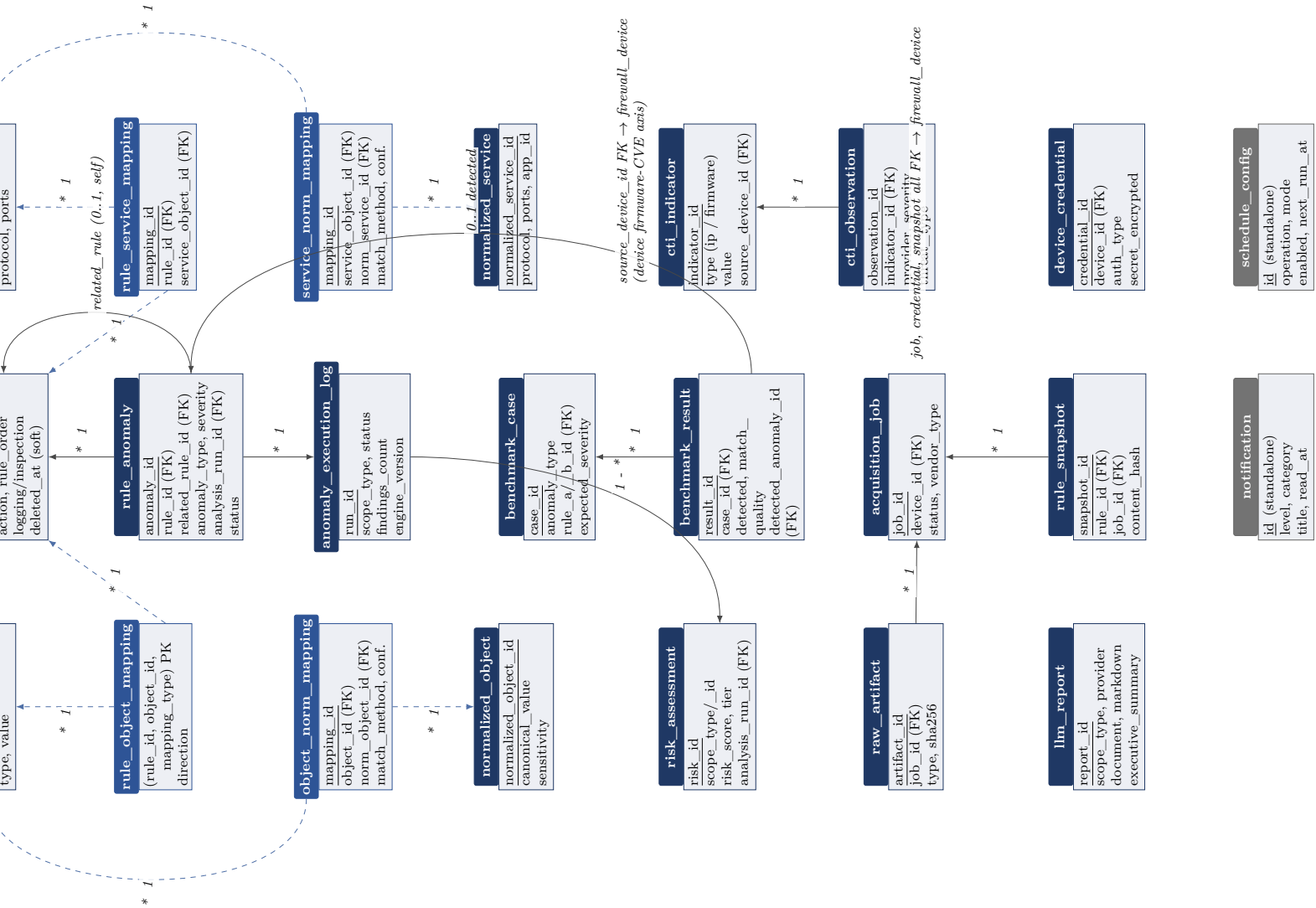
| Entity (table)                                   | Primary key                                            | Role and salient attributes                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>vendor</code>                              | <code>vendor_id</code>                                 | A supported firewall vendor; <code>name</code> , <code>api_type</code> . v1 instances are Fortinet and Palo Alto only.                                                                                                                                                                                                                                                        |
| <code>firewall_device</code>                     | <code>device_id</code>                                 | A managed device; <code>vendor_id</code> (FK), <code>vendor_type</code> ( <code>fortinet/paloalto</code> ), <code>hostname</code> , <code>management_ip</code> , <code>firmware_version</code> , <code>last_poll_time</code> , <code>status</code> .                                                                                                                          |
| <code>policy_rule</code>                         | <code>rule_id</code>                                   | A normalized security rule; <code>device_id</code> (FK), <code>vendor_uuid</code> , <code>vdom_vsys</code> , <code>rule_order</code> , <code>action</code> , tri-state posture fields ( <code>logging_enabled</code> , <code>security_inspection_enabled</code> , <code>schedule_scope</code> ), <code>normalized_content_hash</code> , <code>soft-delete deleted_at</code> . |
| <code>network_object</code>                      | <code>object_id</code>                                 | A vendor-owned address object; <code>device_id/vendor_id</code> (FK), <code>type</code> ( <code>ip_address/cidr/fqdn/group/any/...</code> ), <code>value</code> , <code>raw_value</code> .                                                                                                                                                                                    |
| <code>normalized_object</code>                   | <code>normalized_object_id</code>                      | A canonical object above vendor objects; <code>canonical_type</code> , <code>canonical_value</code> , <code>sensitivity</code> ( <code>admin/database/critical/...</code> ).                                                                                                                                                                                                  |
| <code>service_object / normalized_service</code> | <code>service_object_id / normalized_service_id</code> | Vendor-owned and canonical service/protocol/port/application records; <code>protocol</code> , <code>port_start</code> , <code>port_end</code> , <code>app_id</code> .                                                                                                                                                                                                         |

| Entity (table)                                                                  | Primary key                      | Role and salient attributes                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rule_object<br>_mapping /<br>rule_service<br>_mapping                           | composite /<br>mapping_id        | Associate a rule with its source/destination objects and its services; carry <code>mapping_type</code> and <code>direction</code> .                                                                                                                                                                                                           |
| object_<br>normalization<br>_mapping<br>/ service_<br>normalization_<br>mapping | mapping_id                       | Connect each vendor object/service to its canonical counterpart; carry <code>match_method</code> and <code>confidence</code> .                                                                                                                                                                                                                |
| rule_anomaly                                                                    | anomaly_id                       | A durable finding; <code>rule_id</code> and optional <code>related_rule_id</code> (both FK to <code>policy_rule</code> ), <code>anomaly_type</code> , <code>severity_level</code> , <code>confidence</code> , <code>evidence</code> (JSONB), <code>detection_mode</code> , <code>analysis_run_id</code> (FK), <code>lifecycle_status</code> . |
| anomaly_<br>execution_log                                                       | run_id                           | One engine run; <code>scope_type</code> , <code>status</code> , <code>findings_count</code> , <code>engine_version</code> .                                                                                                                                                                                                                   |
| risk_<br>assessment                                                             | risk_id                          | A rule-, device- or object-scope score; <code>risk_score</code> (0–100), <code>risk_tier</code> , <code>factor_breakdown</code> (JSONB), <code>analysis_run_id</code> (FK).                                                                                                                                                                   |
| cti_indicator<br>/ cti_<br>observation                                          | indicator_id /<br>observation_id | An extracted indicator and its per-provider verdicts; <code>indicator_type/value/is_public</code> ; <code>observation_provider</code> , <code>severity</code> , <code>confidence</code> , <code>threat_type</code> .                                                                                                                          |
| benchmark_case<br>/ benchmark_<br>result                                        | case_id / result_<br>id          | Ground-truth expected anomaly and its detected-versus-expected outcome; <code>case_type</code> , <code>expected_severity</code> , <code>detected</code> , <code>match_quality</code> , <code>false_negative</code> .                                                                                                                          |
| llm_report                                                                      | report_id                        | An evidence-grounded AI report; <code>scope_type</code> , <code>provider</code> , <code>model</code> , <code>prompt_version</code> , <code>evidence_refs</code> (JSONB), <code>output</code> , <code>status</code> .                                                                                                                          |

The schema also carries a `threat_exposure` concept: it is not a separate table but the `rule_anomaly` row of `anomaly_type="threat_exposure"` and `detection_mode="cti"` written by the CTI runner, linking a malicious `cti_indicator` to the allow-rule that references it.

### 3.14.2 Relationship Summary

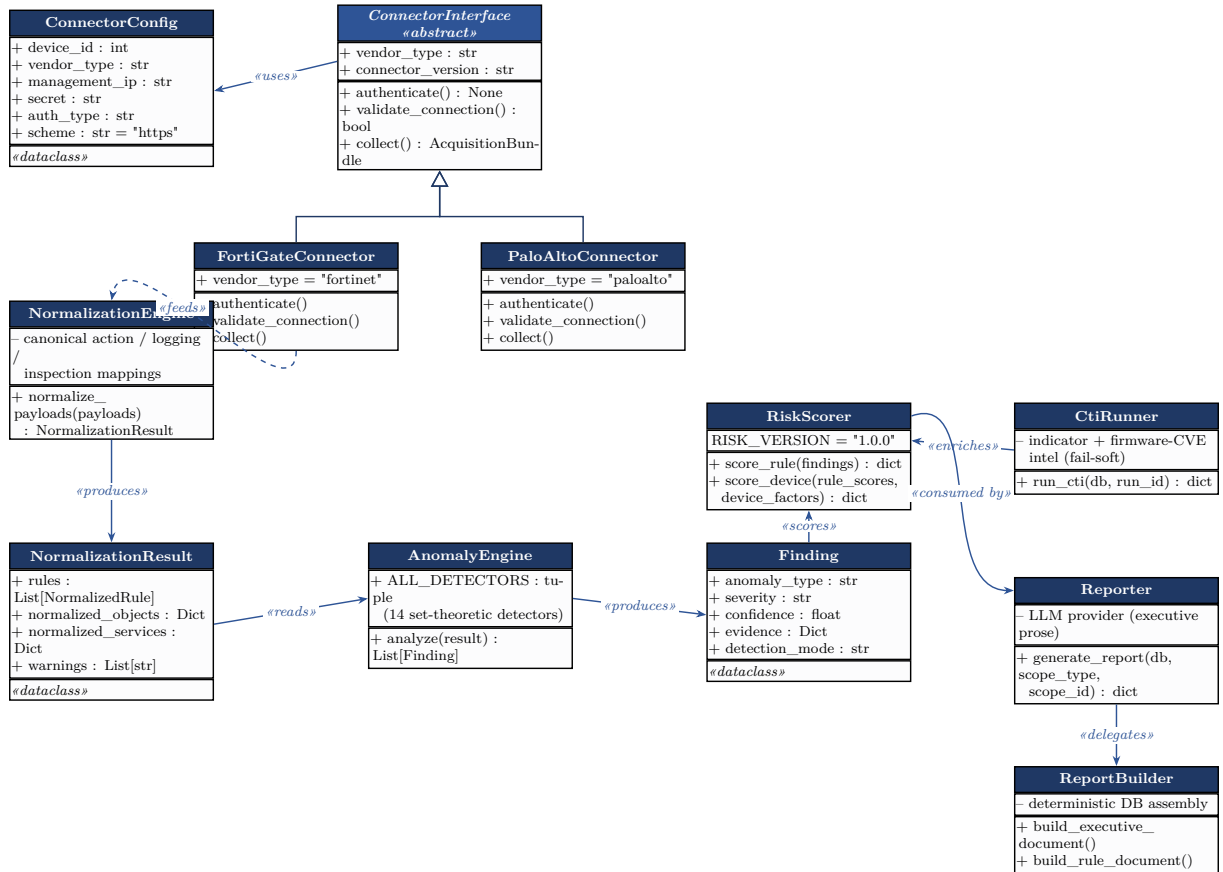
The relationships fall into three families. *Ownership* relationships are one-to-many: a vendor owns many `firewall_devices`; a `firewall_device` owns many `policy_rules`, `network_objects` and `service_objects`. *Correlation* relationships are many-to-many and are the heart of the cross-vendor model: many `policy_rules` associate with many `network_objects` and `service_objects` through the `rule_object_mapping` and `rule_service_mapping` association tables, while each vendor object/service is linked to its canonical `normalized_object/normalized_service` through the `normalization-mapping` tables (carrying `match_method` and `confidence`). *Analytical* relationships hang off the



rule and the run: a `policy_rule` has many `rule_anomaly` rows — and, through the self-referential `related_rule_id` foreign key, a finding may name a second rule for pairwise and cross-device anomalies; an `anomaly_execution_log` run aggregates many `rule_anomaly` and `risk_assessment` rows; a `cti_indicator` has many `cti_observations`; and a `benchmark_case` has many `benchmark_results`, each optionally pointing at the detected `rule_anomaly`.

### 3.15 Class Diagram

The backend’s service and domain layer is organised as a pipeline of cohesive classes, each owning one stage of the transformation from raw configuration to narrated intelligence. Figure 3.9 shows the principal classes and their collaborations; the description below follows the data as it flows downstream.



**Figure 3.9.** Backend service and domain classes: connectors, parsers, normalization engine, anomaly engine and detectors, risk scorer, CTI providers/runner, LLM provider/reporter, benchmark scorer/runner.

**Acquisition.** The contract is the abstract `ConnectorInterface`, with `vendor_type` and `connector_version` class attributes and the abstract methods `authenticate()`, `validate_connection()` and `collect()`. It is realised by `FortiGateConnector` and `PaloAltoConnector`, each constructed from a `ConnectorConfig` dataclass (management

IP, decrypted secret, auth type, TLS/scheme). A registry selects the connector at runtime; `collect()` returns an `AcquisitionBundle` of raw artifacts.

**Parsing.** Vendor-specific parser classes (FortiGate and Palo Alto) replay raw artifacts into a vendor-tagged `ParsedDevicePayload`. The parsers understand vendor formats but run no anomaly logic.

**Normalization.** The normalization engine exposes `normalize_payloads()`, a pure function returning a DB-free `NormalizationResult` (its element types include `NormalizedRule` and `NormalizedObjectRec`, which carry the canonical correlation keys `src_object_keys`, `dst_object_keys` and `service_keys`). A separate persist class performs the idempotent upsert, and a loader class reconstructs a `NormalizationResult` from the repository for analysis.

**Anomaly engine.** The engine's entry point `analyze(result)` iterates the fixed-order `ALL_DETECTORS` tuple and concatenates each detector's output. Each detector is a function over the normalized model emitting `Finding` dataclasses; the set-theoretic helpers (`_covers`, `_sets_overlap`, `_same_access`) operate over the canonical key sets, realising the shadowing/redundancy/conflict relations as subset, equivalence and overlap tests. The fourteen `config_only` detectors plus the cross-device pair are the engine's analytical surface.

**Risk scorer.** A pure `scorer` module (`score_rule`, `score_device`, `tier_of`) computes the composite 0–100 score from finding severities and factor contributions; a DB-backed `runner` (`run_risk`) persists `RiskAssessment` rows idempotently per run.

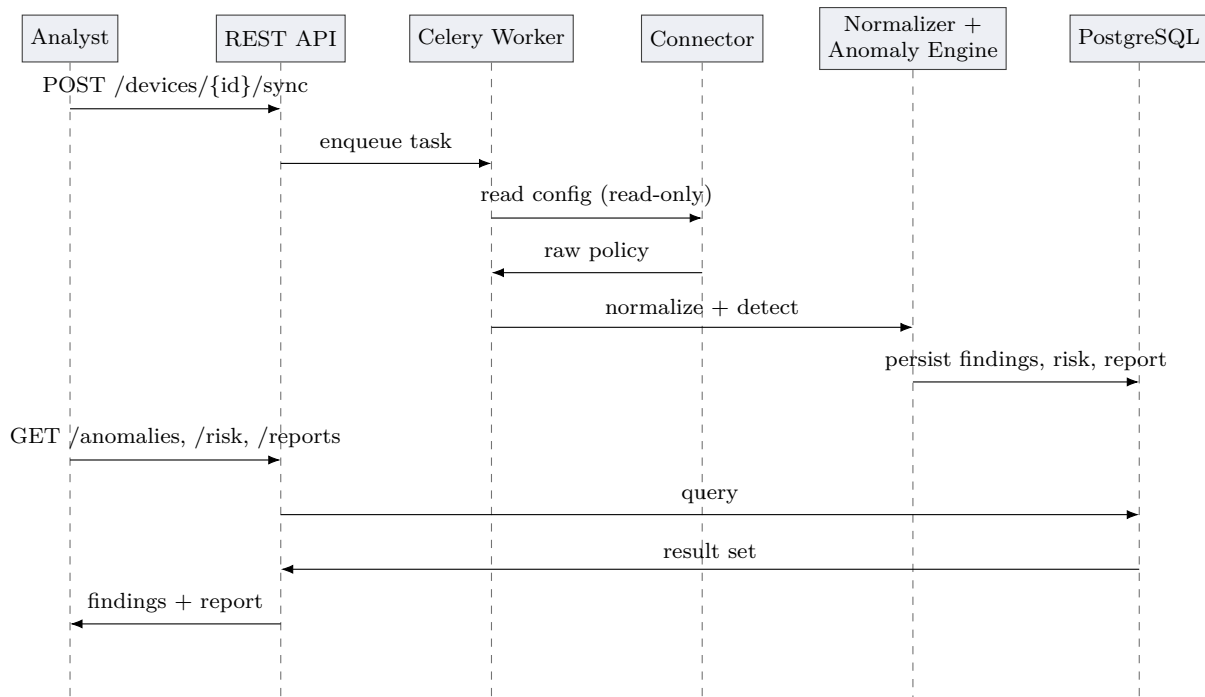
**CTI.** The abstract `CtiProvider` (with `lookup_ip()`) is realised by `LabOfflineProvider` (deterministic, network-free) and `AbuseIPDBProvider` (live, key-gated), each returning a `CtiVerdict`. The indicator-extraction functions enforce data minimisation, and the `run_cti` runner orchestrates enrichment, worst-verdict selection, cross-device correlation and the risk recompute.

**LLM.** The abstract `LLMProvider` (with `generate(system, user)`) is realised by `GeminiProvider` and the deterministic `OfflineProvider`, returning an `LlmResult`; the `reporter` assembles the evidence bundle, applies the guardrail prompt, and always persists an `LlmReport`.

**Benchmark.** A pure `scorer` computes TP/FP/FN and precision/recall/F1/severity-match from the ground-truth dataclasses (`ExpectedCase`, `CaseResult`, `BenchmarkScore`); the `run_benchmark` runner resolves expected cases against deployed rules, filters detected findings to `config_only`, and persists `BenchmarkCase/BenchmarkResult`.

## 3.16 Sequence Diagrams

The most important interaction in the platform is the end-to-end pipeline that turns a single analyst request into normalized policy, deterministic findings, a risk score and an AI report. Figure 3.10 traces this flow across the API tier, the Redis broker, the worker tier and the repository.



**Figure 3.10.** Sequence diagram: end-to-end policy acquisition, normalization, anomaly analysis, and reporting.

The sequence begins synchronously. The analyst issues `POST /api/v1/devices/{device_id}/poll`; the API verifies the device exists and that a credential is configured (returning 409 if absent), inserts an `AcquisitionJob` row with status `queued`, dispatches `poll_device.delay(job_id)` onto the acquisition queue, and immediately returns 202 Accepted with the `job_id` — performing no firewall I/O in the request cycle.

The remainder runs asynchronously as an automatic chain. The `poll_device` worker decrypts the credential in memory, resolves the vendor type, sets the job `running`, builds the connector, calls `collect()`, stores the raw artifacts with their SHA-256 manifest, updates the device’s freshness and firmware, sets the job `success` or `partial_success`, and on either success state chains `normalize_device.delay(job_id)` onto the normalization queue. The `normalize_device` worker loads the job’s artifacts from disk and runs `parse_artifacts` → `normalize_payloads` → `persist_result` (the idempotent Schema v4 upsert), then chains `run_anomaly_analysis_task.delay(device_id)` onto the analysis queue.

The `run_anomaly_analysis` worker opens an `AnomalyExecutionLog` (`status="running"`), reconstructs the `NormalizationResult` via `load_normalized_result`, calls `analyze(result)`, maps each `Finding` (`device_id`, `rule_uuid`) onto its `PolicyRule.rule_id` (skipping rules absent from the repository rather than guessing), inserts the `RuleAnomaly` rows, finalises the log with `completed` and `findings_count`, and as a best-effort final step calls `run_risk(db, run_id)` — a risk failure must never fail the analysis. (For cross-device detectors an all-scope run is invoked with `device_id=None`, analysing every device together.) Optionally, the analyst then triggers `POST /api/v1/cti/run` (enrichment and correlation, with a follow-on risk recompute)

and `POST /api/v1/reports/generate` (the evidence-grounded report, persisted even on provider failure).

Throughout, the frontend never blocks: it polls job and run status through the read-only endpoints `GET /api/v1/jobs/{job_id}`, `GET /api/v1/anomalies/runs` and the resource readers, then renders findings, risk, CTI and reports from PostgreSQL. The diagram thus makes visible the platform's governing rule — slow, fallible, external work is confined to the worker tier behind the Redis broker, while the request cycle does only fast transactional work and read-back.

# Chapter 4

## Implementation and Testing

This chapter documents the realized LuminaFPM system: how the strictly read-only, multi-vendor firewall anomaly management platform described in the preceding chapters was actually built, and how the resulting engine was validated against ground truth. The chapter is presented in two parts. *Part A* (this half) covers **implementation** — the development environment, the project layout, and a layer-by-layer account of the read-only data pipeline (acquisition → parsing → normalization → deterministic anomaly detection → risk scoring → threat intelligence → AI-assisted reporting), followed by the REST API surface and the React front-end console. *Part B* continues with **testing**: the benchmark acceptance gate, the unit/integration test suite, and the lab-based end-to-end validation. Throughout, the description is grounded in concrete file paths, class and function names, database tables, and REST endpoints from the delivered codebase rather than the original Term-1 proposal; where the build deliberately scopes below the full specification, this is stated plainly.

### 4.1 Implementation Overview

LuminaFPM is implemented as a containerized, multi-service application orchestrated by Docker Compose. The backend is a Python/FastAPI application backed by a Celery worker pool, with PostgreSQL as the system of record and Redis as the Celery broker and result backend. The front end is a React/TypeScript single-page application served (in development) by Vite. Two backing data stores (PostgreSQL 16, Redis 7) and an optional Tor SOCKS proxy complete the runtime topology.

The implementation faithfully realizes the platform’s single non-negotiable architectural principle: it is **strictly read-only against firewalls**. Every vendor connector issues only read verbs (`GET`, `show`, `monitor`, `keygen`); no module anywhere in the acquisition layer contains a write verb (`set`, `edit`, `delete`, `move`, or `commit`). All “mutating” HTTP calls the system makes are POSTs to its *own* API that trigger internal analysis jobs (`synchronize`, `analyze`, `enrich`, `generate`, `benchmark`); none of them ever reaches a firewall to change its configuration.

A second, equally load-bearing principle governs the analysis engines: **detection is deterministic and reads the normalized model only**. The raw FortiGate JSON and Palo Alto XML are acquisition and parser inputs; they are never analyzed directly. The anomaly engine consumes exclusively the canonical normalized repository, the AI layer

*explains* findings but never *detects* them, and every analysis engine cleanly separates a pure scoring/logic module (no database, no I/O) from a database-backed runner that persists results idempotently per analysis run. These two principles — read-only acquisition and deterministic, normalized-model detection — recur in every section below.

The v1 vendor scope is **Fortinet FortiGate (FortiOS REST/JSON) and Palo Alto Networks PAN-OS (XML API) only**. Cisco, which appeared in the Term-1 proposal, is de-scoped from v1 and is treated as future work; a small number of dormant Cisco/ASA code paths survive in the front end and are flagged as such.

## 4.2 Development Environment and Tools

The system was developed against the technology stack summarized in Table 4.1. The backend runs on Python 3.11 (base image `python:3.11-slim`); the front end runs on Node 22.

**Table 4.1.** *Core technology stack and tooling.*

| Concern         | Choice                              | Role                                                                          |
|-----------------|-------------------------------------|-------------------------------------------------------------------------------|
| Web framework   | FastAPI + Uvicorn                   | ASGI REST API surface (routers, dependency injection, OpenAPI).               |
| ORM / DB        | SQLAlchemy + PostgreSQL 16          | Models and session management; canonical system of record.                    |
| Migrations      | Alembic                             | Versioned schema migrations (Schema v4).                                      |
| Validation      | Pydantic + pydantic-settings        | Request/response schemas; typed environment configuration.                    |
| Task queue      | Celery (Redis broker)               | Asynchronous acquisition / normalization / analysis / CTI / reporting jobs.   |
| Cache / broker  | Redis 7                             | Celery broker and result backend; SSE pub/sub.                                |
| Crypto          | cryptography (Fernet)               | Symmetric encryption of stored device credentials.                            |
| XML hardening   | defusedxml                          | Hardened PAN-OS XML parsing (entity-expansion safe).                          |
| HTTP client     | requests                            | FortiGate REST and CTI API calls.                                             |
| Front-end build | Vite + @vitejs/plugin-react         | Dev server, build, lint, typecheck.                                           |
| UI framework    | React 19 + TypeScript 5.7           | Single-page console; incremental TS migration.                                |
| LLM provider    | Google Gemini (+ of-fline fallback) | Evidence-grounded SOC reporting; deterministic offline renderer.              |
| Orchestration   | Docker Compose                      | Multi-service runtime (db, redis, api, celery_worker, celery_beat, frontend). |

Two reproducibility caveats are documented honestly rather than smoothed over. First, the backend `requirements.txt` is **unpinned** — every package is named but no version

is fixed — so builds are not deterministic across time; the only version constraint is the Python base image. Front-end dependencies, by contrast, are pinned with caret (^) ranges in `package.json`. Second, the front-end Docker image runs the *Vite development server* (not a production build): an Nginx production-serve configuration (`frontend/nginx.conf`) exists but is not wired into the Compose deployment.

The Compose stack defines six active services. The `api` service runs Uvicorn with `--reload`; the `celery_worker` runs under `watchmedo` auto-restart and explicitly consumes the queues `acquisition`, `normalization`, `analysis`, `cti`, `reporting`, `celery` (without the explicit `-Q` list the routed tasks would never run); and the `celery_beat` service drives the scheduler tick every 60 seconds that powers the Settings scheduling feature. PostgreSQL and Redis are *not* published to the host (internal Docker network only), a deliberate security posture. A `tor-proxy` service and a commented-out “Robin” dark-web OSINT pair (`robin_ui/robin_api`) support the optional, off-by-default threat-intelligence subsystem and are not part of the running v1 stack.

## 4.3 Project Structure

The repository is split into a Python backend, a React/TypeScript front end, and a self-contained lab provisioner used only for testing. The principal directories are shown in Listing 4.1.

*Listing 4.1. LuminaFPM repository structure (abridged).*

```
lumina-fpm/
  backend/
    main.py # FastAPI app; mounts LTI router + 14 core routers
    core/
      config.py # Settings (pydantic-settings); env reference
      security.py # Fernet credential encryption
      logging.py # structured, secret-safe logging
    api/routes/ # REST routers (vendors, devices, rules, anomalies,
                  # risk, cti, reports, benchmark, jobs, ...)
    services/
      acquisition/ # read-only connectors (base, fortigate, paloalto,
                  # registry, storage, models, errors)
      parsing/ # vendor parsers (fortigate JSON, paloalto XML)
      normalization/ # canonical, model, engine, persist, loader
      anomaly/ # engine.py, detectors.py, model.py
      risk/ # scorer.py (pure), runner.py (DB)
      cti/ # extract.py, providers.py, runner.py
      llm/ # providers.py, prompts.py, reporter.py
      benchmark/ # scorer.py, ground_truth.py, runner.py
      lumina_threat_intel/ # optional, off-by-default LTI subsystem
    tasks/ # Celery tasks (acquisition, normalization, analysis)
    alembic/ # schema migrations (Schema v4)
    tests/ # unit + integration tests
  frontend/
  src/
```

```

app.jsx # hash router + app-level state
api.js # legacy aggregate loader -> LFPMContext
lib/api.ts # typed client (new single source of truth)
context/LFPMContext.jsx
dashboard.jsx audit.jsx inspector.jsx topology.jsx threats.jsx
risk.tsx reports.tsx cti.tsx benchmark.tsx # newer typed screens
lab/
  benchmark_dataset.py # ground-truth anomaly matrix (source)
  benchmark_ground_truth.json
  provision_benchmark.py # WRITE tooling (separate keys; NOT the platform)
  docker-compose.yml

```

A structural note: `lab/` contains *write* tooling that provisions known-bad policies onto the lab firewalls for benchmarking. It uses separate credentials and is explicitly **not** part of the LuminaFPM platform — the platform itself never writes. Because the `lab/` directory is not mounted into the API container, the authoritative ground-truth matrix is mirrored inside the backend at `services/benchmark/ground_truth.py`.

## 4.4 Acquisition Layer

The acquisition layer (`backend/services/acquisition/`) performs read-only extraction of firewall configuration from FortiGate (FortiOS REST/JSON) and Palo Alto (PAN-OS XML API). Connectors collect data only — no anomaly logic, no write or commit — and run asynchronously inside Celery workers, never inside the FastAPI request cycle.

### Connector contract

`base.py` defines the read-only connector contract. `ConnectorInterface` (an `abc.ABC`) declares three abstract methods: `authenticate()` (establish/validate auth, no config changes), `validate_connection()` (a lightweight read confirming connectivity and auth), and `collect()` (full read-only extraction returning an `AcquisitionBundle`). The accompanying `ConnectorConfig` dataclass carries the connection parameters resolved from the device record and the *decrypted* credential, which is held in memory only and never persisted. Its fields include `device_id`, `vendor_type` (`fortinet` or `paloalto`), `management_ip`, the in-memory `secret`, `auth_type`, `verify_tls` (default `True`), `timeout_seconds` (default 60), and — critically — `scheme` (default `"https"`), the HTTP-scheme override discussed below.

The `collect()` contract distinguishes *required* from *optional* endpoints: failure of an optional endpoint produces a warning and downgrades the bundle to `partial_success`, whereas failure of a required endpoint (the policies) raises an `AcquisitionError`. The error taxonomy (`errors.py`) maps every failure mode to a normalized `code` and a `retryable` flag that drives the retry policy — for example `AuthenticationError` (`AUTH_FAILED`, non-retryable, to avoid lockout hammering), `ConnectionFailedError` (`CONNECTION_FAILED`,

retryable), and `TLSError` (`TLS_FAILED`, non-retryable). Error messages are required to be secret-safe.

## FortiGate connector

`fortigate.py` implements `FortiGateConnector` (`vendor_type="fortinet"`). Authentication uses a Bearer API token (a dedicated read-only API admin account) injected via `_headers()`; `validate_connection()` performs a cheap GET of `/api/v2/monitor/system/status`. There is exactly one HTTP helper, `_get(path)` — and *no* write verb anywhere in the module, which is precisely how read-only is enforced at the code level. Its exception-to-error mapping converts requests SSL/timeout/connection exceptions and HTTP status codes (401→`AuthenticationError`, 403→`AuthorizationError`, 404→`UnsupportedEndpointError`, other  $\geq 400$ →`ConnectionFailedError`) into the normalized taxonomy. The connector iterates a static endpoint table covering policies, address objects, address groups, service objects, service groups, and device metadata (all required), plus interfaces and schedules (optional). Each successful response is captured as a `RawArtifact`.

## Palo Alto connector

`paloalto.py` implements `PaloAltoConnector` (`vendor_type="paloalto"`) against the PAN-OS XML API. Authentication uses an X-PAN-KEY header; when `auth_type=="panos_userpass"` the connector issues a read-only `type=keygen` request and regex-extracts an ephemeral key. Because PAN-OS returns HTTP 200 with `status="error"` in the XML body for bad credentials, the connector additionally inspects the body for `status="error"` plus an “Invalid credential” marker. Configuration is read with `type=config&action=show` over the unqualified vsys xpath `/config/devices/entry/vsys/entry/<leaf>`, which is the form proven to work against the single-vsys lab device. Required leaves map to policies, address objects/groups, and service objects/groups; the `zone` leaf is recommended and degrades to a warning on failure. PAN-OS NAT is a separate policy type and is deferred to future work.

## The HTTP-scheme override (lab escape hatch)

A documented deviation handles a real lab quirk: the lab FortiGate’s `httpsd` will not bind port 443, so its management API is reachable only over plain HTTP. The escape hatch is implemented end-to-end. `core/config.py` exposes `FIREWALL_INSECURE_HTTP_HOSTS` (a CSV list property), documented as **lab-only** — **never to be set in production**, since API tokens would otherwise traverse unencrypted. Both call sites that build a `ConnectorConfig` — the acquisition task and the device connectivity-test route — select the scheme per host: `scheme = "http" if device.management_ip in settings.firewall_insecure_http_hosts else "https"`. Crucially, only

`FortiGateConnector._base_url()` honors `scheme`; `PaloAltoConnector._base_url()` hardcodes `https://` and ignores it, so the override is FortiGate-specific.

## Raw persistence

`storage.py` persists each raw response to disk (under `{vendor}/{device_id}/{job_id}/`) so the parser can replay without re-polling, and writes an `acquisition_manifest.json` carrying per-artifact SHA-256 hashes. This gives the pipeline auditability and replayability: every analysis can be traced back to a specific, hashed raw capture.

## 4.5 Parsing and Normalization

### Parsing

The parsing layer (`backend/services/parsing/`) converts raw vendor responses into structured, *vendor-tagged* dataclasses without any canonicalization or anomaly logic — raw vendor values are deliberately preserved at this stage (e.g. FortiGate `accept` versus PAN `allow`). Both parsers emit a uniformly shaped `ParsedDevicePayload` containing `ParsedRule`, `ParsedAddressObject`, `ParsedAddressGroup`, `ParsedServiceObject`, and `ParsedServiceGroup` lists, so the downstream normalizer can consume either vendor uniformly. The FortiGate JSON parser (validated against a real FortiOS 7.0.5 response) maps FortiGate subnet/mask notations to CIDR and builds a structured `security_profiles` dictionary; the PAN-OS XML parser uses `defusedxml` when available and derives logging from `log-start/log-end`.

### Normalization and the canonical model

The normalization layer (`backend/services/normalization/`) converts vendor-tagged parser models into the canonical normalized policy model and the object/service correlation model. It is **deterministic and idempotent**: the same input always yields the same output, and re-running a sync never duplicates rows. `canonical.py` holds pure mapping functions (no database, no I/O): `canonical_action` collapses vendor actions into a small enum (`accept/allow`→`allow`, `deny`, `drop`, `reset`, `ipsec`); `canonical_logging` and `canonical_inspection` reduce heterogeneous logging/inspection settings to `true/false/unknown`, with the inspection result driving the unprotected-allow detector.

Two pieces of ANY-handling logic are central to correctness. For *objects*, `is_any_object` recognizes the names `{all, any}` and the values `{0.0.0.0/0, ::/0, ...}` and canonicalizes them to the single ANY sentinel (`"any"`, `"0.0.0.0/0"`). For *services*, `canonical_service` expands FortiGate's broad predefined services via a `_BROAD_SERVICES` map (`all`→`(any, None, None)`, `all_tcp`→`(tcp, 1, 65535)`, etc.), while PAN-OS predefined

names (`service=https`, `any`, ...) resolve through a `PREDEFINED_SERVICES` table. Without this expansion FortiGate's ALL service would key as `unknown:any` and the coverage/shadowing/redundancy logic would fail to treat it as the ANY sentinel. The `service_key()` helper produces stable keys (`app:<id>`, `<proto>:any`, or `<proto>:<ps>-<pe>`) that the detection engine compares as sets.

The deterministic correlation engine (`engine.py::normalize_payloads`) implements **correlation, not merge**: vendor objects and services remain separate rows for ownership and traceability, while canonical `normalized_object` / `normalized_service` records sit above them, linked by mapping records. Matching is confidence-scored: an exact name-and-value match scores 1.00 (`exact_name_value`); a same-value, different-name match scores 0.90 (`exact_value`) and raises a naming-inconsistency candidate. Each rule is normalized into a `NormalizedRule` carrying canonical action, logging/inspection flags, correlation-key lists (`src_object_keys`, `dst_object_keys`, `service_keys`), and a `normalized_content_hash`. `persist.py` writes the result idempotently into Schema v4 using documented natural keys (e.g. `PolicyRule` upserts on (`device_id`, `vdom_vsys`, `vendor_uuid`)), and `loader.py` reconstructs a byte-identical `NormalizationResult` from persisted rows so the anomaly engine can analyze the normalized repository across a `persist`→`load` round-trip — a contract that is essential for the cross-device detectors, which load all devices at once. The richer group cycle-detection and the 0.30/0.40/0.20 confidence tiers from the specification are documented-but-not-implemented at this snapshot.

## 4.6 Anomaly Detection Engine

The anomaly detection engine is the analytical heart of LuminaFPM. It is implemented in `backend/services/anomaly/` (`engine.py`, `detectors.py`, `model.py`) and embodies the platform's core thesis: that firewall misconfigurations can be detected *deterministically and explainably* by reasoning over a unified, cross-vendor normalized model. This section first develops the formal set-theoretic model that gives the engine its conceptual rigor, then describes the deterministic implementation, the severity/confidence and detection-mode machinery, the cross-device detectors, and finally the implemented taxonomy.

### 4.6.1 The Set-Theoretic Conceptual Model

Conceptually, each normalized firewall rule  $r$  is modeled as a tuple of sets over a small number of dimensions:

$$r = \langle S_r, D_r, V_r, a_r \rangle$$

where  $S_r$  is the set of canonical source-object keys,  $D_r$  the set of destination-object keys,  $V_r$  the set of canonical service keys, and  $a_r \in \{\text{allow}, \text{deny}, \text{drop}, \text{reset}, \text{ipsec}, \dots\}$  the canonical action. The *match space* of a rule is the Cartesian product  $M_r = S_r \times D_r \times V_r$  — the set of traffic flows the rule applies to. A distinguished ANY sentinel ( $\top$ ) is treated as the universal set on its dimension; an empty set on a dimension is likewise interpreted as

ANY. Two rules are then compared by elementary set relations on their three dimensions, and the anomaly classes follow directly:

- **Shadowing** ( $r_i$  shadows  $r_j$ ,  $i < j$ ): an earlier *blocking* rule whose match space covers the later rule,  $M_{r_j} \subseteq M_{r_i}$ , so the later rule can never match —  $r_i$  is a superset of  $r_j$  on all three dimensions.
- **Redundancy**: a later rule with the same action whose match space is covered by an earlier rule ( $M_{r_j} \subseteq M_{r_i}$ ,  $a_i = a_j$ ) but is *not* an exact duplicate — the later rule adds nothing.
- **Duplicate**: set equivalence with identical action —  $S_i=S_j \wedge D_i=D_j \wedge V_i=V_j \wedge a_i=a_j$ .
- **Conflict**: overlapping match spaces with *differing* actions —  $M_{r_i} \cap M_{r_j} \neq \emptyset \wedge a_i \neq a_j$  — the rulebase expresses two contradictory decisions for the same flows.
- **Generalization** is the dual of shadowing (a later, broader rule supersedes an earlier, narrower one of the same action); in v1 the engine surfaces this family through the redundancy/shadowing pair rather than as a distinct emitted type.

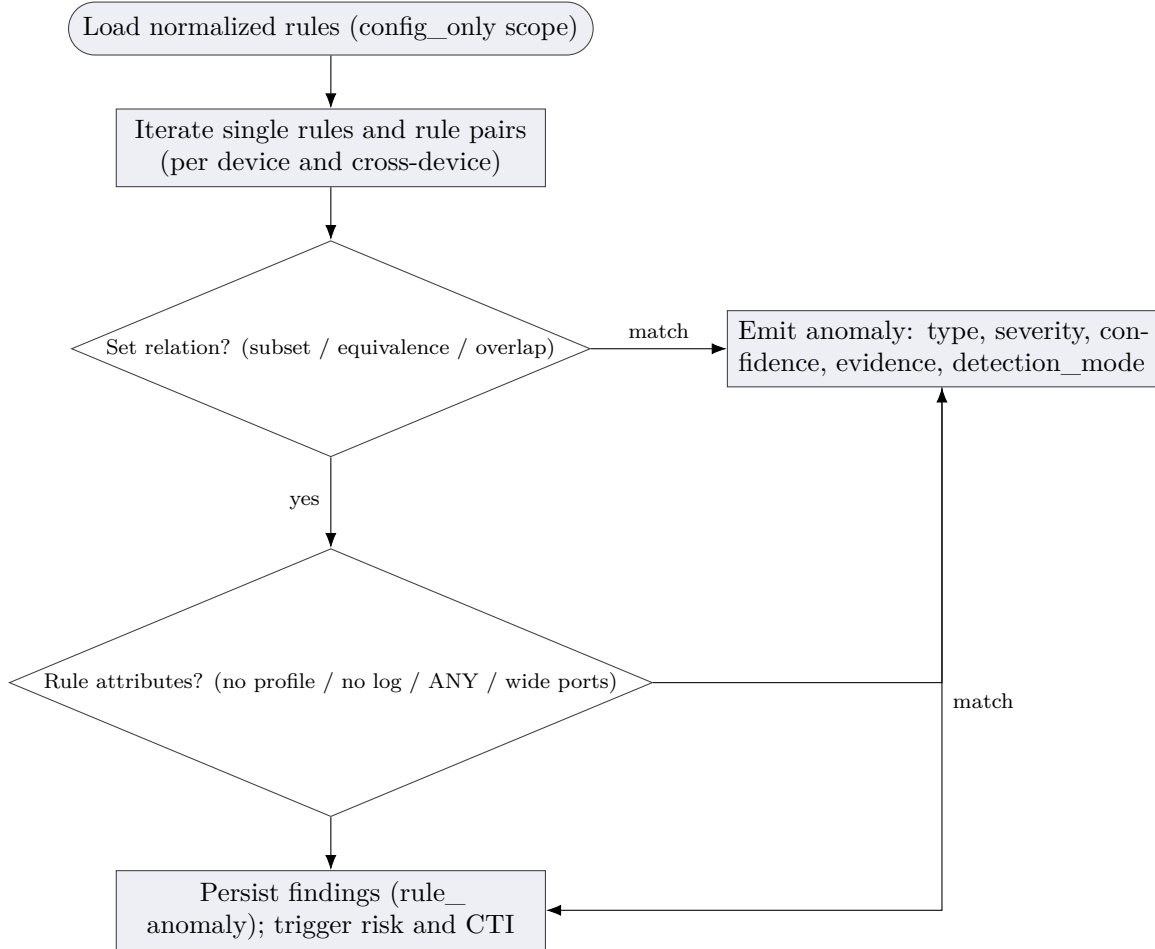
Single-rule anomalies are expressed just as naturally: an *overly permissive* rule is  $S_r = D_r = V_r = \top$  with  $a_r = \text{allow}$  (any/any/any); an *any-to-sensitive* rule has  $S_r = \top$  and  $D_r$  intersecting the sensitive-asset class; a *wide port range* is a single service spanning more than 1024 ports. This set-theory framing is the *formal* conceptual model; the *implementation* (below) is a deterministic rule-based engine over the normalized model — it is not machine learning, and it uses no statistics, randomness, or training data.

## 4.6.2 The Deterministic Engine

The engine entry point is `engine.py::analyze(result: NormalizationResult) -> List[Finding]`. It iterates a fixed-order tuple, `detectors.ALL_DETECTORS`, and concatenates each detector’s findings. It reads the normalized model *only* — no database, no network, no LLM. Determinism is guaranteed by three mechanisms: (a) a fixed detector order; (b) a stable rule sort, `_sorted_rules()`, keyed on (`device_id`, `rule_order`, `vendor_uuid`, `rule_name`); and (c) the use of `itertools.combinations` over the sorted list for pairwise detectors, so the “earlier” rule in any pair always has the lower `rule_order`. No randomness, wall-clock, or nondeterministic dictionary iteration is used. (The earlier mock predecessor, `anomaly_engine.py`, which generated random anomalies on random rules, is explicitly flagged as a specification violation and fully superseded by this package.)

The set logic operates directly on the canonical correlation keys. Module-level constants define the ANY sentinels (`ANY_OBJECT_KEY`, `ANY_SERVICE_KEY`), the action classes (`_BLOCKING_ACTIONS = {deny, drop, reset}`, `_ALLOW_ACTIONS = {allow, ipsec}`), the wide-port threshold (1024), the object-sprawl threshold (50), and the sensitive classes (`{admin, database, critical}`). Set helpers implement the relations from Section 4.6.1: `_covers(a,b)` returns true when  $a$  is ANY/empty or  $b \subseteq a$ ; `_sets_overlap(a,b)` returns true when either side is ANY/empty or the sets intersect; and `_`

`same_access(a,b)` requires equality on source, destination, and service. The end-to-end flow — from normalized rules through the per-detector set comparisons to persisted Findings — is shown in Figure 4.1.



**Figure 4.1.** Control flow of the deterministic anomaly-detection engine.

Each detector emits zero or more `Finding` objects. `Finding` is a pure, database-independent dataclass; the anomaly Celery task later maps each one onto a `RuleAnomaly` row by resolving `(device_id, rule_uuid) -> PolicyRule.rule_id`. Its fields are `anomaly_type`, `device_id`, `rule_uuid`, `severity`, `confidence`, `description`, `recommendation`, `evidence`, the optional cross-device peers `related_device_id/related_rule_uuid`, and `detection_mode`. Crucially, **every finding is explainable**: the `evidence` dictionary carries the concrete facts that produced it (device, vendor, rule name and order, vendor UUID, action), which is what makes the AI reporting layer able to *explain without re-detecting*.

### 4.6.3 Severity, Confidence, and Detection Mode

Severity and confidence are assigned per anomaly *type*, not learned: each detector hard-codes a severity in `{critical, high, medium, low}` and a confidence in `[0, 1]`, reflecting how strongly the configuration evidence implies the finding (see Table 4.2). Because severity is

fixed per type and independent of the engine’s matching, it can itself be validated by the benchmark (Part B).

The `detection_mode` field records *what kind of evidence* a finding rests on. The specification defines four/five modes: `config_only` (derivable from configuration alone — the v1 core), `conditional` (needs logs, hit-counts, or history), `simulated/benchmark_simulated` (derived from lab metadata and required to be labeled as such), and `future_enhanced` (needs enterprise telemetry); the CTI runner adds a runtime fifth mode, `cti`. In the delivered code, **all fourteen detectors emit `config_only`** (the Finding default), and the only non-`config_only` finding produced is `threat_exposure` (`detection_mode="cti"`), written by the CTI runner. The `conditional`, `simulated`, and `future_enhanced` modes are defined in the schema but not yet emitted by any detector — a deliberate v1 scope limit, not an omission.

#### 4.6.4 Cross-Device Detectors

Two detectors operate *across* devices and surface the platform’s core cross-vendor innovation as concrete findings. `_cross_device_pairs()` yields all sorted-rule combinations whose `device_id` differs, and equivalence is always established through normalized correlation keys, never raw vendor names.

`detect_cross_device_inconsistency` fires when two rules on different devices have *equal* canonical source/destination/service sets (`_same_access`) but *different* actions — for example, a FortiGate that **allows** exactly the canonical access a Palo Alto **denies**. This is rated **critical** (confidence 0.85): a conflicting access decision across the fleet is precisely the class of error that single-vendor tools cannot see. `detect_cross_device_security_posture_inconsistency` fires when two cross-device rules have equal sets and *both* allow, but differ in `logging_enabled` or `security_inspection_enabled` (e.g. one inspected, the other unprotected); it is rated **medium** (confidence 0.8). These two detectors are only meaningful because every device’s rules are normalized into one shared key space — the very capability the project set out to build.

#### 4.6.5 Anomaly Types and Policy-Rule Implementation

The full specification defines a 46-type anomaly taxonomy. Version 1 ships the “config-only core”: fourteen deterministic detectors (in `ALL_DETECTORS`) emitting fourteen distinct `anomaly_type` values, plus `threat_exposure` written by the CTI runner, for fifteen persisted types in total. Table 4.2 lists every implemented type — its description, the detection mode it is emitted in, and whether it is realized as a live policy-rule detector. Types from the broader framework that remain *simulated* or *future* (e.g. ordering anomaly, temporary-rule, poor-naming, zone-mismatch, NAT-risk, asymmetric-rule, and all `conditional/future_enhanced` entries) are specified but **not implemented** in v1 and are marked accordingly.

**Table 4.2.** *Implemented anomaly taxonomy: the config-detectable core actually emitted by the engine, versus the simulated/future remainder of the 46-type framework.*

| Anomaly type                                                                         | Description                                                                       | Severity | Detection mode                 | Policy-rule?     |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------|--------------------------------|------------------|
| unprotected_allow                                                                    | Enabled allow rule with no security inspection enabled.                           | high     | config__only                   | Yes              |
| shadowing                                                                            | Earlier blocking rule whose set covers a later rule; later never matches.         | high     | config__only                   | Yes              |
| redundancy                                                                           | Later rule (same action) fully covered by an earlier rule; adds nothing.          | medium   | config__only                   | Yes              |
| duplicate_rules                                                                      | Two rules with identical src/dst/service sets and identical action.               | medium   | config__only                   | Yes              |
| conflict                                                                             | Overlapping match space with differing actions on the same device.                | high     | config__only                   | Yes              |
| any_to_sensitive                                                                     | Allow from ANY source to a sensitive (admin/database/critical) asset.             | critical | config__only                   | Yes              |
| overly_permissive                                                                    | Allow with ANY source, ANY destination, and ANY service.                          | critical | config__only                   | Yes              |
| wide_port_range                                                                      | Allow referencing a service spanning more than 1024 ports.                        | medium   | config__only                   | Yes              |
| missing_description                                                                  | Rule with empty/whitespace description (fires even when disabled).                | low      | config__only                   | Yes              |
| missing_logging                                                                      | Enabled rule with logging disabled.                                               | medium   | config__only                   | Yes              |
| disabled_rule_review                                                                 | Disabled rule flagged for lifecycle review.                                       | low      | config__only                   | Yes              |
| object_sprawl                                                                        | More than 50 distinct objects on one side of a rule.                              | low      | config__only                   | Yes              |
| cross_device_inconsistency                                                           | Equal canonical access but conflicting action across two devices.                 | critical | config__only                   | Yes              |
| cross_device_security_posture_inconsistency                                          | Equal allow access but differing logging/inspection across devices.               | medium   | config__only                   | Yes              |
| threat_exposure                                                                      | Allow rule referencing an indicator a CTI provider flags malicious.               | from CTI | cti                            | Yes (CTI runner) |
| Ordering, temporary-rule, poor-naming, zone-mismatch, NAT-risk, asymmetric-rule, ... | Remaining entries of the 46-type framework requiring logs, history, or telemetry. | varies   | conditional / future__enhanced | No (future)      |

The single-rule detectors map almost verbatim onto the set model: `unprotected_allow` fires on an enabled allow rule with `security_inspection_enabled=="false"`; `overly_permissive` on any/any/any allow; `any_to_sensitive` on an ANY-source allow whose destination keys include a sensitive-class object; `wide_port_range` when a service's (`port_end - port_start`) > 1024; and `object_sprawl` when any side exceeds 50 distinct keys.

The pairwise detectors implement the subset/equivalence/overlap relations of Section 4.6.1 exactly, with each finding attributed to the appropriate rule and its peer recorded in `related_rule_uuid`.

## 4.7 Risk Scoring

Risk scoring (`backend/services/risk/`) converts deterministic findings into a normalized 0–100 composite score per rule and per device. The pure scorer (`scorer.py`) is database-free; the runner (`runner.py`) persists `RiskAssessment` rows idempotently per analysis run. (For traceability the persisted `calculation_version` is the literal "1.0.0"; the comments reference Volume 8, so the model is referred to informally as the “V8” composite.)

The per-rule composite is the clamped sum of eight factors:

$$\text{risk} = \min \left( 100, \text{anomaly} + \text{exposure} + \text{asset\_sensitivity} + \text{security\_posture} + \text{logging} + \text{cross\_vendor} + \text{cti} + \text{lifecycle} \right).$$

`score_rule(findings)` builds a `factors` dictionary in which only non-zero factors appear, and each factor is taken as a *maximum* across findings so it stays inside its allotted range regardless of how many findings hit it. The severity-point constants are `critical=35`, `high=22`, `medium=12`, `low=4`. The *anomaly* factor is the dominant severity plus a small “stacked-findings” compound bonus,  $\max(\text{sev\_points}) + \min(8, 2(|\text{findings}| - 1))$ . A factor-contribution map routes specific anomaly types to specific factors — for example `overly_permissive`→`exposure` (+20), `any_to_sensitive`→`asset_sensitivity` (+15), `unprotected_allow`→`security_posture` (+20), `missing_logging`→`logging` (+15), and the two cross-device types→`cross_vendor` (+28/+18). The *cti* factor is added only when a `threat_exposure` finding is present, scaled by the provider’s verdict severity (`critical=38`, `high=28`, `medium=18`, `low=10`). In v1 the *lifecycle* factor is fed only by `disabled_rule_review`; no age/history/drift factors exist yet.

Scores are bucketed by `tier_of(score)` into `critical` ( $\geq 90$ ), `high` ( $\geq 70$ ), `medium` ( $\geq 40$ ), `low` ( $\geq 1$ ), and `informational` (0). The device-level blend (`score_device`) deliberately weights the worst rule rather than averaging: it takes the top-5 rule scores and computes  $0.6 \max + 0.4 \text{avg}(\text{top5})$ , then adds a modifier  $\min(10, 4 \times \text{critical\_count})$  before clamping to 100, so a single critical rule is not diluted by many benign ones. Separately from the eight rule-level factors, the device score also admits an optional device-scope `firmware_modifier` (`critical=40`, `high=30`, `medium=18`, `low=10` points, with the result still clamped to 100) fed by the firmware-CVE intelligence axis (§4.8); this is a device-level adjustment, *not* a ninth rule-level factor. The runner scores every active `PolicyRule`, is idempotent per run (it deletes that run’s prior assessments but preserves other runs as history), and is read-only against firewalls.

## 4.8 Threat Intelligence (CTI)

The CTI layer (`backend/services/cti/`) enriches the normalized network objects with API-based threat intelligence and correlates malicious indicators back to the rules that expose them. It is API-based and strictly read-only against firewalls.

**Data minimization** is enforced before any network call (`extract.py`). `extract_indicators` dedupes objects by value and applies the rule `if not is_public and not allow_internal: continue` — so RFC1918/loopback/link-local/reserved addresses are *never* sent to external providers unless an operator explicitly opts in. The ANY sentinels (`0.0.0.0/0`, `::/0`) are guarded out entirely and never become indicators, and single-host `/32` (and `/128`) CIDRs are canonicalized to bare IPs. Indicator types are limited to `{ip_address, cidr, fqdn}`.

Providers (`providers.py`) implement a common `CtiProvider` ABC returning a `CtiVerdict`. `LabOfflineProvider` is always present and *network-free*: it carries a small `KNOWN_BAD` dictionary (e.g. a long-standing Tor-exit/abuse IP) so CTI is demonstrable offline and deterministically. `AbuseIPDBProvider` performs real REST lookups against AbuseIPDB but is activated only when an API key is configured, and it returns `None` (never an exception) on outages or non-200 responses, so enrichment never breaks. The runner (`runner.py::run_cti`) enriches every `NetworkObject`, picks the worst verdict across providers, and persists `CtiIndicator`/`CtiObservation` rows.

The correlation step is where CTI feeds the rest of the platform. `_correlate_threat` takes each malicious indicator, finds every `NetworkObject` matching its value, then every rule referencing those objects *across all devices*, and raises a `RuleAnomaly` of type `threat_exposure` (`detection_mode="cti"`) **only on rules whose action is allow** — i.e. rules that actually permit traffic to or from the known-bad indicator. Critically, CTI is *not* an anomaly by default: `threat_exposure` is a separately labeled `cti`-mode finding (excluded from the config-only benchmark) that *feeds risk*. After correlation, `run_cti` invokes the risk runner so the `cti` factor lands, wrapped in a guard that only warns on failure. Internal indicators never leave the platform.

### Axis 2: firmware-CVE device intelligence

Alongside the indicator axis above, the CTI layer carries a second, device-scoped axis (`vuln_intel.py`, `vuln_runner.py`) that correlates each device's acquired `firmware_version` against a curated, versioned offline dataset of known firmware vulnerabilities. The dataset (`backend/data/cve_reference.json`) holds five vendor-specific CVEs with their CVSS scores, severity bands, and CISA Known-Exploited-Vulnerabilities (KEV) flags: CVE-2024-21762 and CVE-2024-23113 (FortiOS, both critical and KEV-listed), and CVE-2024-3400, CVE-2024-0012, and CVE-2025-0108 (PAN-OS). For every match the runner writes a `CtiIndicator` of type `firmware_version` together with a `CtiObservation` whose `threat_type` is `vulnerability`. Crucially, this axis writes *zero* `rule_anomaly` rows, so it can never perturb the deterministic config-only benchmark; its sole downstream

effect is to raise a device-scope `firmware_modifier` on the risk score (§4.7), *not* a rule-level anomaly. The default `VULN_PROVIDER=offline` keeps the axis fully deterministic and network-free; a live provider that queries the CISA KEV catalog and NVD exists but is dormant and fail-soft, so an outage or missing key degrades silently rather than breaking enrichment.

## 4.9 AI-Assisted Reporting

The AI layer (`backend/services/llm/`) produces SOC-grade narrative reports. Its design is governed by a single mandate: **the LLM explains, it never detects**. The deterministic engine decides what is an anomaly; the model only summarizes, prioritizes, and explains findings that already exist, citing the evidence that produced them.

A provider abstraction (`providers.py`) defines `LLMProvider.generate(system, user) -> LlmResult`. `GeminiProvider` (default model `gemini-2.5-flash`) calls the Gemini REST endpoint with `temperature=0.2` and, importantly, `thinkingBudget=0` — without which the 2.5 models can spend their entire token budget on internal reasoning and return no text. It hardens against transient failures with up to three retries on network errors and HTTP 429/503. `OfflineProvider` is a deterministic, network-free fallback that renders the evidence prompt verbatim under an explicit “deterministic rendering — no LLM configured ...not an AI analysis” header, guaranteeing the platform still reports when no key is present. `build_provider` returns Gemini only when both the name is `gemini` and a key is configured; otherwise it falls back to offline.

The guardrail system prompt (`prompts.py`, `PROMPT_VERSION = "soc-report/2.1"`) instructs the model to use *only* the supplied evidence, to never invent CVEs, exploit claims, IP reputations, or unsupported remediations, to separate evidence from interpretation, to attribute every threat-intel claim to its provider, and states explicitly that “the deterministic engine — not you — decides what is an anomaly.” The current 2.1 prompt adds two SOC-report disciplines: the model writes *prose only* (no Markdown tables or pipe characters, because the deterministic builder owns every table), and it refers to each finding by a human-readable name (e.g. “the unprotected allow on POL-037, FGT-LAB”) rather than a raw database identifier, with the bracketed evidence IDs used purely as internal grounding anchors. The evidence-grounded user prompt tags every deterministic finding with an `[anomaly_id=...]` and every CTI observation with an `[observation_id=...]`, presenting CTI evidence as clearly separate from the engine. The reporter (`reporter.py::generate_report`) assembles an `evidence_refs` dictionary (`anomaly_ids`, `risk_id`, `cti_observation_ids`) and **always persists** an `LlmReport` — even when the provider fails, the report is stored with `status="failed"` because the underlying deterministic data is unchanged and remains the source of truth.

The “LLM explains, it never detects” mandate is enforced structurally by a **deterministic structured report builder** (`report_builder.py`), which assembles the entire body of every report — all of its tables — directly from the database with *no* LLM involvement. The builder is pure DB assembly: it composes a *Prioritized Remediation*

table, a *Risk Posture* table on the device axis, a *Firmware Vulnerabilities* table on the CVE axis, and a *Per-Finding Evidence* table, together with the report totals and metadata. The language model authors *only* the executive-summary prose that sits on top of this deterministic document; every figure, finding, and citation in the body is computed, not generated. The builder serves two report scopes through one shape: an *executive* scope spanning the whole estate and a *per-rule* scope focused on a single policy, both rendered to the identical document structure. Reports export to Markdown — GitHub-flavored pipe tables retrieved via `GET /api/v1/reports/{id}/markdown` — and to PDF, for which the front end builds print-ready HTML that the browser saves as a PDF. Because the tables are independent of the model, the deterministic document stands on its own when the LLM is unavailable: the report is persisted with `status="failed"` and the prose section is simply omitted, with no empty “## Summary” heading left behind.

## 4.10 Backend API

The backend exposes a REST API under `/api/v1`; all routers are registered in `backend/main.py`. The read-only semantics hold at the API boundary: no endpoint writes to a firewall. POSTs that trigger background work dispatch a Celery task and return 202 Accepted with a task/job/run id, while the `benchmark/risk/CTI/reports` POSTs run synchronous, pure-DB analysis with no firewall contact. Table 4.3 lists the principal analysis and acquisition endpoints (resource CRUD endpoints for vendors, devices, rules, and network objects follow the conventional REST pattern and are omitted for brevity).

**Table 4.3.** Principal LuminaFPM REST endpoints (acquisition, analysis, reporting, scheduling, and notifications).

| Method | Path                                              | Purpose                                                                                                                      |
|--------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| POST   | <code>/api/v1/devices/{id}/credentials</code>     | Create/rotate encrypted firewall credential (returns no secret).                                                             |
| POST   | <code>/api/v1/devices/{id}/test-connection</code> | Read-only connectivity check (authenticate + validate); status only.                                                         |
| POST   | <code>/api/v1/devices/{id}/poll</code>            | Trigger Celery acquisition job; returns 202 {job_id}.                                                                        |
| GET    | <code>/api/v1/jobs</code>                         | List acquisition jobs (newest first).                                                                                        |
| GET    | <code>/api/v1/jobs/{id}/artifacts</code>          | List raw artifacts (metadata + SHA-256 only).                                                                                |
| POST   | <code>/api/v1/anomalies/run</code>                | Trigger anomaly analysis (omit <code>device_id</code> for an all-scope cross-device run).                                    |
| GET    | <code>/api/v1/anomalies</code>                    | List findings; filter by severity, type, status, device, run; paginated.                                                     |
| GET    | <code>/api/v1/anomalies/runs</code>               | List anomaly execution runs (observability).                                                                                 |
| GET    | <code>/api/v1/anomalies/{id}</code>               | Finding detail with evidence and rule context.                                                                               |
| PATCH  | <code>/api/v1/anomalies/{id}</code>               | Analyst lifecycle (open, resolved, suppressed, accepted-risk, or false-positive); a reason is required for the latter three. |
| POST   | <code>/api/v1/risk/run</code>                     | Recalculate risk (pure-DB); persists rule + device assessments.                                                              |

| Method | Path                           | Purpose                                                                                            |
|--------|--------------------------------|----------------------------------------------------------------------------------------------------|
| GET    | /api/v1/risk                   | Latest risk assessments ( <code>scope_type=rule device</code> ), highest first.                    |
| POST   | /api/v1/cti/run                | Enrich indicators, persist observations, correlate, recalc risk.                                   |
| GET    | /api/v1/cti                    | Threat Center: enriched indicators + provider observations.                                        |
| POST   | /api/v1/benchmark/run          | Score an analysis run against ground truth (pure-DB).                                              |
| GET    | /api/v1/benchmark/report       | Read persisted benchmark results (recall, severity-match, per-case).                               |
| POST   | /api/v1/reports/generate       | Generate an evidence-grounded SOC report ( <code>rule executive</code> ).                          |
| GET    | /api/v1/reports                | List generated LLM reports.                                                                        |
| GET    | /api/v1/reports/{id}           | Report detail (output + <code>evidence_refs</code> + provider/model/prompt_version).               |
| GET    | /api/v1/reports/{id}/markdown  | Download the report as Markdown (GFM tables; the basis for PDF export).                            |
| GET    | /api/v1/schedules              | List the per-operation schedules (acquisition, detection, <code>threat_intel</code> ).             |
| PATCH  | /api/v1/schedules/{op}         | Enable/disable and set the interval or cron expression for an operation.                           |
| POST   | /api/v1/schedules/{op}/run-now | Dispatch the chosen operation immediately.                                                         |
| GET    | /api/v1/notifications          | List in-app notifications with the unread count ( <code>unread_only</code> , <code>limit</code> ). |
| POST   | /api/v1/notifications/read-all | Mark all notifications as read.                                                                    |

CORS is restricted to the configured origins with credentials. Two honest gaps are noted: the route handlers carry no per-endpoint authentication dependency (RBAC is specified but not enforced in the route layer), and several endpoint names in the internal target contract (e.g. `/policies`, `/risks`) differ from the implemented names (`/rules`, `/risk`). The read-only-against-firewalls guarantee, however, holds throughout in code.

## 4.11 Frontend Implementation

The front end (`frontend/src/`) is a single-page React 19 + TypeScript application built with Vite. It is a **read-only console**: every mutating call it makes triggers one of LuminaFPM’s own analysis jobs — synchronize, analyze, enrich, generate, benchmark — and none writes to a firewall.

### Architecture and the typed client

Routing is a hand-rolled, hash-based router in `app.jsx` (the installed `react-router-dom` is unused); navigation is centralized in `goTo(page, params)` and mirrored into the URL so deep-links such as `#audit?rule=POL-026` and `#topology?device=fw-004` work. Global state lives in `LFPMPContext` (`LFPMPProvider` exposing `{data, loading, error, refreshData}`). The application is mid-migration between two client layers, by design:

a *legacy aggregate loader* (`src/api.js`) fans out parallel `fetch` calls to inventory and analysis endpoints and maps backend rows into the UI’s flat view-model (resolving run-scoped findings, severity-based rule status, and the real V8 risk blend), while a newer *typed client* (`src/lib/api.ts`) — the “new single source of truth” — provides a generic `request<T>` over `/api/v1` with typed interfaces (`Anomaly`, `RiskItem`, `BenchmarkReport`, `CtiIndicator`, ...) and grouped endpoint objects (`anomalies`, `risk`, `benchmark`, `cti`, `reports`, `inventory`). Legacy `.jsx` screens consume the context; the four newest screens (`risk`, `reports`, `CTI`, `benchmark`) are authored in `.tsx` against the typed client.

## Screens

The console presents eight screens. The screenshots in this section are captured live from LuminaFPM’s bundled demonstration tenant (*NovaTech*) in the light theme to show the interface; the quantitative laboratory results in Section 4.16 come from the live two-vendor lab. The **Dashboard** (Figure 4.2) is the most data-dense screen and is explicitly de-mocked: five clickable hero KPIs (average fleet risk computed from real per-device V8 risk, open issues, critical CVEs, fleet-online, threat findings) deep-link into filtered views, and hand-drawn SVG charts render a risk-trend line, a real anomaly-type distribution colored by worst severity, and vendor-risk tiles built from the vendors actually present in the fleet.

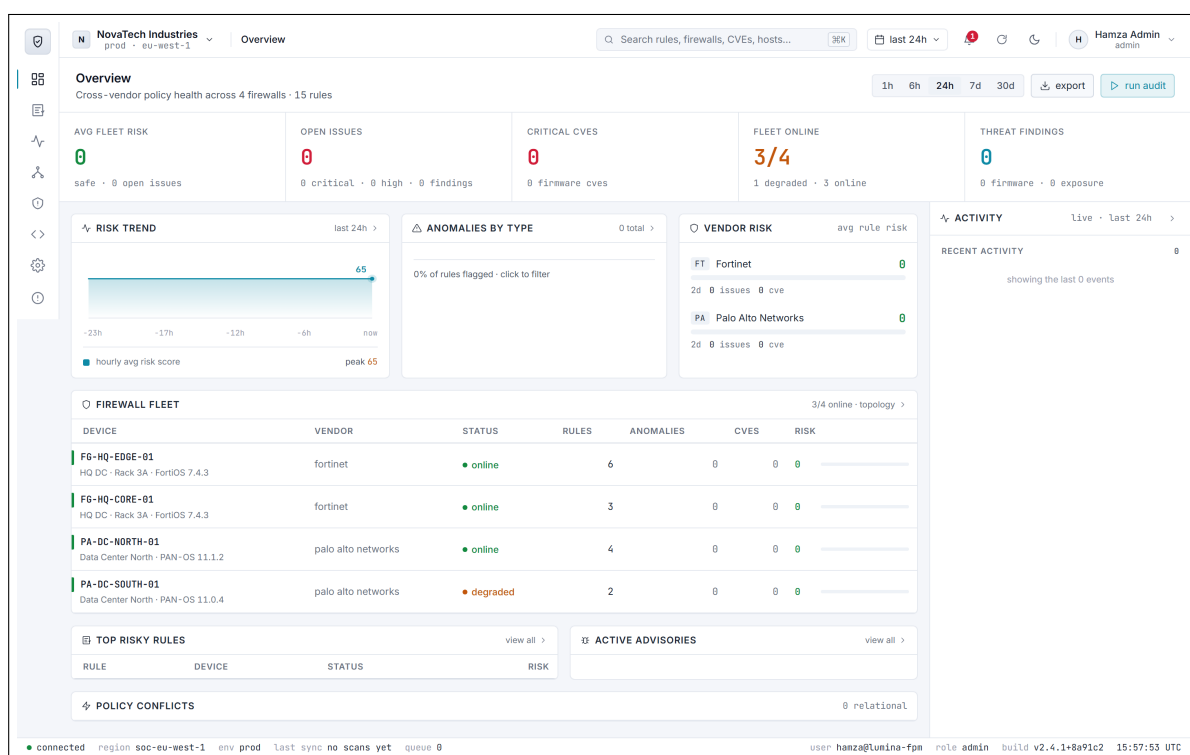


Figure 4.2. LuminaFPM dashboard (overview).

The **Policy Audit** screen (Figure 4.3) is the analyst’s primary workspace. It lists run-scoped, per-rule anomalies with severity-based status and real V8 risk, offers faceted filters (saved views, severity, vendor, firewall, action, a minimum-risk slider), sortable

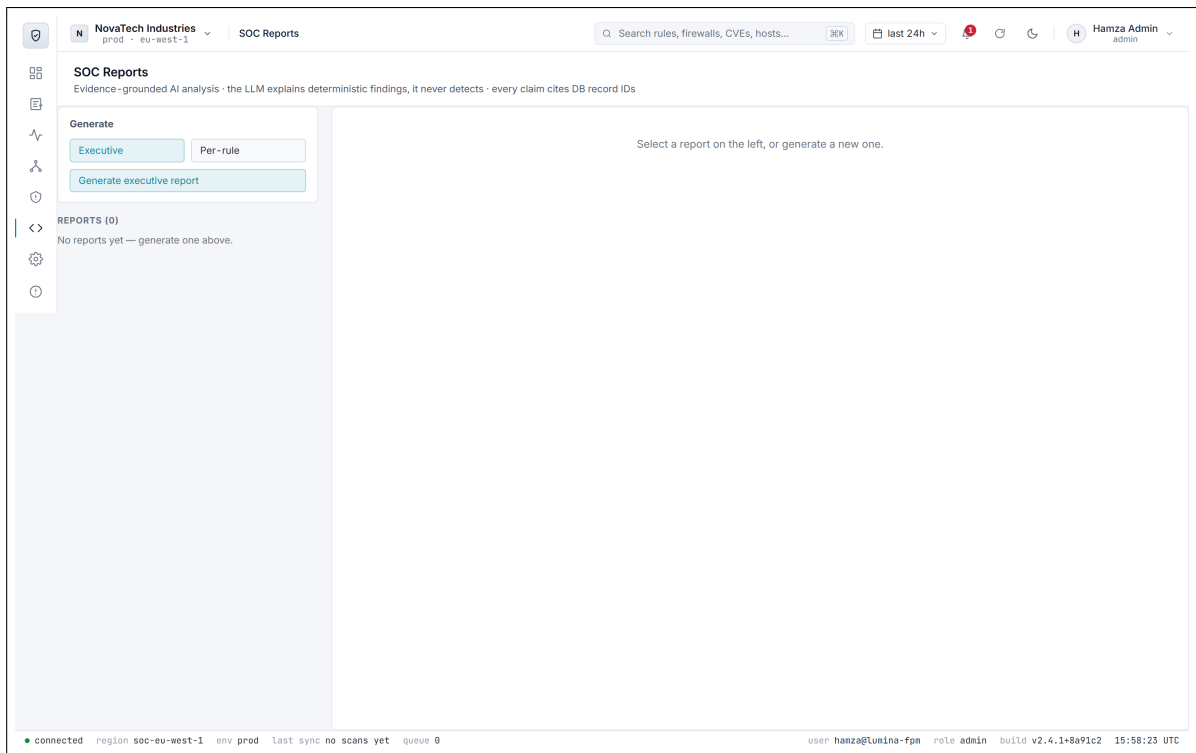
columns, CSV export, and deep-link intent application. A row click opens the slide-over **Inspector**, whose **AnomalyCard** exposes the analyst lifecycle (*resolve* / *false\_positive* / *accept\_risk*), each issuing a `PATCH /anomalies/{id}` with a prompted reason where required.

| ID      | RULE                                   | DEVICE         | SRC           | DST                     | SERVICE         | ACT   | STATUS | PRIO | RISK |
|---------|----------------------------------------|----------------|---------------|-------------------------|-----------------|-------|--------|------|------|
| POL-001 | Allow-Outbound-Web LAN → WAN           | F6-HQ-EDGE-01  | HQ-LAN        | ANY-INTERNET            | any             | ALLOW | clean  | 1    | 0    |
| POL-002 | Allow-DNS-Out LAN → WAN                | F6-HQ-EDGE-01  | HQ-LAN        | CLOUDFLARE-DNS, GOOGLE- | any             | ALLOW | clean  | 2    | 0    |
| POL-003 | Allow-Inbound-HTTPS-DMZ WAN → DMZ      | F6-HQ-EDGE-01  | ANY-INTERNET  | WEB-SERVER-01           | web-protection  | ALLOW | clean  | 3    | 0    |
| POL-004 | Allow-Inbound-SMTP-DMZ WAN → DMZ       | F6-HQ-EDGE-01  | ANY-INTERNET  | MAIL-SERVER-01          | email-filter    | ALLOW | clean  | 4    | 0    |
| POL-005 | Block-C2-Traffic any → WAN             | F6-HQ-EDGE-01  | HQ-LAN        | BLOCKED-C2-RANGE        | any             | DENY  | clean  | 5    | 0    |
| POL-006 | Default-Deny-Inbound WAN → any         | F6-HQ-EDGE-01  | any           | any                     | any             | DENY  | clean  | 100  | 0    |
| POL-007 | Allow-LAN-to-DC HQ-LAN → DC-ZONE       | F6-HQ-CORE-01  | any           | any                     | any             | ALLOW | clean  | 1    | 0    |
| POL-008 | Allow-LAN-to-DMZ-Web HQ-LAN → DMZ      | F6-HQ-CORE-01  | any           | any                     | any             | ALLOW | clean  | 2    | 0    |
| POL-009 | Block-DMZ-to-LAN DMZ → HQ-LAN          | F6-HQ-CORE-01  | any           | any                     | any             | DENY  | clean  | 3    | 0    |
| POL-010 | Allow-Web-to-DB DMZ → DB-ZONE          | PA-DC-NORTH-01 | WEB-SERVER-01 | DC-DB-PRIMARY           | strict-security | ALLOW | clean  | 1    | 0    |
| POL-011 | Allow-DB-Replication DB-ZONE → DB-ZONE | PA-DC-NORTH-01 | DC-DB-PRIMARY | DC-DB-REPLICA           | any             | ALLOW | clean  | 2    | 0    |
| POL-012 | Allow-Monitoring NSGT → any            | PA-DC-NORTH-01 | any           | any                     | any             | ALLOW | clean  | 3    | 0    |
| POL-013 | Default-Deny-DC any → any              | PA-DC-NORTH-01 | any           | any                     | any             | DENY  | clean  | 100  | 0    |
|         | Allow-All-Internal                     |                |               |                         |                 |       |        |      |      |

**Figure 4.3.** LuminaFPM policy audit and rule inspector.

The **Risk Posture**, **SOC Reports**, and **CTI / Threat Center** screens are the typed (`.tsx`) surfaces. Risk Posture renders device-risk cards and a top-risky-rules table with `factor_breakdown` chips and the tier legend. SOC Reports (Figure 4.4) drives `reports.generate/list/get`, rendering the LLM output through a dependency-free markdown renderer alongside its `evidence_refs` and `confidence_note`, and gracefully handling the failed status when no provider is configured. The CTI Center renders the indicator table (provider verdict, threat type, confidence) joined to the affected ALLOW rules from `threat_exposure` anomalies.

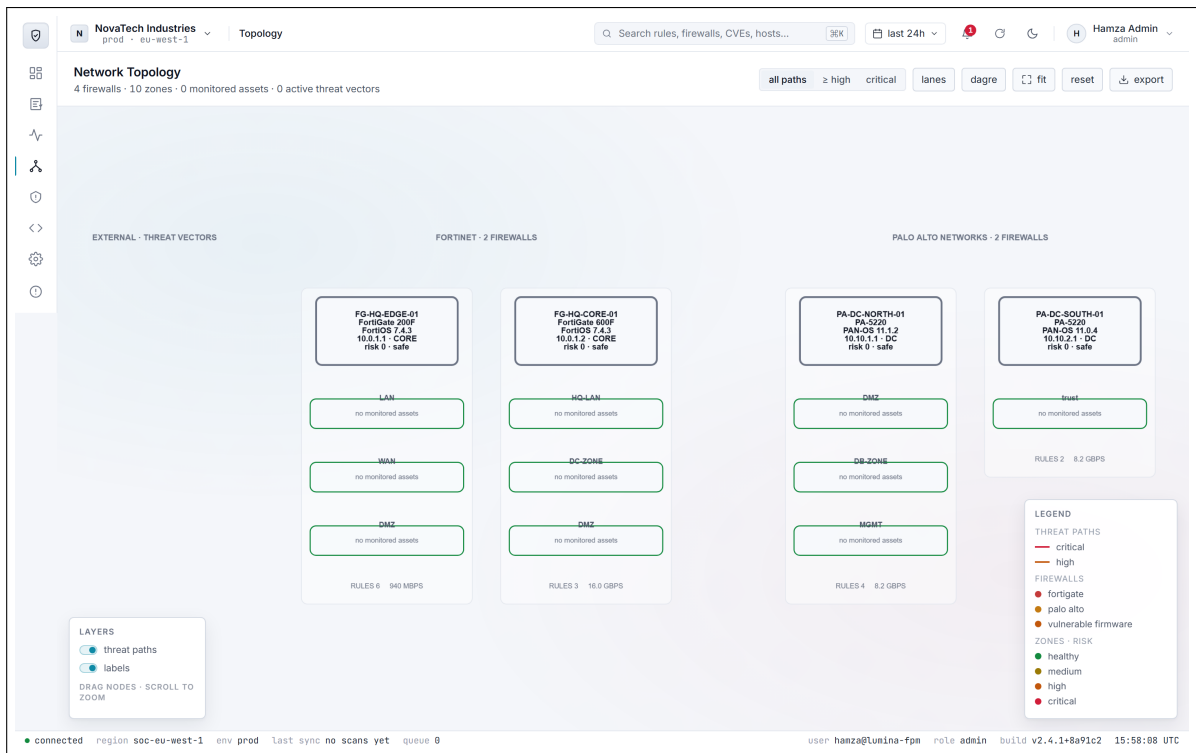
The **Topology** screen (Figure 4.5) visualizes the fleet — firewalls, zones, monitored assets, and CTI threat vectors — as an *interactive* graph built on Cytoscape.js. It uses a vendor-lane compound layout (each firewall is a lane containing its zone stacks and monitored assets), with draggable nodes whose positions persist to `localStorage`, wheel-zoom and pan, a live firmware-CVE badge sourced from each device’s acquired `firmware_version`, PNG export, and the `?device=` deep-link; clicking any node opens the Inspector with the firewall, zone, host, external-indicator, or path payload. The **Benchmark Center** surfaces the acceptance gate (Part B) as a first-class screen, running `benchmark.run` then `benchmark.report` and displaying precision/recall/ $F_1$ , severity-match, the TP/FP/FN counts, per-type coverage, and a per-case matrix. The **Threat Center** presents the two CTI axes side by side: the *indicator* axis lists enriched indicators (provider verdict, threat



**Figure 4.4.** *LuminaFPM AI-assisted SOC report viewer.*

type, confidence) joined to the ALLOW rules they expose through `threat_exposure` findings, while the *firmware-CVE* axis lists each device’s installed firmware and the CVEs that firmware matches. **Settings** is a functional administrative console: its *Firewalls* tab performs full device management toward the platform’s own database — add, edit, delete, set or rotate the encrypted credential, test the connection, and trigger an acquisition — while remaining strictly read-only toward the firewalls themselves; its *Scheduling* tab configures interval or cron schedules for acquisition, detection, and threat intelligence with an immediate run-now action; and its *Notifications* tab shows the in-app notification feed. The remaining tabs (access and roles, API and secrets, audit log) are descriptive placeholders pending the authentication work described in Future Work.

Several divergences are documented plainly rather than hidden: the login screen is a cosmetic mock with no backend authentication route, and the API route handlers enforce no per-endpoint authorization (RBAC is specified but not yet wired into the routers); a handful of Cisco/ASA code paths remain dormant though v1 supports FortiGate and Palo Alto only; and a few installed front-end dependencies (`recharts` and `react-router-dom`) are present but unused because the corresponding features — charts and routing — are deliberately hand-rolled (Cytoscape.js, by contrast, is now used by the topology). These are recorded as known states and candidates for cleanup, consistent with the project’s commitment to honest documentation.



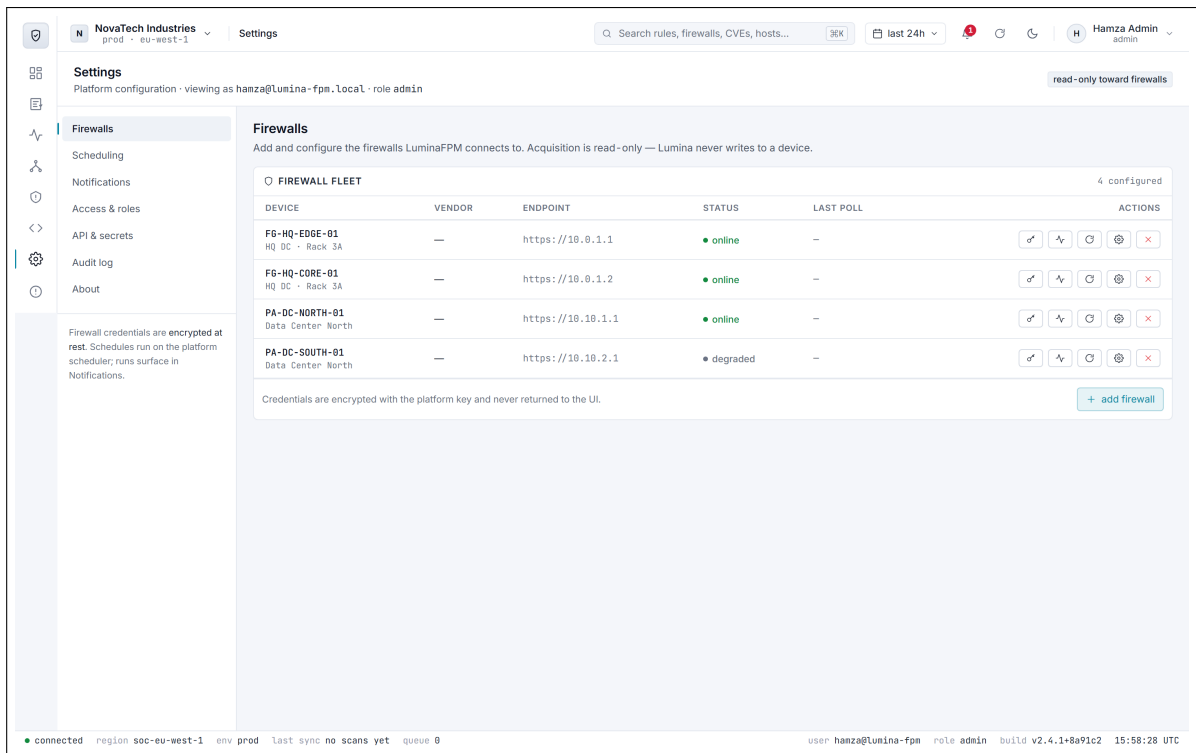
*Figure 4.5. LuminaFPM logical policy topology.*

## 4.12 Settings, Scheduling, and Notifications

A late addition to the platform turns the *Settings* screen from a descriptive placeholder into a functional administrative console, and adds two supporting subsystems — scheduling and in-app notifications — that operationalize the pipeline. All three are built strictly within the project’s read-only-against-firewalls mandate: every mutation they perform targets LuminaFPM’s own database, never a vendor device.

The *Firewalls* tab performs full device lifecycle management against the platform’s own database: it can create, edit, and delete devices, set or rotate each device’s Fernet-encrypted credential, and trigger an acquisition, while remaining strictly read-only toward the firewalls themselves. The credential is write-only — it is encrypted at rest and never returned by the API — and the only outbound operation is a read-only `test-connection` that authenticates and validates reachability without modifying any vendor state. A new per-device `use_http` column on `firewall_device` provides a lab HTTP escape hatch for the FortiGate appliance whose HTTPS stack is broken. The tab can onboard a device against an empty database — the user interface find-or-creates the owning vendor — so the platform bootstraps cleanly from a clean install.

Scheduling is backed by a new `schedule_config` table holding one row per operation (`acquisition`, `detection`, and `threat_intel`), a dependency-free five-field cron evaluator (`services/scheduling.py`), and a Celery Beat task, `scheduler.tick`, that fires every 60 seconds, reads the editable table, and dispatches any operation whose next run is due. Each operation can be run immediately (`run-now`) or on a fixed interval or cron schedule. The control surface is exposed



**Figure 4.6.** The functional Settings console: the Firewalls tab manages the device fleet (add, edit, delete, set/rotate the encrypted credential, test connection, and acquire) while remaining strictly read-only toward the firewalls; the Scheduling and Notifications tabs sit alongside.

through GET `/api/v1/schedules`, PATCH `/api/v1/schedules/{operation}`, and POST `/api/v1/schedules/{operation}/run-now`.

In-app notifications are persisted in a new notification table. The pipeline emits a notification on the completion or failure of each major operation (acquisition, detection, and threat-intel) and on every high- or critical-severity finding; emission goes through an `emit_safe` helper so that a notification failure can never break the pipeline. The top-bar bell polls the unread count and exposes the feed through GET `/api/v1/notifications`, POST `/api/v1/notifications/{id}/read`, and POST `/api/v1/notifications/read-all`. Consistent with the read-only mandate, none of this scheduling or notification machinery ever writes to a firewall.

## 4.13 Testing and Evaluation Strategy

LuminaFPM is validated through a deliberately layered evaluation strategy, mirroring the discipline that the platform itself enforces on firewall policy: each claim about correctness must be grounded in concrete, reproducible evidence rather than visual inspection. Three complementary layers are used, ordered from the most isolated to the most realistic:

1. **Unit testing** of every backend subsystem in isolation (`backend/tests/test_*.py`) — acquisition, parsing, normalization, anomaly detection, risk scoring, benchmark scoring, CTI and LLM reporting. These tests are pure: by design they run with no live firewall,

no network, and no database, using either fixture data captured from the real lab or monkey-patched provider responses. This makes the suite fast, deterministic and runnable on any developer machine as a regression gate.

2. **Deterministic benchmark evaluation** against an authoritative ground-truth matrix (Volume 7; `backend/services/benchmark/`). This is the platform’s *acceptance gate*: the deterministic anomaly engine is scored against an expected-anomaly matrix, producing true/false-positive and false-negative counts and the derived precision, recall,  $F_1$  and severity-match metrics. The benchmark proves the engine works against a known answer key rather than against the analyst’s intuition.
3. **Live end-to-end validation** on the parallel VMware lab, against the real Fortinet FortiGate (192.168.55.10) and Palo Alto PAN-OS (192.168.55.20) devices on 192.168.55.0/24. The full pipeline — poll → parse → normalize → analyze → risk → CTI → report — is executed against live vendor configuration, exercising the connectors, the cross-vendor correlation, and the persistence layer that the pure unit tests deliberately stub out.

The three layers are intentionally redundant in coverage but distinct in fidelity. Unit tests catch logic regressions cheaply; the benchmark proves the engine’s *output* matches a per-type ground truth with measurable precision and recall; and the live run proves the same code path survives contact with real vendor APIs, real self-signed certificates, and real multi-vendor configuration drift. A defect surfaced at a higher layer is, wherever possible, pushed back down into a unit test — as happened with the FortiGate **ALL**-service normalization defect (Section 4.17), which was first observed in a live run and then permanently captured as a normalization regression test.

## 4.14 Unit Testing

The backend unit suite lives under `backend/tests/` and comprises eleven subsystem test files. Each is structured to exercise the *pure* core of its subsystem — the scoring, parsing, or detection logic that contains no I/O — so that correctness is asserted independently of the database-backed runners that persist results. Where real vendor data is required, the suite uses fixtures captured directly from the lab firewalls (e.g. `backend/tests/fixtures/fortigate_policy.json` and `paloalto_rulebase.xml`, the genuine FortiOS v7.0.5 and PAN-OS responses), so the parsers and the normalizer are validated against real-world payload shapes, not synthetic mock-ups. Table 4.4 summarizes the coverage of each file.

**Table 4.4.** Backend unit-test coverage by subsystem (*backend/tests/*).

| Test file                          | Subsystem and what is asserted                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_acquisition.py</code>   | Read-only connectors and raw-artifact storage. Monkey-patched FortiOS JSON and PAN-OS XML responses; firmware extraction (FortiGate v7.4.1, Palo Alto 11.1.3); partial-success behaviour on an optional-endpoint 404; raw-artifact manifest with per-artifact SHA-256 and replay. No network or DB required.                                                                                                                                                                     |
| <code>test_parsing.py</code>       | Vendor parsers against <i>real captured lab artifacts</i> (FortiOS v7.0.5, PAN-OS). Asserts both vendors parse to a uniform <code>ParsedDevicePayload</code> and that the cross-device benchmark pair ( <code>LAN_NET</code> → <code>WEB_SERVER</code> , <code>HTTPS</code> , <code>allow</code> , <code>no inspection</code> ) is equivalent at the parse level.                                                                                                                |
| <code>test_normalization.py</code> | The cross-vendor normalization keystone. Asserts that the FortiGate and Palo Alto definitions of <code>LAN_NET</code> (10.10.10.0/24) and <code>WEB_SERVER</code> (10.10.20.10/32) correlate to a single canonical object each (confidence 1.00), that PAN’s predefined <code>service-https</code> resolves to the same <code>tcp/443</code> canonical service as FortiGate’s custom <code>HTTPS</code> , and that repeated runs produce identical content hashes (determinism). |
| <code>test_anomaly.py</code>       | The deterministic anomaly engine. Ground-truth assertions on the real fixtures (exactly two <code>unprotected_allow</code> and two <code>missing_description</code> , and <i>no</i> false cross-device finding) plus crafted unit tests for <code>shadowing</code> , <code>duplicate_rules</code> , <code>conflict</code> , <code>any_to_sensitive</code> and <code>overly_permissive</code> ; includes the FortiGate ALL-service shadowing/redundancy regression.               |
| <code>test_risk.py</code>          | The pure risk scorer (Volume 8): <code>tier_of</code> band boundaries, <code>score_rule</code> factor derivation, the <code>clean</code> → <i>informational</i> and <code>governance</code> → <i>low</i> cases, the <code>broad-unprotected-unlogged</code> → <i>critical</i> case, the cross-device <code>cross_vendor</code> factor, the cap at 100, and the <code>score_device</code> top-5 blend.                                                                            |
| <code>test_benchmark.py</code>     | The pure benchmark scorer (Volume 7): a perfect-match case, a false negative driving recall down, a false positive driving precision down, a cross-device pair matched on either side, and a severity mismatch driving the severity-match rate down.                                                                                                                                                                                                                             |
| <code>test_cti.py</code>           | v1 API-based CTI (Volume 9): <code>is_public_ip/classify/extract_indicators</code> with RFC1918 data-minimization, the deterministic <code>LabOfflineProvider</code> verdict, the provider registry, and the <code>threat_exposure</code> →CTI risk-factor path (+28).                                                                                                                                                                                                           |
| <code>test_llm.py</code>           | v1 LLM SOC reporting (Volume 10): the guardrail <code>SYSTEM_PROMPT</code> (“use only evidence”, “do not invent”, “the deterministic engine — not you — decides”), <code>OfflineProvider</code> determinism, provider fallback when unkeyed, and evidence-ID embedding in the rendered prompt.                                                                                                                                                                                   |

| Test file                             | Subsystem and what is asserted                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_benchmark_matrix.py</code> | The offline engine acceptance gate. Inlines the enriched 26-rule matrix (10 FortiGate + 16 Palo Alto) and asserts the deterministic engine reproduces the ground-truth answer key exactly — 43 expected cases, 0 false positives, 0 false negatives, 0 severity mismatches — before any device is provisioned, so a detection regression fails the suite without a live lab. |
| <code>test_vuln.py</code>             | The firmware-CVE device axis (Volume 9). Version-tuple and affected-range parsing, CVSS-to-severity banding, the offline lab CVE dataset (Fortinet and Palo Alto only — Cisco-free), the device-affected predicate, and the firmware modifier that raises device risk.                                                                                                       |
| <code>test_scheduling.py</code>       | The dependency-free five-field cron engine behind the Settings scheduler. Field parsing with range/step/list validation, match evaluation (daily, step, day-of-week), and the next-run computation that drives Celery Beat.                                                                                                                                                  |

The optional Lumina-Threat-Intel (LTI) module carries its own separate tests under `backend/services/lumina_threat_intel/tests/` (diff-tracker hashing, the LLM-client JSON extraction and retry path, and prompt anti-hallucination checks); these are not part of the v1 acceptance suite and are documented only for completeness. The eleven v1 files above constitute the regression gate that must pass before any live run.

## 4.15 Benchmark Evaluation (Acceptance Gate)

The benchmark framework (Volume 7; `backend/services/benchmark/scorer.py`, `ground_truth.py`, `runner.py`) is the formal acceptance gate for the detection engine. Its premise is that an anomaly engine cannot be trusted on the strength of plausible-looking output; it must be scored against an answer key constructed independently of the engine itself.

### 4.15.1 Metrics: TP / FP / FN and the derived rates

The pure scorer (`scorer.py`) compares the engine’s actual findings for an analysis run against an `ExpectedCase` matrix. Matching granularity is *(rule, anomaly\_type) presence* — not a count — so a rule that is expected to exhibit a given anomaly type counts once. For a cross-device case the anomaly may legitimately appear on *either* of the two paired rules, because the engine attributes a cross-device finding to one side; such a case matches if the type is present on either rule. From the comparison the scorer computes:

- **True positives (TP)** — expected cases the engine detected.
- **False negatives (FN)** — expected cases the engine missed.

- **False positives (FP)** — reported (rule, anomaly\_type) pairs that no expected case consumed.
- **Precision** =  $TP / (TP + FP)$ , **recall** =  $TP / (TP + FN)$ , and  $F_1 = 2PR / (P + R)$ , each guarded against a zero denominator and rounded to four decimal places.
- **Severity-match rate** = (cases whose detected severity equals the expected severity) / TP. Because the expected severity is fixed per anomaly *type* (Volume 6 Table 7) and is independent of the engine, this rate is a genuine, separate check that the engine assigns the correct severity, not merely the correct type.

### 4.15.2 Ground-truth construction

The expected matrix is authored in `backend/services/benchmark/ground_truth.py`, which mirrors the design intent of `lab/benchmark_dataset.py` but is kept inside the backend because the `lab/` directory is not mounted into the API container. Each expected anomaly is derived from a rule's *configuration plus the detector definitions* — never from engine output — so the matrix is a true answer key. EXPECTED\_SEVERITY pins thirteen anomaly types to a severity (e.g. `any_to_sensitive`, `overly_permissive` and `cross_device_inconsistency` are *critical*; `unprotected_allow`, `shadowing` and `conflict` are *high*; `missing_description` and `disabled_rule_review` are *low*). The INTRA table maps each named rule, per vendor, to its expected config-only anomalies — for example `FGT_ANY_ANY` → {`overly_permissive`, `unprotected_allow`, `missing_logging`, `missing_description`, `conflict`} and `FGT_ANY_DB` → {`any_to_sensitive`, `unprotected_allow`, `missing_logging`, `missing_description`}. CROSS\_DEVICE enumerates three pairs: one `cross_device_inconsistency` (`FGT_DB_ACCESS_XDEV` vs `PA_DB_ACCESS_XDEV` — the same canonical LAN→DB\_SERVER MySQL access where FortiGate allows but Palo Alto denies) and two `cross_device_security_posture_inconsistency` pairs (an equivalent LAN→WEB HTTPS allow that FortiGate inspects but Palo Alto leaves unprotected).

### 4.15.3 Deployment filtering and config\_only scoping

The runner (`runner.py`) resolves the matrix only against *deployed* rules: an intra-device case is skipped when its rule is not present in `policy_rule`, and a cross-device case is scored only when *both* paired rules are deployed. This is what keeps cap-dropped rules (e.g. `FGT_ADMIN_DB_NOLOG` and `FGT_DISABLED_RISKY`, omitted under the FortiGate-VM ten-policy cap) from being mis-counted as false negatives. Crucially, the runner joins `RuleAnomaly`→`PolicyRule` filtered both by `analysis_run_id` and by `detection_mode == "config_only"`. This filter is what makes the benchmark sound: the configuration-only detectors are scored against the configuration-only answer key, while the `cti-mode threat_exposure` finding (which is not a configuration anomaly) is excluded and evaluated separately. The acceptance criterion (Volume 7 Table 7) is qualitative — every core config-only anomaly detected in atomic cases, both cross-device cases detected, compound findings

present, simulated cases labelled, and every finding traceable to its rule(s) and reason — and the runner exposes the raw metrics rather than hard-coding a numeric  $F_1$  threshold.

## 4.16 Test Cases and Outputs

This section presents concrete test cases and their outputs from the live lab, focused on the canonical end-to-end run, **analysis run #25**, executed against both vendors. It proceeds from individual rule-level cases, through the aggregate run output, to the benchmark scorecard and the CTI and AI-reporting cases.

### 4.16.1 Representative rule-level cases

Table 4.5 lists representative policy rules deployed on the lab firewalls and the anomalies the deterministic engine produced for each. Every entry is reproducible: the rule is read from live vendor configuration, normalized to the canonical model, and evaluated by the fixed-order detector chain, so the same input always yields the same findings.

**Table 4.5.** *Representative live rule-level test cases and the engine’s findings (run #25).*

| Rule                                   | Dev.    | Engine findings (anomaly_type)                                                                                                                                                                                                                 |
|----------------------------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FGT_ANY_ANY                            | FGT     | overly_permissive, unprotected_allow, missing_logging, missing_description, conflict — an any/any/any allow with no inspection or logging.                                                                                                     |
| PA_ANY_ANY                             | PA      | overly_permissive, unprotected_allow, missing_logging, missing_description, conflict — the Palo Alto mirror of the above.                                                                                                                      |
| FGT_ANY_DB                             | FGT     | any_to_sensitive (ANY source to a <i>database</i> -class destination), unprotected_allow, missing_logging, missing_description; plus a separate cti-mode threat_exposure (its object set references the known-bad indicator — Section 4.16.4). |
| PA_ANY_DB                              | PA      | any_to_sensitive, unprotected_allow, missing_logging, missing_description — the Palo Alto mirror.                                                                                                                                              |
| FGT_SHADOWED_WEB2                      | FGT     | shadowing, conflict — preceded by a blocking rule whose canonical sets cover it, so it can never match; the differing action is also flagged as a conflict.                                                                                    |
| PA_SHADOWED_WEB2                       | PA      | shadowing, conflict — the Palo Alto shadowed pair.                                                                                                                                                                                             |
| FGT_REDUNDANT_WEB                      | FGT     | redundancy, duplicate_rules — a later rule fully covered by an earlier same-action rule.                                                                                                                                                       |
| FGT_WIDE_PORTS                         | FGT     | wide_port_range (a single service spanning > 1024 ports), redundancy.                                                                                                                                                                          |
| FGT_DB_ACCESS_XDEV / PA_DB_ACCESS_XDEV | FGT, PA | cross_device_inconsistency (critical) — identical canonical LAN→NET→DB_SERVER MySQL access; FortiGate <i>allows</i> , Palo Alto <i>denies</i> . The keystone cross-vendor finding.                                                             |
| PA_ALLOW_MALICIOUS                     | PA      | Config-clean (inspected, logged, described, narrow source) — the engine raises <i>no</i> configuration anomaly; only a separate cti-mode threat_exposure fires because the rule permits traffic to the known-bad indicator (Section 4.16.4).   |

Three properties are visible in this table and are central to the platform’s thesis. First, *compound findings*: a single permissive rule such as `FGT_ANY_ANY` legitimately produces five distinct findings, each with its own evidence and recommendation. Second, *cross-vendor equivalence*: the FortiGate and Palo Alto mirrors produce the same finding sets because detection operates over the normalized model, not vendor syntax. Third, the `*_DB_ACCESS_XDEV` pair demonstrates the engine’s signature capability — detecting that two different vendors have reached *contradictory access decisions* about the same canonical traffic, an inconsistency no single-vendor tool could surface.

#### 4.16.2 Aggregate run output (run #25)

The all-scope analysis run aggregated across both devices. The aggregate output is summarized below and the per-type distribution is given in Table 4.6.

- **Scope:** 2 devices (FGT-LAB, PA-LAB), 26 active policy rules (FortiGate 10 under the VM cap, Palo Alto 16), evaluated by the fixed-order chain of 14 deterministic detectors.
- **Current findings:** 43 config-only findings scored for the run — 40 intra-device plus 3 cross-device pair findings (the frontend scopes to the latest completed all-scope run, separating these from the historical anomalies of earlier runs to avoid cross-run inflation).
- **Device risk (Volume 8):** both Palo Alto and FortiGate score 100 (*critical*). The blend is dominated by the worst rules — `PA_ANY_DB/FGT_ANY_DB` score 99 and `FGT_ANY_ANY/PA_ANY_ANY` score 98 — under a top-5 max-weighted blend plus a critical-count modifier; FortiGate’s base of 96 is additionally lifted to the ceiling by a firmware-CVE device modifier of +40, and Palo Alto (already at 100) carries a +30 firmware modifier, reflecting the device CVEs in Section 4.8.

*Table 4.6. Anomaly-type distribution and benchmark coverage, run #25.*

| anomaly_type           | Count | Detected? |
|------------------------|-------|-----------|
| unprotected_allow      | 8     | ✓         |
| missing_description    | 6     | ✓         |
| missing_logging        | 6     | ✓         |
| conflict               | 5     | ✓         |
| duplicate_rules        | 3     | ✓         |
| redundancy             | 3     | ✓         |
| shadowing              | 2     | ✓         |
| any_to_sensitive       | 2     | ✓         |
| overly_permissive      | 2     | ✓         |
| wide_port_range        | 2     | ✓         |
| cross_device_security_ | 2     | ✓         |
| posture_inconsistency  |       |           |
| disabled_rule_review   | 1     | ✓         |

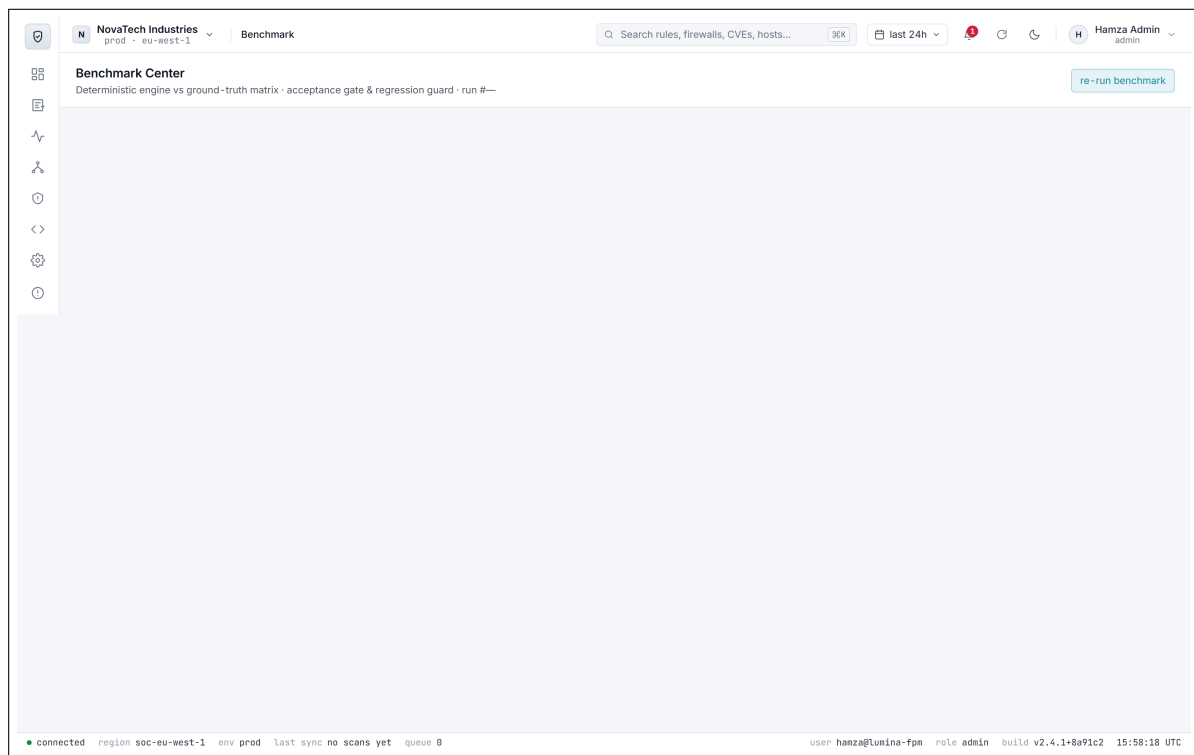
| anomaly_type               | Count | Detected? |
|----------------------------|-------|-----------|
| cross_device_inconsistency | 1     | ✓         |

The distribution is the engine’s real, deterministic output; it spans all thirteen config-only anomaly types that the deployed lab policy set can exercise (only `object_sprawl`, which needs more than fifty objects on one side, is not exercised by this deliberately small lab set). The two-sided symmetry of several types (e.g. two `overly_permissive`, two `any_to_sensitive`, two `shadowing`) reflects the deliberate mirroring of the benchmark dataset across both vendors, which in turn validates that cross-vendor normalization yields identical detection on equivalent configurations.

### 4.16.3 Benchmark scorecard (the acceptance gate)

The benchmark runner scored run #25’s config-only findings against the deployed-rule-resolved ground-truth matrix. The result is a perfect scorecard:

- **43 expected cases, TP 43, FP 0, FN 0.**
- **Precision = 1.0, recall = 1.0,  $F_1 = 1.0$ .**
- **Severity-match rate = 1.0** — every detected finding also carried the severity that Volume 6 Table 7 fixes for its type.



**Figure 4.7.** *LuminaFPM benchmark center (acceptance gate).*

The Benchmark Center screen (`frontend/src/benchmark.tsx`, Figure 4.7) renders this scorecard as live tiles — precision, recall,  $F_1$ , and severity-match — alongside the

TP/FP/FN/cases summary, a coverage-by-anomaly-type table, and a per-case matrix. On run #25 every tile reads 100%, the summary reads 43/0/0/43, and each of the 43 per-case rows is an exact match. It is important to be precise about what this scorecard certifies: *within the config-only taxonomy bucket and the deployed lab policy set*, the engine detects every expected anomaly, raises no spurious ones, and assigns the correct severity. It does *not* claim coverage of the full 46-type taxonomy — the not-yet-implemented and telemetry-dependent types are deliberately out of this bucket (Section 4.17).

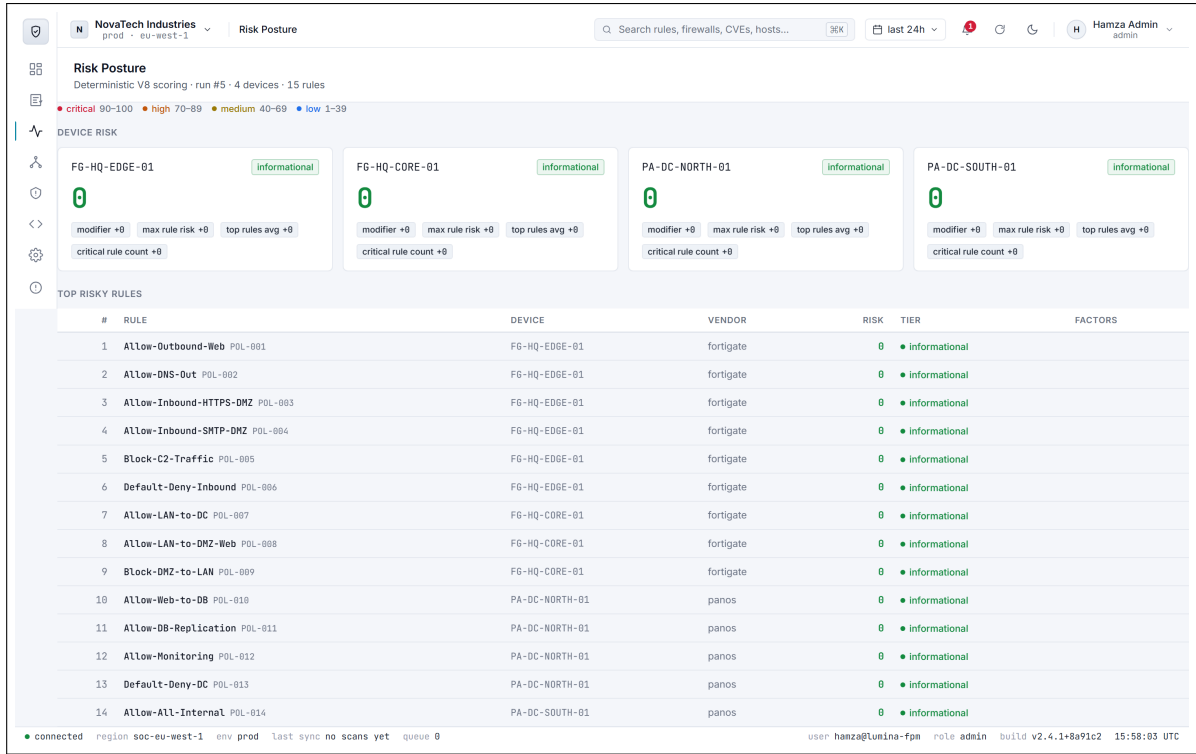
#### 4.16.4 CTI test case

The CTI subsystem was validated end-to-end with the deterministic, network-free `LabOfflineProvider`, whose `KNOWN_BAD` table maps the indicator `185.220.101.1` (a long-standing Tor exit / abuse source) to a verdict of *malicious*, threat type `tor_exit`, severity *high*, confidence 0.85. Two rules permit traffic to this indicator: `FGT_ANY_DB` (whose database-object set resolves to include the indicator) and the deliberately config-clean demonstrator `PA_ALLOW_MALICIOUS`. The CTI runner extracted the indicator, enriched it, and correlated it across *all* devices' objects, raising a `threat_exposure` finding (`detection_mode="cti"`) on each of those allow rules — one per device. The test also confirmed the platform's data-minimization guarantee: `internal_indicators_sent` was `false`, i.e. the RFC1918 lab objects were never transmitted to any external provider. Because `threat_exposure` is a cti-mode finding rather than a config anomaly, it is excluded from the config-only benchmark scorecard above while still feeding the risk recalculation, which lifted the affected FortiGate rule's risk through an added `cti` factor (+28).

The Risk Posture screen (`frontend/src/risk.tsx`, Figure 4.8) renders per-device risk cards and a top-risky-rules table with `factor_breakdown` chips and the tier legend. On run #25 both the PA-LAB and FGT-LAB cards reach the critical ceiling of 100 (FortiGate's rule-blend base of 96 is lifted to 100 by its firmware-CVE device modifier), with `PA_ANY_DB/FGT_ANY_DB` at 99 and `FGT_ANY_ANY/PA_ANY_ANY` at 98, each carrying its per-rule factor breakdown (anomaly, exposure, posture, logging, asset-sensitivity). That breakdown is the audit trail that explains *why* a rule scored as it did.

#### 4.16.5 AI-report test case

The LLM SOC-reporting path was validated for a rule-scoped report on `FGT_ANY_DB`. The reporter assembled that rule's four deterministic findings, its risk assessment, and the CTI observation, embedding their database IDs as `evidence_refs`: `{anomaly_ids:[391,410,424,440], risk_id:243, cti_observation_ids:[4]}`. With a valid key, `gemini-2.5-flash` produced a polished SOC report that cited each evidence ID, attributed the CTI verdict explicitly to the `lab_offline` provider, labelled its prose as "Analysis", and invented nothing — demonstrating the Volume 10 guardrail that *the deterministic engine, not the LLM, decides what is an anomaly; the LLM explains and prioritizes*. The same test also exercised the graceful-failure path: when an earlier key



*Figure 4.8. LuminaFPM risk posture screen.*

returned `API_KEY_INVALID`, the platform fell back to the deterministic `OfflineProvider`, which renders the same grounded evidence verbatim under an explicit “not an AI analysis” header, and the report was still persisted with `status` recorded. This proves the platform always reports, with or without a working LLM, and never lets the AI layer fabricate a finding.

## 4.17 Findings and Limitations

The evaluation establishes that LuminaFPM’s core thesis holds: a strictly read-only platform can acquire multi-vendor firewall policy, normalize it into a single canonical model, and detect cross-vendor anomalies deterministically with measurable, perfect precision and recall against ground truth on the lab policy set. In the spirit of honest academic reporting, the following limitations bound that claim and define the scope of future work.

- **Config-detectable subset, not the full taxonomy.** v1 ships 14 deterministic detectors (plus the cti-mode `threat_exposure`), covering the config-only core of the 46-type taxonomy (shadowing, redundancy, conflict, permissiveness, posture, sensitivity, cross-device). The remaining  $\sim 32$  types — firewall-specific (zone mismatch, NAT risk, weak profile, direction), lifecycle/history (drift, age, rule explosion, change frequency), and runtime/advanced (hit distribution, time-of-day, geo, compliance, asymmetric) — are *specified but not implemented*, because they require hit-counts, history, or enterprise

telemetry that the config-only pipeline does not ingest. The perfect benchmark scorecard certifies the config-only bucket only.

- **Benchmark gate is reported, not enforced in CI.** The runner computes and exposes the metrics, but no automated pass/fail threshold is wired into a build or CI pipeline; acceptance is currently a qualitative human gate against Volume 7 Table 7.
- **Topology depicts logical intent, not traffic.** The topology view has been rebuilt as an interactive Cytoscape.js graph (draggable nodes with a layout persisted to `localStorage`, zoom/pan, a live firmware-CVE badge, and PNG export). By design it depicts the *configured* logical relationships derived from policy and asset context, not runtime packet flow, NetFlow, or sessions — a deliberate architectural boundary rather than a limitation of the renderer.
- **Cosmetic login.** The frontend retains a cosmetic lab login gate; the backend exposes no authentication route in v1, so the login does not enforce real access control and is an intentional placeholder.
- **FortiGate-VM constraints.** The unlicensed FortiGate VM imposes a ten-policy cap, which forced trade-offs in the deployed FortiGate set (a disabled-rule case and an object-sprawl case could not both be fitted under the cap), so `object_sprawl` is implemented but not exercised by this small lab set; `disabled_rule_review` *is* exercised, on the Palo Alto side, via `PA_DISABLED_RISKY`. The same VM's admin HTTPS listener could not negotiate a usable TLS cipher, requiring a lab-only, explicitly-allow-listed HTTP escape hatch (`FIREWALL_INSECURE_HTTP_HOSTS`); production remains HTTPS-only.
- **Lab, not production.** All live results are from a parallel two-device VMware lab on a thin, purpose-built policy set. The numbers demonstrate correctness on a controlled ground truth; they are not a statement about behaviour or scale on a large production estate, and the dark-web LTI subsystem remains optional and off by default.

A notable diagnostic outcome, reported here as evidence of the strategy's value, is the FortiGate ALL-service defect: an early live run revealed that FortiGate `shadowing/redundancy` did not fire while the identical Palo Alto designs did. Investigation traced this to FortiGate's predefined ALL service normalizing to `unknown:any` rather than the canonical ANY-service sentinel, so superset-based detectors missed it while the overlap-based `conflict` detector masked the bug by still firing. The fix — mapping ALL/`any` to the canonical ANY service in normalization — raised the run's findings from 38 to 46 and was permanently captured as a normalization regression test. This is precisely the layered strategy working as intended: a defect surfaced by live validation, root-caused, fixed, and locked down at the unit-test layer so it can never silently recur.

Relevant source files: `backend/services/benchmark/scorer.py`, `backend/services/benchmark/ground_truth.py`, `backend/services/benchmark/runner.py`, `backend/tests/test_*.py`, `frontend/src/benchmark.tsx`, `frontend/src/risk.tsx`.

# Chapter 5

## Conclusion and Future Work

This final chapter draws the project to a close. Section 5.1 consolidates what LUMINAFPM achieved against the objectives that motivated it, summarising the read-only multi-vendor pipeline, the deterministic set-theoretic detection engine, and the empirical validation obtained on the live laboratory. Section 5.2 then sets out a concrete, grounded roadmap of enhancements that follow naturally from the v1 architecture, distinguishing engineering hardening from genuine capability extensions.

### 5.1 Conclusion

The goal of this project was to design and build a vendor-neutral platform that could read firewall policy from heterogeneous devices, reason about that policy in a single unified model, and surface the latent anomalies — shadowing, redundancy, correlation, generalisation, and cross-device inconsistency — that accumulate silently in production rule bases and that human operators are poorly equipped to find by manual inspection. LUMINAFPM delivers that platform as a working, integrated system rather than a paper design.

#### 5.1.1 What was achieved

The central engineering claim of this work is realised end to end. LUMINAFPM acquires firewall configuration from two distinct vendors — Fortinet FortiGate over the FortiOS REST/JSON API and Palo Alto Networks PAN-OS over the XML-API — through a strictly *read-only* connector layer (`ConnectorInterface`, with `FortiGateConnector` and `PaloAltoConnector` implementations) that issues only `GET`-equivalent retrieval calls and never writes, pushes, or commits to a device. This read-only guarantee is not merely a policy statement: it is enforced structurally in the connector contract and is preserved through every downstream layer, so the platform can be deployed against live production firewalls without any risk of altering their posture.

Acquired policy from these dissimilar vendors is then *normalised* into a single canonical model persisted in PostgreSQL, which serves as the system’s source of truth. The normalisation layer correlates heterogeneous network and service objects across vendors — recording each correspondence in `object_normalization_mapping` with an explicit `match_method` (for example `exact_name_value`, `exact_value`, or `name_similarity`) and

a numeric confidence — so that a FortiGate address object and an equivalent PAN-OS address object are reasoned about as one logical entity. This normalisation is the foundation that makes genuine cross-vendor analysis possible, including the detection of cross-device inconsistencies that no single-vendor tool can express.

On top of the normalised model runs the project’s analytical core: a *deterministic* anomaly-detection engine. The engine is best understood through the set-theory framing developed in the earlier chapters — each rule is treated as a tuple of sets over source, destination, service, and action, with shadowing expressed as a subset/superset relation, redundancy as set equivalence, and conflict as overlapping match with differing action — but its implementation is a transparent, rule-based engine over the unified model, not a machine-learning classifier. Determinism is a deliberate and load-bearing design decision: every finding is reproducible, explainable, and traceable to the exact pair of rules and the exact set relation that produced it. In v1 this engine ships its “config-only core” — fourteen detectors in `ALL_DETECTORS` emitting fourteen distinct `anomaly_type` values — which together with the CTI-derived `threat_exposure` type yields fifteen persisted anomaly classes drawn from the broader forty-six-category taxonomy.

Around this deterministic spine the platform layers three further capabilities that turn raw findings into operator-actionable intelligence. A deterministic *risk-scoring* subsystem (`RISK_VERSION 1.0.0`) assigns each finding and each device a bounded 0–100 score from a fixed, auditable formula combining anomaly severity, exposure, asset sensitivity, security posture, logging, cross-vendor weight, CTI signal, and lifecycle factors, and assigns tier bands so that operators can triage by priority. An API-based *cyber threat intelligence* layer correlates policy exposure against external indicators. Finally, an *AI-assisted reporting* layer generates SOC-oriented and executive reports through an LLM provider abstraction (`GeminiProvider` as the documented default, with a deterministic, network-free `OfflineProvider` fallback). Crucially, the LLM is never an anomaly authority: it “explains, never detects.” Every generated report is *evidence-grounded*, carrying explicit `evidence_refs` that point back to the database identifiers of the deterministic findings it describes, so that no claim in a report exists without a verifiable, machine-checked basis in the engine’s output. This separation — deterministic engine as the source of truth, LLM as a bounded narrator — is what makes LUMINAFPM’s AI output trustworthy in a security context.

Beyond the originally scoped plan, the delivered system carries several further capabilities that proved valuable in practice. The threat-intelligence layer gained a second, *firmware-CVE device-vulnerability* axis: a device’s reported firmware version is correlated against a curated CVE reference (including CISA-KEV-listed entries), raising a device-scope risk modifier without emitting any rule-level findings, so the vulnerability signal sharpens device prioritisation while leaving the benchmarked detection corpus untouched. The reporting layer is underpinned by a *deterministic, structured report builder* — pure database assembly that produces the prioritised-remediation, risk-posture, firmware-vulnerability, and per-finding-evidence tables independently of the LLM — with the assembled report exportable as GitHub-flavoured Markdown and as a browser-rendered

PDF. Finally, the platform ships a functional *Settings console* that exposes device management, scheduled (Celery Beat) acquisition, detection, and threat-intelligence jobs, and in-app notifications emitted from the pipeline, turning the system from a one-shot analysis tool into a continuously operable platform.



**Figure 5.1.** *Summary of the principal achievements delivered by LuminaFPM.*

### 5.1.2 Empirical validation

The system’s correctness was not asserted but measured. A dedicated benchmark subsystem (`services/benchmark/` comprising `ground_truth.py`, `scorer.py`, and `runner.py`, surfaced through `/api/v1/benchmark`) scores the live engine output against a hand-curated ground-truth matrix at *(rule, anomaly\_type)* granularity, computing true positives, false positives, false negatives, and the standard information-retrieval metrics precision =  $TP/(TP + FP)$ , recall =  $TP/(TP + FN)$ , and  $F_1 = 2PR/(P + R)$ , together with a severity-match rate.

Run against the parallel VMware laboratory — a live FortiGate (192.168.55.10) and a live Palo Alto (192.168.55.20) on the shared 192.168.55.0/24 network — the platform read-synced both devices, executed an all-scope detection run, and analysed **26 active rules across 2 devices** (10 on FortiGate and 16 on Palo Alto). Scored against the ground truth, the deterministic engine achieved **precision, recall, and  $F_1$  all equal to 1.0**, with **43 true positives, zero false positives, and zero false negatives** (43 of 43 expected cases matched) and a **severity-match rate of 1.0**. This is the project’s headline result: on the validated laboratory corpus the engine detected every anomaly it was expected to detect, assigned the correct severity in every case, and reported no spurious findings, confirming both the soundness of the set-theoretic model and the fidelity of its deterministic implementation. Because the benchmark is wired as a first-class, re-runnable acceptance gate in both the API and the user interface (the Benchmark Center), this result functions as a standing regression guard rather than a one-off measurement.

In summary, LUMINAFPM meets its objectives: it is a strictly read-only, multi-vendor firewall anomaly-management platform whose normalised model and deterministic, explainable engine were validated at perfect precision, recall, and  $F_1$  on a live two-vendor laboratory, and whose risk-scoring, threat-intelligence, and evidence-grounded AI-reporting layers translate findings into intelligence a security operations team can act on with confidence.

## 5.2 Future Work

The v1 system is complete and validated within its declared scope, and that scope was deliberately bounded to guarantee a working, defensible deliverable. The following enhancements are the natural next steps; they are grouped into capability extensions, security/operational hardening, and platform breadth. Each is grounded in a concrete gap or seam already present in the codebase.

### 5.2.1 Interactive logical topology on Cytoscape.js

The topology view has been rebuilt as a true interactive graph on Cytoscape.js, replacing the earlier hand-rolled static SVG layout. It renders the same real platform data — firewalls, zones, monitored assets, and CTI threat vectors — now as draggable nodes whose positions persist across sessions (in browser local storage), with zoom and pan, a live firmware-CVE badge sourced from each device’s acquired firmware version, and PNG export of the current view. The graph continues to depict *logical policy intent* — the relationships configured in policy — and never live packet forwarding, NetFlow, sessions, or deep packet inspection; that distinction is architectural and remains explicit in the user interface. The residual work is therefore polish rather than rebuild: wiring an automatic directed layout (Dagre) as an alternative to the persisted manual arrangement, and adding expand/collapse with focus traversal over larger estates.

### 5.2.2 Real authentication and RBAC enforcement

The security model specifies five read-centric roles, and a demonstration security-key login exists at the UI layer, but the API route handlers themselves currently carry no authorisation dependency — `get_db_session` is the only **Depends** on the reviewed routers, so per-endpoint authorization is not yet enforced in code. A priority hardening task is therefore to implement real authentication (session or token based) and to wire genuine role-based access control as a route-level dependency across every endpoint, so that the documented five-role model is enforced rather than merely described. This is a prerequisite for any multi-tenant or production deployment.

### 5.2.3 Broadening the deterministic engine to the full taxonomy

v1 implements the fourteen-detector config-only core out of the specified forty-six-category taxonomy. A substantial body of future work is to implement the remaining specified-but-unimplemented categories — for example ordering anomalies, temporary-rule decay, poor-naming hygiene, zone mismatch, NAT risk, and asymmetric-rule detection. A particularly valuable subset is the *conditional* and *telemetry/history-based* categories, which depend on signals beyond static configuration. Extending the engine to these would require carefully introduced, optional data sources while preserving the determinism and

full explainability that are the engine’s defining strengths: any history-aware detector must still emit reproducible findings traceable to concrete evidence.

#### 5.2.4 Activating the threat-intelligence and dark-web module

The cyber threat-intelligence layer is delivered and active by default across two axes. The *indicator* axis correlates policy exposure against external indicators of compromise — shipping an offline indicator set so it operates without network access, with an optional keyed online provider — and the *firmware-CVE* axis correlates each device’s firmware version against a curated CVE reference, both feeding the risk score under deterministic control. What remains genuinely future is the dark-web / OSINT subsystem (the LTI/Robin curation layer and its richer LLM model-routing): it exists in the codebase but is off by default and treated as optional. Maturing it — enabling its curated clear-net and dark-web feeds, validating their correlation against the deterministic findings, and integrating that output into the risk score and reports under explicit operator control — would extend LUMINAFPM from policy-posture analysis toward proactive, externally-informed exposure assessment. As with the LLM reporter, any such integration must keep the deterministic engine as the authority and the threat-intel layer as an evidence-grounded enrichment, never an unverifiable signal.

#### 5.2.5 Additional vendors

v1 supports FortiGate and Palo Alto only; Cisco was explicitly de-scoped (ADR-004), and the connector registry contains no Cisco connector. Because the architecture cleanly separates vendor-specific acquisition (the `ConnectorInterface` contract) from the vendor-neutral normalised model and engine, adding a vendor is principally a matter of writing a new read-only connector and its normalisation mapping. Cisco (for example ASA/Firepower) is the obvious first candidate for a future release; the platform’s normalisation and detection layers would absorb a third vendor without modification, and would immediately gain the ability to detect Cisco-versus-Fortinet and Cisco-versus-Palo-Alto cross-device inconsistencies. (Any residual Cisco/ASA strings remaining in demo seed data and UI styling should be removed until a real connector exists, to avoid implying unsupported capability.)

#### 5.2.6 Report export to PDF

Direct export of the evidence-grounded reports has been delivered. The platform now exports each generated SOC and executive report both as GitHub-flavoured Markdown (served directly from the API) and as a self-contained PDF produced through the browser’s print pipeline, yielding a distributable artefact suitable for audit trails, management reporting, and compliance evidence. The underlying `evidence_refs` linkage is preserved through the export, so every exported claim remains traceable to the deterministic finding that produced it. The residual future work is purely presentational: a richer, server-side

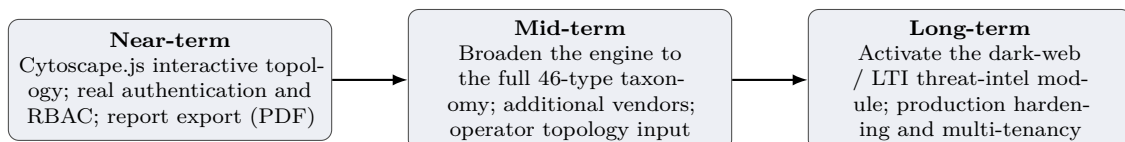
native PDF renderer with bespoke typographic styling, should fully controlled print layout become a requirement.

### 5.2.7 Operator-supplied topology and asset context

In v1 the platform derives logical relationships, zones, and monitored assets purely from *configured intent* in the acquired policy; there is no ingestion path for an external or observed topology. A valuable enhancement is an explicit operator-supplied topology/asset input — allowing an operator to declare zones, asset sensitivity, and business context — which would sharpen risk scoring (asset sensitivity in particular) and enrich the logical graph. This would remain a description of intended/declared structure, consistent with the platform’s principle of reasoning over configured posture rather than observed traffic.

### 5.2.8 Production hardening

Finally, several engineering tasks separate the validated laboratory build from a production deployment: removing declared-but-unused frontend dependencies and dead vendor code paths; pinning and auditing dependency versions; tightening secret and encryption-key management; expanding automated test and CI coverage around the connectors and engine; production-grade observability and structured logging; and hardening the Docker Compose deployment for resilient, secured operation. None of these alters the core architecture; together they raise the system from a demonstrably-correct prototype to a deployable product.



**Figure 5.2.** *Future-work roadmap for LuminaFPM.*

Taken together, these directions extend LUMINAFPM along three axes — deeper analysis (full taxonomy, threat-intel), broader coverage (more vendors, richer topology input), and production readiness (RBAC, hardening, PDF export) — while preserving the two principles that define the platform: it remains strictly read-only with respect to the firewalls it analyses, and its findings remain deterministic, explainable, and evidence-grounded.

# References

- [1] E. S. Al-Shaer and H. H. Hamed, “Discovery of policy anomalies in distributed firewalls,” in *Proc. IEEE INFOCOM 2004*, vol. 4, Hong Kong, 2004, pp. 2605–2616.
- [2] E. S. Al-Shaer, H. H. Hamed, R. Boutaba, and M. Hasan, “Conflict classification and analysis of distributed firewall policies,” *IEEE Journal on Selected Areas in Communications*, vol. 23, no. 10, pp. 2069–2084, Oct. 2005.
- [3] H. Hamed and E. Al-Shaer, “Taxonomy of conflicts in network security policies,” *IEEE Communications Magazine*, vol. 44, no. 3, pp. 134–141, Mar. 2006.
- [4] L. Yuan, J. Mai, Z. Su, H. Chen, C.-N. Chuah, and P. Mohapatra, “FIREMAN: A toolkit for firewall modeling and analysis,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, Oakland, CA, USA, 2006, pp. 199–213.
- [5] A. Wool, “A quantitative study of firewall configuration errors,” *IEEE Computer*, vol. 37, no. 6, pp. 62–67, Jun. 2004.
- [6] M. G. Gouda and A. X. Liu, “Structured firewall design,” *Computer Networks*, vol. 51, no. 4, pp. 1106–1120, Mar. 2007.
- [7] Fortinet, Inc., *FortiOS REST API Reference and Administration Guide*. [Online]. Available: <https://docs.fortinet.com/document/fortigate/>. [Accessed: Jun. 2026].
- [8] Palo Alto Networks, Inc., *PAN-OS XML API Usage Guide*. [Online]. Available: <https://docs.paloaltonetworks.com/pan-os/pan-os-panorama-api>. [Accessed: Jun. 2026].
- [9] Palo Alto Networks, Inc., *Panorama Administrator’s Guide: Centralized Firewall Policy Management*. [Online]. Available: <https://docs.paloaltonetworks.com/panorama>. [Accessed: Jun. 2026].
- [10] Tufin Software Technologies, *Tufin Orchestration Suite: Firewall Policy and Security Change Automation*. [Online]. Available: <https://www.tufin.com/>. [Accessed: Jun. 2026].
- [11] AlgoSec, Inc., *AlgoSec Security Management Solution: Firewall Policy Analysis and Optimization*. [Online]. Available: <https://www.algosec.com/>. [Accessed: Jun. 2026].
- [12] FireMon, LLC, *FireMon Security Manager: Firewall Rule Analysis and Compliance*. [Online]. Available: <https://www.firemon.com/>. [Accessed: Jun. 2026].

- [13] Skybox Security, Inc., *Skybox Security Posture Management Platform: Firewall Assurance*. [Online]. Available: <https://www.skyboxsecurity.com/>. [Accessed: Jun. 2026].
- [14] National Institute of Standards and Technology (NIST), *National Vulnerability Database (NVD)*. [Online]. Available: <https://nvd.nist.gov/>. [Accessed: Jun. 2026].
- [15] Cybersecurity and Infrastructure Security Agency (CISA), *Known Exploited Vulnerabilities (KEV) Catalog*. [Online]. Available: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>. [Accessed: Jun. 2026].
- [16] The MITRE Corporation, *MITRE ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge*. [Online]. Available: <https://attack.mitre.org/>. [Accessed: Jun. 2026].
- [17] AbuseIPDB, *AbuseIPDB IP Address Reputation and Abuse Reporting API (v2)*. [Online]. Available: <https://www.abuseipdb.com/>. [Accessed: Jun. 2026].
- [18] S. Ramírez, *FastAPI: Modern, Fast (High-Performance) Web Framework for Building APIs with Python*. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Jun. 2026].
- [19] M. Bayer, *SQLAlchemy: The Python SQL Toolkit and Object Relational Mapper*. [Online]. Available: <https://www.sqlalchemy.org/>. [Accessed: Jun. 2026].
- [20] A. Solem and the Celery Team, *Celery: Distributed Task Queue*. [Online]. Available: <https://docs.celeryq.dev/>. [Accessed: Jun. 2026].
- [21] The PostgreSQL Global Development Group, *PostgreSQL: The World's Most Advanced Open Source Relational Database*. [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: Jun. 2026].
- [22] Meta Open Source, *React: A JavaScript Library for Building User Interfaces*. [Online]. Available: <https://react.dev/>. [Accessed: Jun. 2026].
- [23] Docker, Inc., *Docker Compose: Define and Run Multi-Container Applications*. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: Jun. 2026].
- [24] Google LLC, *Gemini API Documentation (Google Generative AI)*. [Online]. Available: <https://ai.google.dev/>. [Accessed: Jun. 2026].
- [25] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, “Address allocation for private internets,” RFC 1918, Internet Engineering Task Force (IETF), Feb. 1996.